

**DETEKSI GERAKAN PEMAIN PADA GAME FIGHTING
TEKKEN 8 MENGGUNAKAN TRACKING DAN ACTION
RECOGNITION BERBASIS SKELETON**

PLAYER ACTION DETECTION IN THE TEKKEN 8 FIGHTING GAME
USING SKELETON-BASED TRACKING AND ACTION RECOGNITION



TUGAS AKHIR

DIAJUKAN SEBAGAI SALAH SATU SYARAT UNTUK MEMPEROLEH
GELAR SARJANA KOMPUTER (S.Kom) JENJANG PENDIDIKAN STRATA-1

NIM : 2210101019

NAMA : Julian Yang

PROGRAM STUDI INFORMATIKA

UNIVERSITAS PRADITA

TANGERANG

2026

LEMBAR PERSETUJUAN TUGAS AKHIR

Nama : Julian Yang

NIM : 2210101019

Program Studi : Informatika

Bentuk Tugas Akhir : Skripsi

Peminatan Tugas Akhir : Artificial Intelligence

Judul Tugas Akhir : DETEKSI GERAKAN PEMAIN PADA GAME
FIGHTING TEKKEN 8 MENGGUNAKAN TRACKING DAN ACTION
RECOGNITION BERBASIS SKELETON

Kab. Tangerang, 1 Juli 2026

Menyetujui

Pembimbing Skripsi



Dr. Eng. Handri Santoso, S.
Si., M.Eng.



LEMBAR PERNYATAAN TIDAK PLAGIAT

Yang bertanda tangan dibawah ini:

Nama : Julian Yang

NIM : 2210101019

Program Studi : Informatika

Judul Tugas Akhir : DETEKSI GERAKAN PEMAIN PADA GAME
FIGHTING TEKKEN 8 MENGGUNAKAN TRACKING DAN ACTION
RECOGNITION BERBASIS SKELETON

Kab. Tangerang, 1 Juli 2026



Julian Yang

2210101019

LEMBAR PENGESAHAN TUGAS AKHIR

Nama: : Julian Yang
NIM : 2210101019
Program Studi : Informatika
Bentuk Tugas Akhir : Skripsi
Peminatan Tugas Akhir : Artificial Intelligence
Judul Tugas Akhir : DETEKSI GERAKAN PEMAIN PADA GAME
FIGHTING TEKKEN 8 MENGGUNAKAN TRACKING DAN ACTION
RECOGNITION BERBASIS SKELETON

Mengajukan pada tanggal **Rabu, 1 Juli 2026**

Telah disetujui oleh:

Pembimbing



Dr. Eng. Handri Santoso, S.Si.,
M.Eng

Penguji

(Nama Penguji)

Ketua Sidang,

(Nama Ketua Sidang)

Disahkan oleh:

Kepala Program Studi Informatika

Erick Dazki, S.Kom., M.Kom.

LEMBAR PERNYATAAN PERSETUJUAN PUBLIKASI UNTUK
KEPENTINGAN AKADEMIS

Dengan ini saya sebagai civitas akademik Universitas Pradita yang bertandatangan di bawah ini

Nama : Julian Yang

NIM : 2210101019

Program Studi : Informatika

Bentuk Tugas Akhir : Skripsi

untuk meningkatkan pengembangan ilmu pengetahuan, memberikan skripsi/tugas akhir kepada Universitas Pradita Hak Bebas Royalti Non-eksklusif (*Non-exclusive Royalty Free Right*) dengan judul:

DETEKSI GERAKAN PEMAIN PADA GAME FIGHTING TEKKEN 8
MENGGUNAKAN TRACKING DAN ACTION RECOGNITION
BERBASIS SKELETON

Beserta dokumen tugas akhir yang ada sesuai ketentuan yang berlaku. Dengan Hak Bebas Royalti Non-eksklusif (*Non-exclusive Royalty Free Right*) ini, maka Universitas Pradita berhak menyimpan dan mengelola dalam bentuk database, dan mempublikasikan tugas akhir ini dengan tetap mencantumkan nama saya sebagai penulis tugas akhir ini sebagai penulis/pencipta dan pemilik hak cipta. Demikianlah pernyataan ini saya buat dengan sebenarnya,

Kab. Tangerang, 1
Juli 2026



Julian Yang
2210101019

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan tugas akhir yang berjudul “DETEKSI GERAKAN PEMAIN PADA GAME FIGHTING TEKKEN 8 MENGGUNAKAN TRACKING DAN ACTION RECOGNITION BERBASIS SKELETON”. Tugas akhir ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Komputer (S.Kom.) pada Program Studi Informatika, Universitas Pradita.

Penyusunan tugas akhir ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, penulis menyampaikan terima kasih kepada:

1. Bapak Prof. Dr. Ir. Richardus Eko Indrajit, M.Sc., M.B.A., M.Phil., M.A., selaku Rektor Universitas Pradita.
2. Bapak Dr. Hendro Susanto, S.E., M.M., selaku Wakil Rektor I Bidang Akademik Universitas Pradita.
3. Bapak Dr. Eng. Handri Santoso, S.Si., M.Eng., selaku Dekan Fakultas Sains dan Teknologi sekaligus dosen pembimbing, yang telah memberikan motivasi, bimbingan, arahan, dan masukan sejak tahap awal hingga penyelesaian tugas akhir ini.
4. Bapak Erick Dazki, S.Kom., M.Kom., selaku Ketua Program Studi Informatika Universitas Pradita.
5. Seluruh dosen dan staf Program Studi Informatika Universitas Pradita yang telah memberikan ilmu, pengalaman, dan dukungan selama masa perkuliahan.
6. Orang tua dan keluarga penulis atas doa, kasih sayang, dukungan moral dan material, serta semangat yang senantiasa diberikan selama penyusunan tugas akhir ini.
7. Rekan-rekan mahasiswa seperjuangan yang telah memberikan dukungan, semangat, bantuan, dan kebersamaan sejak awal perkuliahan hingga penyelesaian tugas akhir ini.

Penulis menyadari bahwa tugas akhir ini masih memiliki keterbatasan dan kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan sebagai bahan perbaikan dan pengembangan pada penelitian selanjutnya.

Akhir kata, penulis berharap tugas akhir ini dapat memberikan manfaat bagi pembaca serta menjadi referensi bagi pengembangan penelitian computer vision dan pengenalan aksi berbasis skeleton, khususnya dalam konteks video game fighting dan penyajian informasi esports.

Kab. Tangerang, 1

Juli 2026

A handwritten signature in black ink, appearing to be 'Julian Yang', with a stylized, cursive script.

Julian Yang

2210101019

DAFTAR ISI

DAFTAR ISI	VII
DAFTAR TABEL	XI
DAFTAR GAMBAR	XII
DAFTAR LAMPIRAN	XIV
ABSTRAK	XV
ABSTRACT	XVI
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Perumusan Masalah	6
1.3 Tujuan dan Sasaran	6
1.4 Manfaat	7
1.4.1 Manfaat Subjektif	7
1.4.2 Manfaat Objektif	8
1.5 Ruang Lingkup Pembahasan	8
1.5.1 Ruang Lingkup Substansial:	8
1.5.2 Ruang Lingkup Spasial / Objek Penelitian:	9
1.6 Kerangka Penelitian	9
1.7 Sistematika Penulisan	10
BAB II TINJAUAN PUSTAKA	12
2.1 Kerangka Teoritis	12
2.1.1 Ekosistem Fighting Game dan Tekken 8 sebagai Objek Penelitian	12

2.1.2 Kompleksitas Visual dan Mekanika Invisible.....	13
2.1.3 <i>Beban Kognitif (Cognitive Load) dan Kesenjangan Pengetahuan Penonton</i>	14
2.1.4 Computer Vision sebagai Solusi Real-Time Annotation	16
2.2 Tinjauan Teoritis	18
2.2.1 Computer Vision (CV)	18
2.2.2 Human Action Recognition (HAR)	19
2.2.3 Real-Time Object Detection, Multi-Object Tracking, dan Ekstraksi Region of Interest (RoI)	20
2.2.4 Estimasi Pose 2D (2D Pose Estimation) dan Pose Candidate Selection.....	22
2.2.5 Skeleton-Based Action Recognition (Pengenalan Aksi Berbasis Kerangka).....	25
2.3 Studi Preseden	28
2.3.1 Penjelasan Diagram dan Studi Preseden	28
2.3.2 Relevansi untuk Penelitian Tekken 8	36
2.3.3 Kesimpulan Strategis	37
BAB III Metodologi Penelitian	38
3.1 Gambaran Umum Objek Penelitian	38
3.2 Teknik Pengumpulan Data	39
3.2.1 Survei Pendahuluan (Preliminary Survey).....	39
3.2.2 Studi literatur dan Eksperimen Awal	40
3.2.3 Pengumpulan Dataset dan Anotasi (Custom Annotator)	41
3.3 Analisis Sistem.....	44
3.3.1 Gambaran Sistem (System Overview)	44

3.3.2 Analisis Fungsional (Functional Analysis)	45
3.3.3 Diagram Aktivitas (Activity Diagram)	46
3.4 Analisis Masalah yang Dihadapi.....	48
3.5 Metode Pengolahan Data	49
3.6 Metode Evaluasi.....	51
BAB IV ANALISIS DAN PEMBAHASAN.....	54
4.1 Implementasi Pipeline Sistem	54
4.2 Analisis Deteksi Pemain dan Parameter YOLO	55
4.3 Analisis Tracking Identitas	57
4.4 Analisis ROI, Padding, dan Overlap	59
4.5 Analisis Pemilihan Model Pose	59
4.6 Analisis Pose Candidate Selection dan Quality Gate.....	61
4.7 Analisis Dataset, Windowing, dan Training ST-GCN++	62
4.8 Analisis Inference, Threshold, dan Runtime.....	64
4.8.1 Konfigurasi Inference dan Evaluasi Runtime	64
4.8.2 Evaluasi Ketepatan End-to-End terhadap Ground Truth	67
4.9 Pembahasan Umum.....	69
BAB V KESIMPULAN DAN SARAN.....	71
5.1 Kesimpulan	71
5.2 Keterbatasan Penelitian	73
5.3 Saran Pengembangan	74
5.4 Implikasi Penelitian.....	75
DAFTAR PUSTAKA	76

LAMPIRAN.....	82
---------------	----

DAFTAR TABEL

Tabel 3.1: Overview Pengumpulan Data Survei Multi-Platform	39
Tabel 4.1: Perbandingan Parameter Confidence dan IoU YOLO.....	56
Tabel 4.2: Perbandingan Latency Model Pose pada ROI Pair	60
Tabel 4.3: Perbandingan Eksperimen ST-GCN++	62
Tabel 4.4: Hasil Pengukuran Latency End-to-End Pipeline	64
Tabel 4.5: Hasil Evaluasi Internal End-to-End	67
Tabel 4.6: Event Recall Berdasarkan Toleransi Setelah Akhir Anotasi	69

DAFTAR GAMBAR

Gambar 1.1: Jumlah viewer tertinggi selama periode EVO Japan 2024 (Esports Charts, 2024)	1
Gambar 1.2: Perbandingan gerakan Bryan: df+2, dan SWA.2 (Wavu Wiki contributors, 2025)	3
Gambar 1.3: Post Reddit yang diinisiasi penulis	4
Gambar 1.4: Salah satu jawaban dari pemain berpengalaman	5
Gambar 1.5: Contoh hasil akhir	7
Gambar 1.6: Flowchart kerangka penelitian	9
Gambar 2.1: Jumlah viewer tertinggi per pertandingan dan keseluruhan, serta jumlah total jam ditonton selama periode Esports World Cup 2025 (Esports Charts, 2025)	12
Gambar 2.2: Hasil Polling mixed-methods	15
Gambar 2.3: Pemetaan komponen studi preseden dan pendekatan yang dipilih .	28
Gambar 3.1: Metodologi Penelitian	38
Gambar 3.2: Contoh Anotasi Json	42
Gambar 3.3: Aplikasi Custom Annotator.....	43
Gambar 3.4: Anotasi manual dataset training YOLO11m	44
Gambar 3.5: Activity Diagram Proses Inferensi Pengenalan Gerakan Tekken 8 Berbasis Skeleton	46
Gambar 4.1: Contoh deteksi pemain menggunakan bounding box	54
Gambar 4.2: Contoh ROI crop dan pose estimation pada karakter target.....	55
Gambar 4.3: Hasil training YOLOv11m	56
Gambar 4.4: Perbandingan visual parameter YOLO conf=0,35/iou=0,35 dan conf=0,5/iou=0,35	57
Gambar 4.5: Contoh kegagalan ekstraksi skeleton pada MoveNet	60

Gambar 4.6: Contoh skeleton YOLO-Pose pada sequence uji	61
Gambar 4.7: Visualisasi end-to-end latency, pipeline FPS, target FPS, dan perangkat pengujian pada video inference	65
Gambar 4.8: Contoh output overlay anotasi pada video inference	66

DAFTAR LAMPIRAN

Lampiran 1 Custom Anotator untuk tekken (anotator_5.py):	82
Lampiran 2 Script pembuatan dataset dan training model YOLO detector (fighter_dataset_creation.ipynb):	100
Lampiran 3 Konfigurasi dataset YOLO (fighter_dataset/data.yaml):	109
Lampiran 4 Pipeline lengkap object detection, tracking, ROI cropping, pose estimation, penyimpanan data ke dalam bentuk .npy, hingga training STGCN++ (Tekken_recognition_yoloPose_final_cleaned.ipynb):	110
Lampiran 5 Membentuk dataset STGCN++ (make_all_moves_idle_dataset.py):	129
Lampiran 6 Konfigurasi training STGCN++ (tekken_stgcn_all_moves_idle.py):	143
Lampiran 7 Script end-to-end Inference beserta evaluasi (tekken_video_inference.py):.....	148
Lampiran 8 Benchmark untuk pose model (benchmark_yolopose_runtime.py):	197
Lampiran 9 Requirements.txt :.....	217
Lampiran 10 Semua code penting di luar pipeline utama (cv_utility_code.ipynb) :	219
Lampiran 11 Kuesioner google form untuk survey:	242
Lampiran 12 Kuesioner reddit r/Tekken:	254

ABSTRAK

Tekken 8 memiliki animasi yang cepat dan kompleks sehingga penonton, khususnya pemain pemula, sering mengalami kesulitan dalam mengidentifikasi gerakan yang dilakukan oleh karakter. Penelitian ini bertujuan merancang prototipe sistem anotasi gerakan Tekken 8 berbasis *skeleton-based action recognition* yang mampu mengenali 16 gerakan terpilih karakter Bryan Fury melalui input video gameplay tanpa mengakses memori internal atau melakukan modifikasi terhadap game. Sistem dikembangkan menggunakan pipeline *visual-only* yang terdiri atas YOLO11m untuk mendeteksi pemain, OC-SORT untuk mempertahankan identitas pemain, ekstraksi *region of interest*, YOLO11l-Pose untuk menghasilkan 17 keypoint COCO, serta ST-GCN++ untuk mengklasifikasikan urutan skeleton menjadi label gerakan. Video kerja diproses pada 29,97 FPS. Dataset skeleton akhir terdiri atas 510 window berdurasi 30 frame yang terbagi menjadi 17 kelas, yaitu 16 kelas gerakan dan satu kelas *idle*, dengan 408 window training dan 102 window validation. Pada evaluasi internal, model ST-GCN++ setelah koreksi anotasi memperoleh akurasi Top-1 sebesar 78,43% dan Top-5 sebesar 100,00%. Evaluasi internal end-to-end pada split validation menghasilkan strict Top-1 sebesar 73,53% dan event-level Top-1 sebesar 78,13%. Pengujian runtime end-to-end menghasilkan mean latency sebesar 80,89 ms atau throughput 12,36 FPS, dibandingkan target video masukan sebesar 29,97 FPS. Hasil tersebut menunjukkan bahwa pendekatan berbasis skeleton dapat digunakan untuk membangun prototipe anotasi gerakan Tekken 8 secara visual. Namun, sistem belum memenuhi target real-time penuh dan masih memerlukan optimasi pada tahap deteksi dan estimasi pose serta pengujian pada sesi rekaman yang berbeda untuk menilai kemampuan generalisasinya.

Kata kunci: Tekken 8, pengenalan aksi, skeleton, estimasi pose, ST-GCN++.

ABSTRACT

Tekken 8 features fast and visually complex animations, making it difficult for viewers, particularly novice players, to identify the moves performed by each character. This study aims to develop a prototype skeleton-based action recognition system for annotating 16 selected moves of Bryan Fury from gameplay video without accessing the game’s internal memory or modifying the game. A visual-only pipeline was developed using YOLO11m for player detection, OC-SORT for maintaining player identities, region-of-interest extraction, YOLO11l-Pose for generating 17 COCO keypoints, and ST-GCN++ for classifying skeleton sequences into move labels. The working video was processed at 29.97 FPS. The final skeleton dataset consisted of 510 30-frame windows across 17 classes, comprising 16 move classes and one idle class, with 408 training windows and 102 validation windows. In the internal evaluation, the ST-GCN++ model achieved 78.43% Top-1 accuracy and 100.00% Top-5 accuracy after an annotation error was corrected. Internal end-to-end evaluation on the validation split yielded 73.53% strict Top-1 accuracy and 78.13% event-level Top-1 accuracy. End-to-end runtime evaluation produced a mean latency of 80.89 ms, corresponding to a throughput of 12.36 FPS, compared with the input-video target of 29.97 FPS. These results demonstrate the feasibility of using skeleton-based action recognition to build a visual prototype for annotating Tekken 8 moves. However, the system has not yet achieved full real-time performance and requires further optimization of the detection and pose-estimation stages, as well as evaluation on independent recording sessions to assess its generalization capability.

Keywords: Tekken 8, action recognition, skeleton, pose estimation, ST-GCN++.

BAB I PENDAHULUAN

1.1 Latar Belakang

Industri esports dan video game telah berkembang menjadi fenomena global dengan daya tarik lintas generasi. Salah satu genre tertua dan paling kompetitif dalam ekosistem ini adalah fighting game, yang terus menunjukkan pertumbuhan signifikan dari sisi viewership turnamen internasional. Evolution Championship Series (EVO) menunjukkan lonjakan drastis pada jumlah penonton. Pada EVO Japan 2024, Tekken 8 mencatat 168.689 peak viewers, sebuah peningkatan tajam sebesar 68% dibandingkan Tekken 7 pada tahun 2023 yang hanya meraih 100.500 penonton (Esports Charts, 2023, 2024). Momentum pertumbuhan ini berlanjut pada Esports World Cup 2025, di mana Tekken mencetak rekor franchise dengan total 1,35 juta jam tonton dan puncak hampir 90.000 penonton bersamaan (Esports Charts, 2025). Data ini menegaskan bahwa Tekken 8 bukan sekadar permainan kompetitif, melainkan produk tontonan massal dengan basis audiens yang terus berkembang.



Gambar 1.1: Jumlah viewer tertinggi selama periode EVO Japan 2024 (Esports Charts, 2024)

Namun, di balik popularitas yang tinggi ini, Tekken 8 menghadapi tantangan aksesibilitas informasi yang unik dan kompleks. Sebagai fighting game tiga dimensi, Tekken 8 beroperasi sebagai sistem deterministik berkecepatan tinggi di mana logika permainan diproses secara ketat pada 60 frame per detik (FPS). Dalam ekosistem komputasi ini, setiap aksi karakter didefinisikan oleh properti matematis yang presisi, meliputi frame data, hitbox, dan hurtbox (Demetrio, 2021). Bagi mesin komputer, parameter ini adalah data absolut yang jelas. Namun, bagi pengamat manusia, baik pemain pemula maupun penonton esports, informasi teknis ini sering kali menjadi "invisible mechanics" (mekanika tak terlihat) karena tertutup oleh lapisan visual grafis yang kompleks dan kecepatan animasi yang tinggi.

Permasalahan mendasar muncul karena antarmuka pengguna (*User Interface*) bawaan game tidak menampilkan data input atau properti gerakan secara *real-time* saat pertandingan berlangsung. Hal ini menciptakan fenomena '**Ambiguitas Visual**', di mana dua gerakan berbeda memiliki pose akhir (*impact frame*) yang sangat mirip, sehingga membingungkan persepsi penonton.

Sebagai contoh studi kasus pada karakter Bryan Fury, gerakan *Blackout Combo* (**df+2**) dan *Smash* (**SWA.2**) memiliki karakteristik visual pukulan yang sangat mirip. Perbedaan utama keduanya hanya terletak pada **pola temporal**, di mana **SWA.2** didahului oleh animasi langkah mundur (*sway*) yang berdurasi singkat. Dalam kecepatan permainan 60 FPS, mata manusia sering kali gagal menangkap transisi *sway* tersebut, membuat kedua gerakan ini tampak tidak dapat dibedakan. Padahal, keduanya memiliki properti strategis dan input eksekusi yang berbeda. Ketidadaan anotasi visual menyebabkan penonton kehilangan konteks mengenai strategi serangan yang sedang dibangun oleh pemain.



Gambar 1.2: Perbandingan gerakan Bryan: df+2, dan SWA.2 (Wavu Wiki contributors, 2025)

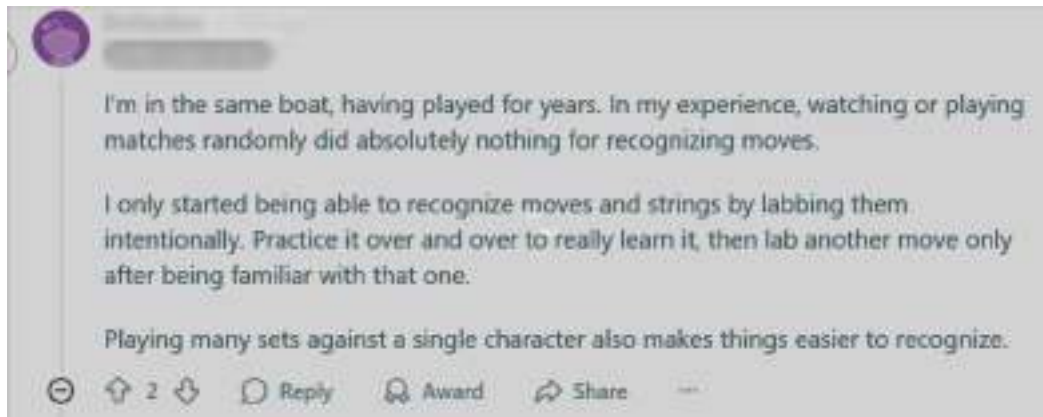
Kesenjangan antara popularitas yang tinggi dan kompleksitas teknis ini menciptakan paradoks dalam ekosistem Tekken 8. Sebuah analisis mengenai hambatan masuk dalam fighting game menempatkan *knowledge checks* sebagai salah satu kendala utama, yaitu situasi ketika penonton tidak memahami mengapa suatu interaksi terjadi karena tidak tersedia indikator visual yang memadai (Ma, 2024). Tanpa bantuan sistem visual, penonton dituntut untuk menghafal ratusan properti gerakan melalui latihan intensif di luar konteks menonton pertandingan.

Kesulitan dalam memproses informasi visual berkecepatan tinggi ini bukan sekadar asumsi teoretis, melainkan masalah nyata yang dialami komunitas. Berdasarkan survei multi-platform yang penulis lakukan terhadap komunitas pemain Tekken (Tabel 3.1, N=43), 69.8% responden melaporkan kesulitan mengidentifikasi gerakan spesifik saat menonton pertandingan tingkat tinggi. Data ini konsisten di berbagai platform pengumpulan (Google Form: 71.4%, Reddit Poll: 68.2%), mengindikasikan bahwa masalah ini bersifat sistemik dan tidak bergantung pada platform atau demografi responden. Temuan kuantitatif ini diperkuat oleh data kualitatif dari thread diskusi di forum r/Tekken yang penulis inisiasi.



Gambar 1.3: Post Reddit yang diinisiasi penulis

Dalam diskusi tersebut, setiap responden menyampaikan pengalaman pribadi yang menyoroti aspek berbeda dari kesulitan mengenali gerakan spesifik. Seorang pengguna menekankan bahwa kemampuan identifikasi gerakan hanya meningkat setelah latihan intensif melalui mode practice (labbing), dengan pernyataan tegas: *"watching matches randomly did absolutely nothing for recognizing moves"*. Sementara itu, pengguna lain menyoroti bahwa pemain profesional memiliki kemampuan menyamarkan gerakan melalui pergerakan yang sangat halus, sering kali membuat gerakan tertentu "tidak terlihat jelas" bagi penonton maupun pemain kasual. Pengguna tersebut menekankan bahwa *"pros are very good at hiding moves with excellent movement"*, menjelaskan mengapa tingkat permainan profesional berbeda secara signifikan dibandingkan bahkan dengan pertandingan di peringkat tinggi.



Gambar 1.4: Salah satu jawaban dari pemain berpengalaman

Kondisi ini dapat dijelaskan sebagai peningkatan beban kognitif, karena pengamat harus memproses informasi visual yang bergerak cepat sekaligus menghubungkannya dengan informasi semantik yang tidak ditampilkan secara eksplisit, seperti input command, frame property, dan konteks situasional gerakan. Berdasarkan Cognitive Load Theory, informasi baru harus diproses melalui working memory yang memiliki kapasitas dan durasi terbatas (Sweller et al., 2019). Dalam konteks Tekken 8, keterbatasan ini dapat menyebabkan kesenjangan pemahaman ketika penonton melihat output visual seperti hit, launch, combo, atau wall splat, tetapi tidak memahami penyebab teknis di baliknya. Oleh karena itu, sistem anotasi real-time diposisikan sebagai bantuan visual yang menerjemahkan aksi menjadi label semantik agar proses identifikasi gerakan tidak sepenuhnya bergantung pada hafalan dan interpretasi manual.

Oleh karena itu, pendekatan berbasis Computer Vision (CV) menjadi solusi teknis yang relevan untuk mengatasi masalah kognitif ini. Dengan memanfaatkan metode Skeleton-Based Action Recognition, sistem komputer dapat dilatih untuk mengenali pola koordinat sendi karakter dan mengklasifikasikannya menjadi label gerakan yang dapat dibaca manusia. Pada penelitian ini, pose estimation digunakan sebagai tahap ekstraksi fitur visual, sedangkan ST-GCN++ digunakan sebagai model pengenalan aksi berbasis skeleton karena mampu memodelkan relasi spasial antar keypoint dan perubahan temporal gerakan.

Hingga saat ini belum ditemukan implementasi publik maupun penelitian akademik yang mampu mengenali moveset Tekken secara langsung dari input visual. Tools seperti Tekken Bot Prime (GitHub - WAZAAAAA0/TekkenBot) memerlukan akses *internal memory* game (RAM hooking), bukan computer vision. Dengan demikian, pendekatan yang diusulkan dalam penelitian ini bukan hanya alternatif, tetapi membuka jalur baru yang independen dari game modification dan

dapat diaplikasikan pada konteks streaming atau siaran esports tanpa memerlukan akses ke sistem internal game.

Penelitian ini bertujuan untuk merancang sistem anotasi moveset berbasis visual yang dapat menjembatani kesenjangan antara tampilan grafis game dan pemahaman teknis pengamat. Dengan menghadirkan overlay informasi gerakan, sistem ini diharapkan dapat membantu penonton memahami aksi yang terjadi pada pertandingan Tekken 8 secara lebih informatif, khususnya bagi audiens yang belum familier dengan istilah teknis moveset pada fighting game.

Berdasarkan kompleksitas teknis tersebut, penelitian ini tidak diarahkan untuk langsung mengenali seluruh moveset Tekken 8 dalam semua kondisi pertandingan. Penelitian ini difokuskan sebagai prototipe sistem anotasi visual berbasis skeleton untuk mengenali 16 gerakan terpilih dari karakter Bryan Fury. Pembatasan ini dilakukan agar proses pengumpulan data, anotasi, pelatihan model, dan evaluasi sistem dapat dilakukan secara terukur, sekaligus tetap merepresentasikan tantangan utama pada game fighting, yaitu gerakan cepat, close contact, overlap karakter, oklusi, dan perubahan pose yang dinamis.

1.2 Perumusan Masalah

Berdasarkan uraian tersebut, permasalahan yang akan diselesaikan dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana merancang pipeline skeleton-based action recognition untuk mengenali 16 gerakan terpilih karakter Bryan Fury pada video gameplay Tekken 8?
2. Bagaimana mengintegrasikan object detection, tracking, ROI extraction, dan pose estimation agar skeleton yang diproses tetap konsisten terhadap karakter target?
3. Bagaimana kinerja sistem dalam mengenali gerakan karakter berdasarkan kualitas deteksi pemain, stabilitas tracking, kualitas keypoint, akurasi klasifikasi gerakan, dan latency inference?

1.3 Tujuan dan Sasaran

Penelitian ini bertujuan untuk merancang dan mengimplementasikan prototipe sistem anotasi gerakan Tekken 8 berbasis skeleton-based action recognition yang mampu mengenali 16 gerakan terpilih karakter Bryan Fury dari input video gameplay. Sistem dirancang menggunakan pipeline visual-only yang

terdiri dari deteksi pemain, pelacakan identitas, ekstraksi ROI, estimasi pose, dan klasifikasi gerakan menggunakan ST-GCN++.

1. Mengembangkan pipeline computer vision untuk mendeteksi dan melacak karakter pada video gameplay Tekken 8.
2. Mengekstraksi skeleton atau keypoint karakter target menggunakan pose estimation berbasis ROI.
3. Menyusun dataset gerakan Bryan Fury yang terdiri dari anotasi frame awal, frame akhir, label gerakan, serta idle class.
4. Melatih model ST-GCN++ untuk mengklasifikasikan 16 selected moves karakter Bryan Fury dan idle class.
5. Mengevaluasi sistem berdasarkan kualitas detection, tracking, pose estimation, classification, dan latency inference.



Gambar 1.5: Contoh hasil akhir

1.4 Manfaat

1.4.1 Manfaat Subjektif

1. Penulis dapat mengasah kemampuan dalam bidang computer vision, deep learning, dan integrasi model pose estimation dengan action recognition.

2. Penulis dapat berkontribusi secara akademik dalam penelitian yang menggabungkan teknologi kecerdasan buatan dengan konteks hiburan interaktif (fighting game).
3. Melalui proyek ini, penulis memperoleh pemahaman lebih dalam mengenai desain sistem real-time AI dan penerapannya dalam industri kreatif.

1.4.2 Manfaat Objektif

1. Membantu pemain baru memahami mekanisme dasar fighting game melalui anotasi moveset otomatis, sehingga dapat menurunkan learning curve.
2. Memberikan nilai tambah bagi penonton esports dengan menghadirkan tayangan yang lebih informatif, edukatif, dan inklusif.
3. Menjadi acuan bagi pengembang game atau penyelenggara turnamen dalam mengadopsi teknologi anotasi real-time untuk meningkatkan kualitas siaran dan memperluas basis audiens.
4. Menyediakan kerangka implementasi sistem yang dapat diadaptasi untuk genre game lain yang juga memiliki kompleksitas tinggi.

1.5 Ruang Lingkup Pembahasan

1.5.1 Ruang Lingkup Substansial:

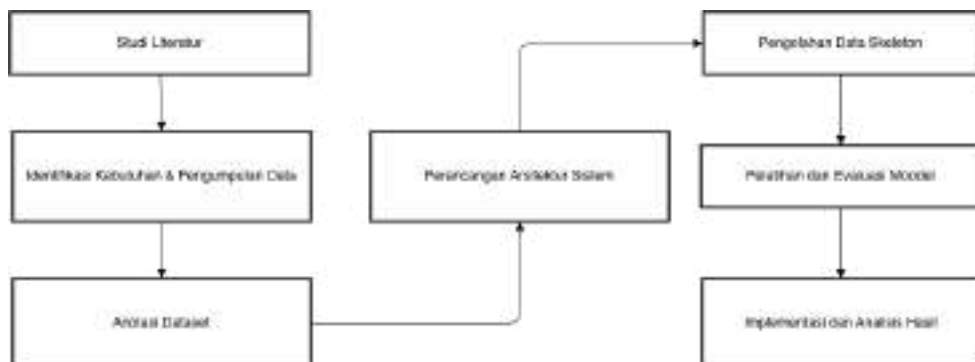
1. Penelitian dibatasi pada pengenalan 16 gerakan terpilih dari karakter Bryan Fury pada game Tekken 8.
2. Sistem yang dikembangkan berbasis *visual-only*, tanpa akses internal game, memory reading, atau game modification.
3. Pipeline sistem mencakup deteksi pemain, tracking, ROI crop, pose estimation, pembentukan skeleton, dan klasifikasi gerakan.
4. Penelitian menitikberatkan pada overlay anotasi gerakan berdasarkan prediksi model.
5. Penelitian belum mencakup character recognition otomatis, pengenalan seluruh karakter, atau pengenalan seluruh moveset Tekken 8.

1.5.2 Ruang Lingkup Spasial / Objek Penelitian:

1. Objek penelitian berupa rekaman gameplay Tekken 8 dalam format video dua pemain.
2. Dataset dan video uji utama berasal dari kondisi practice-mode yang terkontrol.
3. Pengujian pada pertandingan nyata atau skenario turnamen profesional tidak termasuk dalam cakupan penelitian ini.
4. Implementasi diuji pada lingkungan komputer lokal dengan perangkat consumer-grade.
5. Evaluasi difokuskan pada kelayakan pipeline, kualitas output visual, stabilitas skeleton, akurasi klasifikasi internal, dan latency inference.
6. Video kerja untuk anotasi, pelatihan, dan pengujian dibatasi pada 29,97 FPS.

1.6 Kerangka Penelitian

Penelitian ini merupakan experimental applied research dengan kerangka end-to-end machine learning pipeline yang berfokus pada pengembangan dan pengujian sistem berbasis computer vision untuk mengenali aksi karakter dalam permainan Tekken 8. Tahapan penelitian meliputi studi literatur, pengumpulan data, anotasi, ekstraksi skeleton, pelatihan model, evaluasi modular, dan implementasi overlay video.



Gambar 1.6: Flowchart kerangka penelitian

1. Studi

Literatur

Mengkaji penelitian terdahulu mengenai human action recognition, pose estimation, object tracking, dan skeleton-based action recognition pada konteks real-time.

2. **Identifikasi Kebutuhan dan Pengumpulan Data**
Menentukan karakter target, gerakan yang dianalisis, serta mengumpulkan rekaman gameplay practice-mode sebagai data utama penelitian.
3. **Anotasi Dataset**
Menyusun anotasi gerakan berdasarkan rentang frame, identitas pemain, karakter, dan label gerakan menggunakan custom annotator.
4. **Perancangan Arsitektur Sistem**
Merancang pipeline visual-only yang mencakup deteksi pemain, tracking identitas, ekstraksi ROI, estimasi pose, pembentukan data skeleton, klasifikasi gerakan, dan visualisasi overlay.
5. **Pengolahan Data Skeleton**
Mengolah keypoint hasil pose estimation menjadi format dataset skeleton-based action recognition yang sesuai untuk proses pelatihan dan inference.
6. **Pelatihan dan Evaluasi Model**
Melatih model action recognition berbasis skeleton dan mengevaluasi performa melalui validasi internal, kualitas visual output, stabilitas skeleton, dan latency inference.
7. **Implementasi dan Analisis Hasil**
Menjalankan inference pada video uji, menampilkan hasil prediksi dalam bentuk overlay, serta menganalisis sumber kesalahan pada setiap tahap pipeline.

1.7 Sistematika Penulisan

Sistematika penulisan Tugas Akhir ini terdiri dari lima bab utama sebagai berikut:

• **BAB I Pendahuluan**

Berisi penjelasan mengenai Latar Belakang, Perumusan Masalah, Tujuan dan Sasaran, Manfaat, Ruang Lingkup Pembahasan, Kerangka Penelitian, dan Sistematika Penulisan.

• **BAB II Tinjauan Pustaka**

Berisi penjelasan mengenai teori-teori yang digunakan untuk mendukung

analisis dan pembahasan, termasuk teori skeleton-based action recognition, pose estimation, dan penelitian terdahulu yang relevan.

• **BAB III Metodologi Penelitian**

Berisi gambaran objek penelitian, tahapan penelitian, metode pengumpulan data, arsitektur sistem rinci, model yang digunakan, metode analisis data, dan metode pengolahan data secara rinci.

• **BAB IV Analisis dan Pembahasan**

Berisi analisis hasil implementasi sistem, evaluasi performa, serta pembahasan hasil pengujian model dan sistem anotasi real-time pada Tekken 8.

• **BAB V Kesimpulan dan Saran**

Berisi kesimpulan dari hasil penelitian serta saran untuk pengembangan dan penelitian lanjutan.

BAB II TINJAUAN PUSTAKA

2.1 Kerangka Teoritis

2.1.1 Ekosistem Fighting Game dan Tekken 8 sebagai Objek Penelitian

Esports atau olahraga elektronik didefinisikan sebagai bentuk kompetisi permainan video yang terorganisir dan terstruktur, yang membedakannya dari permainan video kasual atau single-player. Kompetisi ini memiliki aturan resmi, institusi penyelenggara, dan karakteristik hasil menang atau kalah yang jelas (Kim et al., 2025). Dalam spektrum esports kompetitif, Tekken 8 merupakan entri terbaru dari waralaba fighting game tiga dimensi legendaris yang dikembangkan oleh Bandai Namco. Waralaba Tekken sendiri telah bertahan sejak era 1990-an, menunjukkan loyalitas komunitas pemain sekaligus relevansi berkelanjutan dalam industri esports global. Pentingnya penelitian pada objek ini didasari oleh relevansi industri yang kuat, tercermin dari data viewership turnamen internasional yang mengesankan.



Gambar 2.1: Jumlah viewer tertinggi per pertandingan dan keseluruhan, serta jumlah total jam ditonton selama periode Esports World Cup 2025 (Esports Charts, 2025)

Sebagaimana dijelaskan pada Bab 1, peningkatan viewership Tekken 8 menunjukkan bahwa fighting game memiliki daya tarik kompetitif yang kuat sebagai tontonan esports. Namun, meningkatnya jumlah penonton tidak serta-merta menghilangkan tantangan aksesibilitas dalam memahami permainan. Hal ini menimbulkan kebutuhan untuk menyajikan informasi

permainan secara lebih mudah dipahami oleh penonton, terutama bagi audiens yang tidak memiliki pengetahuan teknis mendalam mengenai mekanika Tekken.

Sebuah penelitian mengenai persepsi publik terhadap lima genre esports utama menunjukkan bahwa fighting game memperoleh penilaian lebih rendah pada aspek *entertaining* dibandingkan genre seperti FPS atau MOBA (Kim et al., 2025). Kondisi ini mengindikasikan bahwa daya tarik visual Tekken belum selalu diikuti oleh kemudahan penonton dalam memahami konteks mekanis di balik sebuah aksi. Penonton dapat melihat hasil akhir seperti serangan mengenai lawan, launch, combo, atau wall splat, tetapi belum tentu mengetahui gerakan atau command input yang menyebabkan situasi tersebut. Oleh karena itu, sistem anotasi dalam penelitian ini diposisikan bukan sebagai alat yang membuat penonton memahami keseluruhan strategi Tekken secara langsung, melainkan sebagai tahap awal untuk menjembatani aksi visual permainan dengan informasi semantik berupa label gerakan yang lebih eksplisit.

2.1.2 Kompleksitas Visual dan Mekanika Invisible

Berbeda dengan genre permainan lain, fighting game seperti Tekken 8 memiliki densitas informasi yang sangat tinggi dalam durasi yang sangat singkat. Sebuah interaksi serangan umumnya terdiri dari tiga fase: startup (persiapan), active (saat serangan aktif dan dapat mengenai lawan), dan recovery (pemulihan) (Demetrio, 2021). Fase startup seringkali terjadi hanya dalam rentang sekitar 9–15 frame, atau setara dengan 0,15–0,25 detik pada sistem 60 FPS (Wavu Wiki contributors, 2024). Dalam jendela waktu sesingkat itu, pengamat dituntut untuk mengenali animasi karakter, memprediksi arah serangan (High/Mid/Low), dan memahami implikasi strategisnya.

Tantangan visual ini diperburuk oleh konsep Invisible Mechanics. (Ma, 2024) menjelaskan dalam analisis desain game bahwa hambatan teknis (technical barrier) terbesar dalam fighting game adalah "knowledge checks", yaitu situasi di mana pemain harus menghafal properti spesifik dari ratusan gerakan yang tidak dijelaskan secara visual oleh game itu sendiri. Tanpa indikator visual eksternal pada User Interface, informasi krusial seperti apakah sebuah serangan "aman" (safe on block) atau "berbahaya" (punishable) tersembunyi di balik animasi. Hal ini memaksa pengamat

untuk mengandalkan ingatan hafalan (memori jangka panjang) daripada persepsi visual langsung.

Inti dari mekanika fighting game terletak pada konsep frame data. Frame data adalah representasi internal tentang cara kerja sebuah gerakan, yang diukur dalam satuan frame (di mana game seperti Tekken berjalan pada 60 frame per detik). Setiap gerakan dibagi menjadi tiga fase yang telah disebutkan sebelumnya: startup, active, dan recovery (Demetrio, 2021). Pengetahuan tentang frame data dianggap sebagai fondasi untuk memahami fighting games secara kompetitif. Seperti dijelaskan dalam analisis mekanika fighting game oleh publikasi TheGamer (Kennedy, 2022), pemahaman solid tentang frame data memberikan pemahaman yang jauh lebih baik terhadap setiap fighting game.

Frame data dalam Tekken dapat dipahami sebagai semacam 'tata bahasa' yang mengatur giliran dan keamanan gerakan, sebuah struktur yang tidak terlihat oleh penonton awam. Dua gerakan dengan animasi yang hampir identik dapat memiliki properti teknis yang sangat berbeda, menciptakan apa yang disebut sebagai "Ambiguitas Visual". Penonton baru mungkin dapat melihat efek visual seperti juggling (memukul lawan berulang kali di udara) sebagai hasil akhir yang jelas, tetapi mereka tidak mengerti mengapa itu terjadi atau bagaimana itu dimulai, karena mereka tidak dapat "membaca" interaksi frame data yang mendahului efek tersebut.

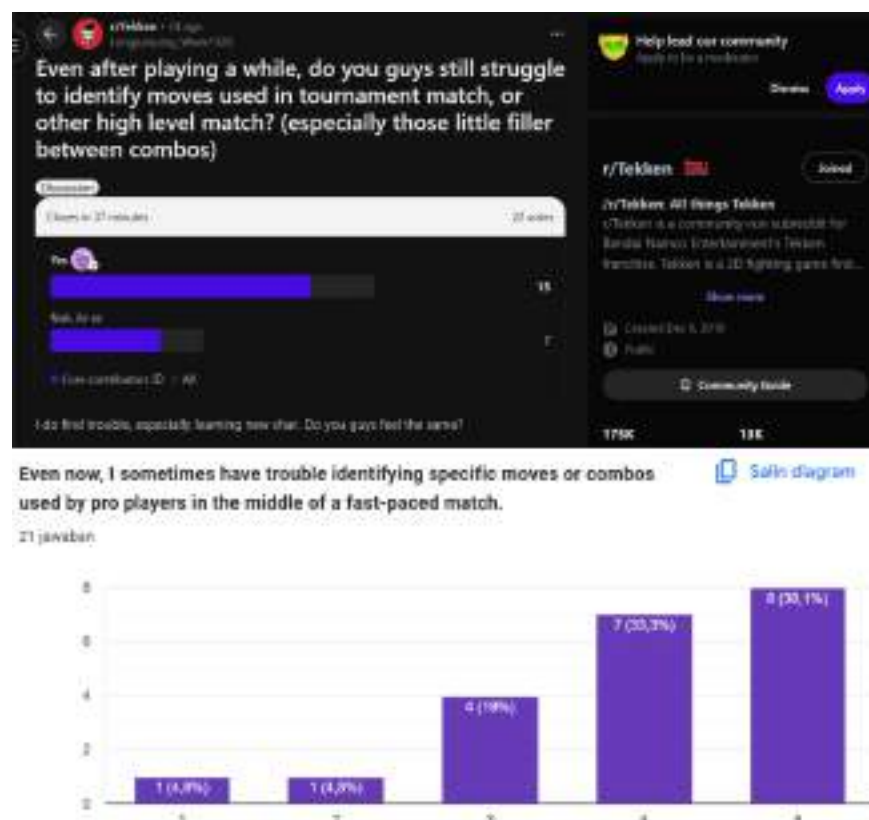
Sistem anotasi yang diusulkan dalam penelitian ini menampilkan anotasi gerakan secara real-time seperti "Bryan: f,b+2", sehingga berfungsi sebagai 'subtitle' yang memberi konteks semantik. Walaupun tidak menampilkan data frame secara eksplisit, anotasi ini membantu penonton mengaitkan gerakan tertentu dengan efek yang muncul (misalnya peluncuran ke udara atau launcher), sehingga mengurangi cognitive load yang diperlukan untuk memahami interaksi yang terjadi.

2.1.3 Beban Kognitif (Cognitive Load) dan Kesenjangan Pengetahuan Penonton

Kesulitan penonton dalam memahami informasi teknis pada tayangan Tekken 8 dapat ditinjau melalui Cognitive Load Theory. Teori ini menjelaskan bahwa pemrosesan informasi baru dibatasi oleh kapasitas working memory, sehingga penyajian informasi visual yang cepat dan kompleks berpotensi meningkatkan beban kognitif pengamat (Baxter et al.,

2025; Sweller et al., 2019). Dalam tayangan Tekken 8, penonton perlu memproses animasi karakter, efek visual, posisi kedua pemain, serta makna teknis suatu gerakan dalam waktu yang singkat. Akibatnya, penonton dapat mengenali hasil visual seperti hit, launch, combo, atau wall splat, tetapi belum tentu memahami gerakan yang menyebabkan kondisi tersebut.

Perolehan pengetahuan merupakan salah satu motivasi penting dalam menonton esports. Konstruk *acquisition of knowledge* dilaporkan memiliki hubungan paling kuat dengan niat untuk menonton esports (Macey et al., 2025). Temuan ini menunjukkan bahwa sebagian penonton tidak hanya mencari hiburan, tetapi juga ingin memahami permainan dan mempelajari strategi yang ditampilkan. Dalam konteks Tekken 8, kesenjangan antara aksi yang terlihat dan mekanika yang tidak dijelaskan secara eksplisit berpotensi menghambat proses tersebut. Kondisi ini dapat memperlebar kesenjangan pengetahuan penonton ketika mereka melihat hasil suatu aksi, tetapi tidak mengetahui gerakan atau *command* yang menyebabkannya.



Gambar 2.2: Hasil Polling mixed-methods

Validasi empiris diperoleh melalui survei mixed-methods (detail metodologi di Tabel 3.1) dengan distribusi multi-channel yang dilakukan penulis. Temuan kuantitatif menunjukkan:

- 69.8% responden mengalami kesulitan identifikasi moveset saat menonton pertandingan tingkat tinggi (dikumpulkan dari Google Form dan Reddit poll dengan konsistensi tinggi: 71.4% dan 68.2%)
- 33.3% responden harus menonton ulang dengan kecepatan lambat untuk memahami interaksi yang terjadi (N=21)
- 71.5% responden menilai sistem anotasi real-time akan berguna (rating $\geq 4/5$ pada skala Likert) (N=21)

Lebih krusial lagi, data kualitatif dari diskusi komunitas mengungkap bahwa bahkan pemain dengan pengalaman bertahun-tahun mengonfirmasi kesulitan ini. Seorang responden menekankan dengan tegas: "watching matches randomly did absolutely nothing for recognizing moves", mengindikasikan bahwa pembelajaran melalui observasi pasif tidak efektif tanpa sistem bantuan. Responden lain menambahkan bahwa "pros are very good at hiding moves with excellent movement", menjelaskan mengapa animasi halus pada tournament-level gameplay menyebabkan visual recognition failure bahkan bagi pemain berpengalaman.

Secara keseluruhan, temuan tersebut mengindikasikan adanya kesulitan identifikasi gerakan pada sampel responden penelitian, termasuk di antara pemain yang telah memiliki pengalaman bermain. Temuan ini belum membuktikan efektivitas sistem anotasi secara langsung, tetapi memberikan dasar empiris bagi kebutuhan untuk mengevaluasi sistem bantuan visual yang dapat menghubungkan aksi permainan dengan label gerakan.

2.1.4 Computer Vision sebagai Solusi Real-Time Annotation

Untuk mengatasi batasan persepsi manusia dan ambiguitas visual tersebut, diperlukan intervensi teknologi berupa sistem Computer Vision (CV). Dalam domain analisis olahraga dan aktivitas fisik, CV digunakan untuk mengekstraksi semantik dari video piksel mentah, mengubahnya

menjadi data terstruktur yang dapat dipahami (C. Wang & Yan, 2023). Prinsip yang sama dapat diaplikasikan pada konteks esports untuk menjembatani knowledge gap.

Dengan mengadopsi prinsip pengelolaan beban kognitif (Sweller et al., 2019), anotasi real-time mengurangi Extraneous Cognitive Load melalui penyajian label gerakan yang eksplisit. Pendekatan ini berfungsi sebagai cognitive offloading, yaitu memindahkan beban pemrosesan identifikasi pola dari memori kerja (working memory) pengamat ke sistem komputasi. Dengan menampilkan nama gerakan (misalnya: "Bryan: f,b+2") di layar secara real-time, sistem mengambil alih tugas kognitif berat berupa "identifikasi", sehingga pengamat dapat mengalokasikan sumber daya kognitifnya untuk tugas tingkat lebih tinggi yaitu "pemahaman konteks" dan analisis strategis.

Kebutuhan sistem anotasi otomatis divalidasi oleh dua indikator konvergen dari data survei:

1. **Tingkat penerimaan tinggi: 71.5% responden menilai sistem overlay real-time akan "useful" atau "very useful" (rating $\geq 4/5$), menunjukkan demand yang kuat dari komunitas (N=21)**
2. **Keterbatasan solusi sementara: Pemain saat ini harus melakukan "intentional labbing" di Practice Mode untuk menghafal setiap gerakan secara manual, sebuah proses yang memakan waktu dan tidak scalable untuk ratusan moveset per karakter**

Dalam implementasinya, penelitian terbaru menunjukkan bahwa representasi berbasis kerangka (skeleton-based) lebih unggul daripada berbasis piksel (RGB) untuk tugas pengenalan aksi. Metode berbasis RGB memiliki kelemahan signifikan berupa sensitivitas terhadap noise latar belakang, perubahan pencahayaan, dan efek partikel visual yang umum ditemukan dalam video game (Liu et al., 2025). Sebaliknya, metode skeleton-based yang memanfaatkan topologi graf spasial-temporal terbukti lebih robust dan efisien secara komputasi, menjadikannya pendekatan ideal untuk aplikasi real-time (Duan, Zhao, et al., 2022; Xie et al., 2025).

Pendekatan skeleton-based telah terbukti efektif dalam domain analisis olahraga (Phalke et al., 2025) dan sistem keamanan (Noor & Park, 2023), namun belum pernah diterapkan untuk meningkatkan pemahaman

penonton dalam konteks fighting game. Berdasarkan penelusuran literatur yang dilakukan, belum ditemukan penelitian yang secara spesifik menerapkan pipeline visual-only berbasis object tracking, pose estimation, dan skeleton-based action recognition untuk mengenali moveset Tekken 8.

Premis penggunaan CV untuk meningkatkan pengalaman penonton esports juga didukung oleh penelitian pada konteks esports lain. Sebuah sistem *hierarchical scene detection and tracking* berbasis Vision Transformer dikembangkan untuk meningkatkan pengalaman penonton melalui deteksi adegan dan pelacakan otomatis (Joo et al., 2025). Temuan tersebut memberikan preseden bagi penggunaan CV untuk mendukung kualitas penyajian informasi dalam siaran esports. Ini memvalidasi penggunaan CV untuk meningkatkan kualitas siaran dan pemahaman penonton esports secara real-time, memperkuat argumen bahwa pendekatan teknologi dapat secara efektif mengatasi spectator knowledge gap.

2.2 Tinjauan Teoritis

2.2.1 Computer Vision (CV)

Computer Vision (CV) atau Visi Komputer adalah bidang ilmu interdisipliner yang berfokus pada pengembangan sistem buatan yang mampu memproses, menganalisis, dan memahami data visual, seperti gambar dan video, untuk mengekstraksi informasi bermakna dari dunia visual (Khan et al., 2020). Ruang lingkupnya mencakup berbagai penerapan, mulai dari deteksi dan klasifikasi objek, segmentasi citra, hingga pelacakan pergerakan.

Secara historis, bidang ini telah berevolusi secara signifikan. Sebuah tinjauan (Gülsoylu et al., 2025) yang mencakup 35 tahun (1990-2025) menelusuri evolusi dari metode pemrosesan citra digital (DIP) dan machine learning (ML) tradisional—yang sangat bergantung pada ekstraksi fitur manual (hand-crafted features)—menuju "dominasi teknik deep learning (DL)" saat ini. Dalam paradigma Deep Learning, fitur-fitur yang relevan untuk sebuah tugas (misalnya, bentuk, tekstur, pola) dipelajari secara otomatis oleh Jaringan Saraf Dalam (Deep Neural Networks) melalui proses

pelatihan pada data dalam jumlah besar. Penelitian ini sepenuhnya berada dalam paradigma Deep Learning modern.

2.2.2 Human Action Recognition (HAR)

Pengenalan Aksi Manusia (Human Action Recognition, HAR) adalah cabang spesifik dari Computer Vision yang berfokus pada identifikasi dan klasifikasi aksi manusia dari data visual, seperti sekuens video atau sekuens rangka. HAR melibatkan deteksi fitur statis dalam satu frame serta fitur dinamis antar beberapa frame berurutan. Tujuannya adalah mengembangkan sistem otomatis yang meniru kemampuan manusia untuk memahami dan mendeskripsikan aksi dalam suatu adegan (C. Wang & Yan, 2023).

Secara umum, terdapat dua pendekatan dominan dalam HAR yang memiliki karakteristik, kelebihan, dan keterbatasan berbeda. Perbandingan langsung antara pendekatan berbasis RGB dan skeleton telah dibahas dalam sebuah survei komprehensif (C. Wang & Yan, 2023):

1. Metode Berbasis RGB: Metode ini memproses seluruh data piksel dari video. Kelemahannya adalah "biaya komputasi tinggi" (high computational costs) dan "sensitivitas terhadap lingkungan" (environmental sensitivity) (C. Wang & Yan, 2023). Dalam konteks Tekken 8, sensitivitas ini menjadi masalah besar: latar belakang game yang sangat dinamis, efek partikel yang mencolok (misalnya saat serangan mengenai lawan), dan perubahan pencahayaan drastis dapat "menipu" model RGB dan dianggap sebagai bagian dari aksi.
2. Metode Berbasis Kerangka (Skeleton-Based): Metode ini hanya menggunakan data posisi sendi tubuh (kerangka) sebagai representasi aksi. Keunggulannya adalah representasi yang "ringkas dan kuat" (compact and robust). Metode ini menghindari biaya komputasi tinggi dan tidak sensitif terhadap faktor lingkungan (latar belakang, pencahayaan, tekstur pakaian karakter) (C. Wang & Yan, 2023).

Pemilihan pendekatan "Skeleton-Based" dalam judul penelitian adalah keputusan desain yang krusial. Tujuan utama

penelitian adalah membangun sistem real-time yang akurat. Video gameplay Tekken 8 penuh dengan "gangguan" visual yang akan merusak performa model berbasis RGB. Selain itu, model RGB terlalu berat secara komputasi (C. Wang & Yan, 2023) untuk mencapai performa real-time (misalnya, >30 FPS) pada hardware konsumen.

Dengan demikian, metode berbasis kerangka memecahkan kedua masalah ini secara serentak: (a) ia ringan secara komputasi, dan (b) ia tahan terhadap gangguan visual game karena hanya fokus pada pose karakter. Pendekatan berbasis kerangka adalah pilihan metodologis yang paling layak untuk mencapai tujuan penelitian (akurasi, kecepatan, dan ketahanan) dalam konteks gameplay Tekken 8.

2.2.3 Real-Time Object Detection, Multi-Object Tracking, dan Ekstraksi Region of Interest (RoI)

Pipeline skeleton-based action recognition tidak dapat dimulai tanpa terlebih dahulu mengidentifikasi posisi karakter target di dalam frame video. Oleh karena itu, tahap awal sistem membutuhkan object detection untuk memperoleh bounding box pemain, kemudian dilanjutkan dengan tracking agar identitas pemain dapat dipertahankan antar-frame. Dalam konteks Tekken 8, kebutuhan ini menjadi penting karena dua karakter sering berada dalam kondisi close contact, overlap, side switch, throw, sidestep, dash, atau animasi berputar yang dapat menyebabkan posisi kiri-kanan pemain berubah secara cepat.

Object detection merupakan proses untuk menemukan dan melokalisasi objek tertentu dalam gambar atau video. Pada penelitian ini, object detection digunakan untuk mendeteksi posisi karakter pemain dalam setiap frame gameplay. Salah satu keluarga model deteksi objek yang banyak digunakan dalam aplikasi real-time adalah YOLO (You Only Look Once), karena pendekatan single-stage detector mampu melakukan lokalisasi dan klasifikasi objek secara efisien dalam satu tahap inferensi. Penelitian ini menggunakan YOLOv11 sebagai detector karena YOLOv11 dirancang untuk mendukung berbagai tugas computer vision,

termasuk object detection dan pose estimation, dengan penekanan pada keseimbangan antara akurasi dan efisiensi komputasi (Jocher & Qiu, 2024; Khanam & Hussain, 2024).

YOLOv11 memperkenalkan beberapa peningkatan arsitektur dibandingkan versi sebelumnya, seperti penggunaan C3k2 block, C2PSA block, dan strategi feature aggregation yang bertujuan meningkatkan ekstraksi fitur sekaligus menjaga efisiensi inference (Khanam & Hussain, 2024; Li et al., 2025). Dalam penelitian ini, YOLOv11 tidak diposisikan sebagai satu-satunya komponen utama, melainkan sebagai tahap awal dari pipeline end-to-end yang terdiri dari detector, tracker, ROI extraction, pose estimation, dan action recognition. Dengan demikian, efisiensi pada tahap deteksi menjadi penting karena keterlambatan pada modul pertama dapat mempengaruhi latency keseluruhan pipeline.

Namun, deteksi objek per-frame saja belum cukup untuk menganalisis video gameplay secara kontinu. Detektor hanya menghasilkan bounding box pada frame tertentu, tetapi tidak secara otomatis mempertahankan identitas objek dari frame ke frame. Dalam pertandingan Tekken 8, karakter dapat saling bertukar posisi akibat throw, sidestep, combo, heat/rage animation, atau gerakan non-linear lainnya. Jika sistem hanya mengandalkan posisi kiri-kanan, identitas Player 1 dan Player 2 dapat tertukar ketika terjadi side switch. Oleh karena itu, penelitian ini menggunakan pendekatan multi-object tracking berbasis tracking-by-detection.

Multi-Object Tracking (MOT) bertujuan untuk mempertahankan identitas objek dalam video dengan mengasosiasikan hasil deteksi antar-frame. Salah satu tracker yang banyak digunakan adalah ByteTrack, yang mengasosiasikan hampir semua detection box, termasuk box dengan confidence rendah, untuk mengurangi fragmentasi lintasan dan memulihkan objek yang sempat memiliki skor deteksi rendah (Zhang et al., 2022). Pendekatan ini relevan sebagai baseline karena dapat mempertahankan objek yang mengalami oklusi singkat. Namun, dalam konteks Tekken 8, detection box dengan confidence rendah tidak selalu merepresentasikan karakter target secara akurat, terutama ketika dua karakter saling overlap, terkena efek visual, atau berada dalam pose yang tidak umum.

Pada tahap akhir penelitian ini, OC-SORT dipilih sebagai tracker karena lebih sesuai untuk karakteristik gerakan Tekken 8 yang dinamis. OC-SORT atau Observation-Centric SORT dirancang untuk meningkatkan ketahanan SORT terhadap oklusi dan gerakan non-linear melalui pendekatan observation-centric association (J. Cao et al., 2023). Karakteristik ini relevan dengan gameplay Tekken 8 karena gerakan pemain tidak selalu linear, melainkan dapat berubah secara cepat akibat dash, sidestep, throw, tailspin, wall interaction, dan side switch. Dengan menggunakan OC-SORT, sistem memiliki peluang lebih baik untuk mempertahankan identitas pemain tanpa hanya bergantung pada posisi kiri-kanan karakter.

Setelah proses tracking, sistem melakukan ekstraksi Region of Interest (RoI) berdasarkan bounding box karakter target. RoI extraction bertujuan membatasi area pemrosesan pose estimation hanya pada wilayah tubuh karakter yang sedang dianalisis. Pendekatan ini penting karena frame gameplay Tekken 8 mengandung banyak elemen non-target, seperti background stage, health bar, particle effects, UI, dan tubuh lawan. Dengan melakukan cropping pada area target, sistem dapat mengurangi gangguan visual dan memfokuskan pose estimation pada karakter yang relevan.

Meskipun demikian, ukuran RoI perlu dirancang secara hati-hati. RoI yang terlalu sempit dapat memotong bagian tubuh seperti tangan, kaki, atau kepala, sehingga keypoint yang dihasilkan menjadi tidak lengkap. Sebaliknya, RoI yang terlalu lebar dapat memasukkan tubuh lawan ke dalam area crop, sehingga model pose estimation dapat menghasilkan kandidat skeleton yang tidak sesuai dengan karakter target. Oleh karena itu, dalam pipeline penelitian ini, RoI extraction tidak berdiri sendiri, tetapi dikombinasikan dengan tracking identity dan pose candidate selection agar skeleton yang diproses oleh model action recognition tetap konsisten terhadap karakter target.

2.2.4 Estimasi Pose 2D (2D Pose Estimation) dan Pose Candidate Selection

Setelah posisi karakter target diperoleh melalui object detection, tracking, dan RoI extraction, tahap berikutnya adalah mengekstraksi keypoint tubuh karakter. Proses ini dikenal sebagai

Human Pose Estimation (HPE), yaitu proses memperkirakan lokasi titik-titik tubuh manusia seperti kepala, bahu, siku, pergelangan tangan, pinggul, lutut, dan pergelangan kaki dari gambar atau video. Dalam penelitian ini, pose estimation berfungsi sebagai tahap feature extraction, karena model action recognition tidak menerima citra mentah sebagai input, melainkan sequence koordinat keypoint yang merepresentasikan gerakan karakter.

Secara umum, metode *multi-person 2D pose estimation* dapat dibedakan menjadi pendekatan *top-down* dan *bottom-up* (Toshpulatov et al., 2022). Pada pendekatan *bottom-up*, *keypoint* seluruh individu dideteksi terlebih dahulu, kemudian dikelompokkan berdasarkan afiliasi tubuh, seperti pada metode berbasis *Part Affinity Fields* (Z. Cao et al., 2017). Sebaliknya, pendekatan *top-down* terlebih dahulu mendeteksi *bounding box* setiap individu sebelum menjalankan *single-person pose estimator*, seperti pada RMPE (Fang et al., 2018). RMPE dikembangkan untuk mengurangi kegagalan estimasi pose akibat *bounding box* yang tidak akurat melalui *Symmetric Spatial Transformer Network*, *Parametric Pose Non-Maximum Suppression*, dan *Pose-Guided Proposals Generator*. Dalam pipeline penelitian ini yang menggunakan pendekatan *top-down*, kualitas *bounding box* menentukan area masukan model pose. Oleh karena itu, ketika RoI masih memuat lebih dari satu karakter, diperlukan pemilihan kandidat pose agar skeleton tetap sesuai dengan karakter target.

Dalam konteks real-time, MoveNet menjadi salah satu model pose estimation yang relevan karena dirancang untuk mendeteksi 17 keypoint tubuh secara cepat. MoveNet memiliki dua varian utama, yaitu Lightning untuk aplikasi yang menekankan latency rendah dan Thunder untuk aplikasi yang menekankan akurasi lebih tinggi (TensorFlow, 2024). MoveNet juga telah dibandingkan dengan model lain seperti OpenPose dan PoseNet dalam konteks perangkat mobile, sehingga tetap relevan sebagai baseline awal untuk eksperimen pose estimation cepat (Jo & Kim, 2022). Namun, dalam penelitian ini, MoveNet tidak lagi diposisikan sebagai model utama pada pipeline akhir, melainkan sebagai bagian dari eksperimen awal dan baseline pembanding.

Keterbatasan utama model pose dalam konteks Tekken 8 bukan hanya kecepatan inference, tetapi juga konsistensi skeleton

terhadap karakter target. Pada video fighting game, dua karakter sering berada sangat dekat, saling menutupi, atau berada dalam satu area crop. Model SinglePose dapat menghasilkan skeleton dengan cepat, tetapi berisiko memilih tubuh yang salah jika RoI berisi lebih dari satu karakter. Sebaliknya, model MultiPose dapat menghasilkan beberapa kandidat pose, tetapi tetap membutuhkan mekanisme pemilihan kandidat agar skeleton yang digunakan benar-benar sesuai dengan bounding box target hasil tracking.

Berdasarkan kebutuhan tersebut, penelitian ini menggunakan YOLO11l-Pose sebagai model pose estimation pada pipeline akhir. YOLO11 mendukung beberapa tugas computer vision, termasuk pose estimation, sedangkan format COCO-Pose menggunakan 17 keypoint tubuh manusia yang umum digunakan dalam human pose estimation (Jocher & Qiu, 2024; Lin et al., 2015). Pada penelitian ini, YOLO11l-Pose dijalankan pada RoI karakter target dengan ukuran input $\text{imgsz}=384$. Pemilihan varian YOLO11l-Pose tidak semata-mata didasarkan pada kecepatan tertinggi, melainkan pada keseimbangan antara stabilitas keypoint dan latency inference. Dalam konteks action recognition berbasis skeleton, skeleton yang stabil lebih penting dibanding pose estimation yang sangat cepat tetapi sering berpindah ke karakter lawan.

Karena YOLO11l-Pose dapat menghasilkan lebih dari satu kandidat skeleton dalam satu RoI, penelitian ini menerapkan pose candidate selection. Mekanisme ini memilih kandidat pose terbaik berdasarkan kecocokan antara bounding box kandidat pose dan bounding box target hasil tracking. Kriteria yang digunakan mencakup Intersection over Union (IoU), jarak center bounding box, dan confidence pose. Kandidat dengan kecocokan terbaik dipilih sebagai skeleton target, sedangkan kandidat lain diabaikan. Strategi ini bertujuan mengurangi risiko identity switching pada level skeleton, terutama ketika dua karakter saling overlap atau ketika RoI masih memuat sebagian tubuh lawan.

Dengan demikian, pose estimation pada penelitian ini tidak hanya dipahami sebagai proses menghasilkan keypoint, tetapi sebagai proses menjaga konsistensi representasi tubuh karakter target. Output dari tahap ini berupa koordinat keypoint COCO-17 beserta confidence score yang kemudian disusun menjadi sequence skeleton. Kualitas tahap pose estimation sangat menentukan

performa tahap action recognition, karena kesalahan keypoint, missing keypoint, atau skeleton yang tertukar akan langsung mempengaruhi input ST-GCN++.

2.2.5 Skeleton-Based Action Recognition (Pengenalan Aksi Berbasis Kerangka)

Skeleton-Based Action Recognition adalah pendekatan pengenalan aksi yang menggunakan representasi tubuh manusia dalam bentuk titik-titik sendi atau keypoint. Berbeda dengan metode berbasis RGB yang memproses seluruh piksel video, metode berbasis skeleton hanya menggunakan informasi posisi tubuh, sehingga representasinya lebih ringkas dan relatif lebih tahan terhadap gangguan visual seperti perubahan latar belakang, pencahayaan, tekstur pakaian, atau efek visual (C. Wang & Yan, 2023). Karakteristik ini sangat relevan untuk Tekken 8 karena video gameplay mengandung banyak elemen visual non-target, seperti particle effects, health bar, stage background, dan efek benturan serangan.

Pada penelitian ini, skeleton diperoleh dari hasil YOLO11-Pose dalam format COCO-17 keypoints. Setiap frame direpresentasikan sebagai kumpulan koordinat keypoint dan confidence score, kemudian disusun menjadi sequence temporal. Karena gerakan dalam Tekken 8 tidak hanya dibedakan oleh pose pada satu frame, tetapi juga oleh perubahan pose antar-frame, model action recognition harus mampu mempelajari pola spasial dan temporal secara bersamaan. Sebagai contoh, dua gerakan dapat memiliki pose akhir yang mirip, tetapi berbeda pada fase startup atau transisi tubuh sebelum impact. Oleh karena itu, representasi temporal menjadi bagian penting dalam pengenalan moveset.

Salah satu pendekatan penting dalam skeleton-based action recognition adalah Spatial-Temporal Graph Convolutional Network (ST-GCN). ST-GCN memodelkan skeleton sebagai graf, di mana keypoint tubuh diperlakukan sebagai node dan hubungan antar sendi diperlakukan sebagai edge. Dengan pendekatan ini, model dapat mempelajari relasi spasial antar bagian tubuh sekaligus perubahan temporal antar-frame (Yan et al., 2018). Pendekatan berbasis graf

sesuai dengan karakteristik skeleton karena struktur tubuh manusia secara alami memiliki hubungan topologis, misalnya antara bahu, siku, dan pergelangan tangan.

Penelitian ini menggunakan ST-GCN++ sebagai model klasifikasi aksi berbasis skeleton. ST-GCN++ diperkenalkan melalui *PYSKL: Towards Good Practices for Skeleton Action Recognition*, sebuah toolbox berbasis PyTorch yang menyediakan berbagai metode skeleton action recognition dalam satu kerangka eksperimen. Model ini dirancang sebagai baseline yang sederhana, kuat, dan efisien, serta menunjukkan bahwa praktik preprocessing, augmentasi, dan representasi data yang baik dapat memberikan performa kompetitif tanpa menggunakan arsitektur yang terlalu kompleks (Duan, Wang, et al., 2022).

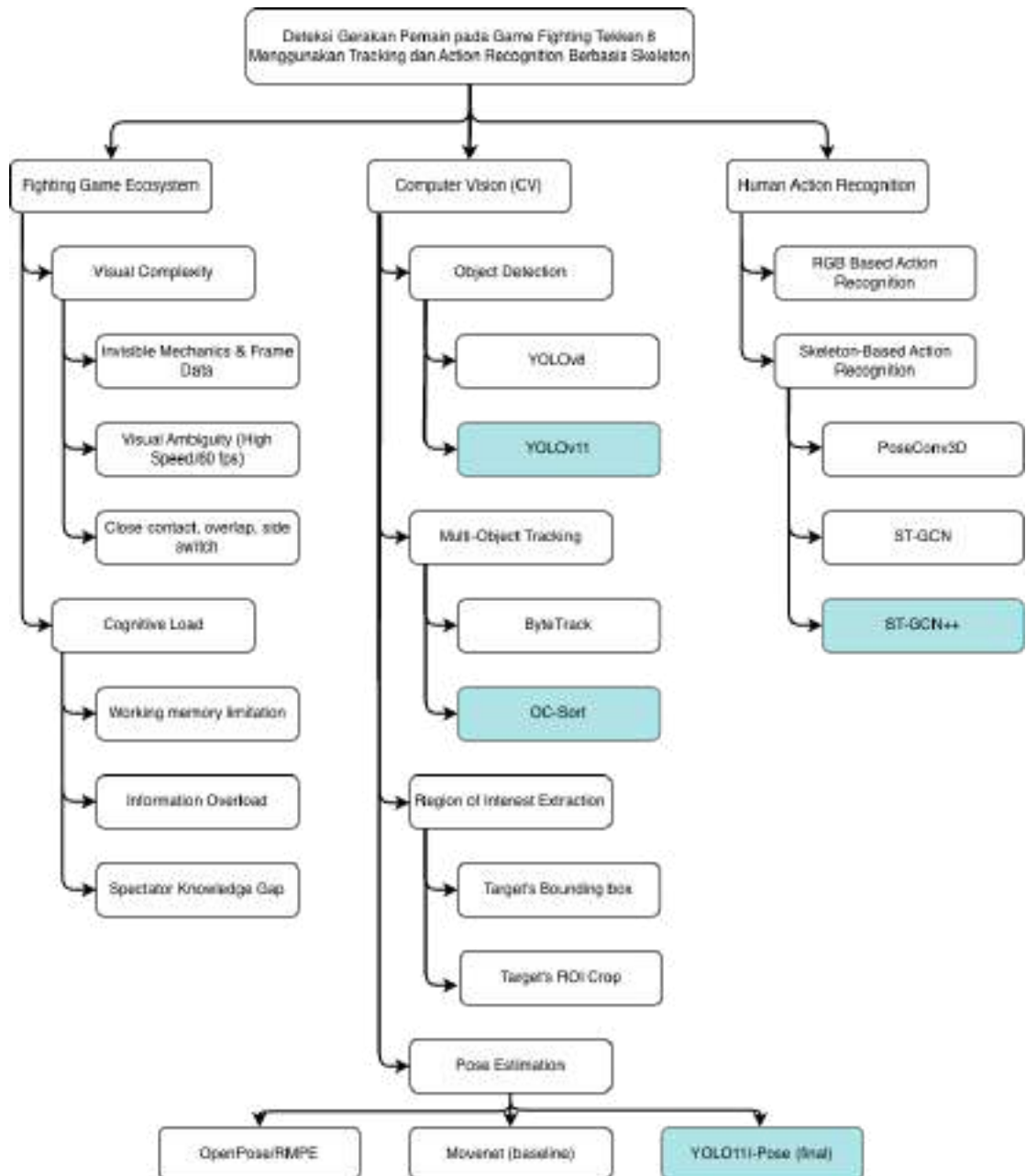
Pemilihan ST-GCN++ dalam penelitian ini didasarkan pada kebutuhan keseimbangan antara akurasi dan efisiensi komputasi. Sistem yang dikembangkan ditujukan sebagai prototipe anotasi real-time, sehingga model action recognition tidak boleh menjadi bottleneck utama setelah tahap detection, tracking, RoI extraction, dan pose estimation. Literatur terbaru menunjukkan bahwa model skeleton-based ringan masih relevan untuk aplikasi real-time. Model 3D-CNN ringan telah dikembangkan untuk *fall detection* dan action recognition real-time (Noor & Park, 2023), sedangkan sistem skeleton-based dengan jumlah parameter rendah telah diarahkan untuk perangkat embedded (Noor et al., 2024). Selain itu, arsitektur ST-GCN modern dapat mengalami *over-parameterization*, sehingga pemilihan model yang lebih sederhana tetapi efisien menjadi pertimbangan yang rasional (Xie et al., 2025).

Dalam pipeline penelitian ini, sequence skeleton dibentuk menggunakan fixed 30-frame window. Window ini digunakan agar format data pelatihan konsisten dengan mode inference yang membaca urutan keypoint dalam rentang waktu tertentu. Dalam konteks gameplay Tekken 8, 30 frame pada sistem 60 FPS secara konseptual merepresentasikan sekitar setengah detik. Namun, karena video kerja pada penelitian ini diproses pada 29,97 FPS, fixed 30-frame window merepresentasikan sekitar satu detik data visual. Pemilihan window ini tidak diposisikan sebagai standar universal, melainkan sebagai keputusan metodologis yang disesuaikan dengan

format video kerja, kebutuhan eksperimen, durasi gerakan, dan implementasi rolling-window inference.

Dengan demikian, ST-GCN++ berperan sebagai modul akhir yang menerima sequence skeleton dari YOLO11l-Pose dan mengklasifikasikannya ke dalam label gerakan Bryan Fury atau idle class. Kinerja model ini sangat dipengaruhi oleh kualitas modul sebelumnya, terutama kestabilan tracking, ketepatan RoI, kualitas keypoint, dan keberhasilan pose candidate selection. Oleh karena itu, skeleton-based action recognition dalam penelitian ini tidak dapat dipisahkan dari keseluruhan pipeline computer vision yang membentuk input skeleton secara konsisten.

2.3 Studi Preseden



Gambar 2.3: Pemetaan komponen studi preseden dan pendekatan yang dipilih

2.3.1 Penjelasan Diagram dan Studi Preseden

Gambar 2.3 memetakan tiga kelompok utama yang menjadi dasar penelitian, yaitu konteks fighting game, komponen computer vision, dan

human action recognition. Cabang fighting game menjelaskan permasalahan kompleksitas visual dan beban kognitif penonton. Cabang computer vision menunjukkan perkembangan pendekatan deteksi pemain, tracking identitas, ekstraksi ROI, dan pose estimation. Sementara itu, cabang human action recognition menunjukkan perkembangan dari pendekatan berbasis RGB menuju skeleton-based action recognition menggunakan PoseConv3D, ST-GCN, dan ST-GCN++. Komponen yang diberi warna berbeda menunjukkan pendekatan yang digunakan dalam pipeline akhir penelitian.

Bagian ini menyajikan tinjauan terhadap penelitian-penelitian terdahulu yang paling relevan dengan implementasi sistem yang diusulkan. Berbeda dengan Tinjauan Teoritis (2.2) yang membedah komponen individu (seperti YOLO atau GCN), Studi Preseden (2.3) berfokus pada makalah utuh yang mengimplementasikan pipeline serupa (Pose Estimation + Action Recognition) atau menangani masalah dalam konteks serupa (eSports, real-time).

Sintesis dari studi-studi kunci yang dipetakan pada Gambar 2.3 menunjukkan tren yang jelas menuju model yang lebih ringan dan efisien untuk aplikasi real-time.

- Efisiensi dalam Sistem Real-Time: Sebuah model 2D-CNN ringan dengan kurang dari satu juta parameter dikembangkan untuk memproses heatmap skeleton dari YOLOv8s-pose pada perangkat embedded (Noor et al., 2024). Model 3D-CNN ringan juga telah digunakan untuk deteksi jatuh dan pengenalan aksi real-time, yang menunjukkan kemungkinan integrasi pose estimation dan action recognition dalam satu pipeline (Noor & Park, 2023).
- Validasi Arsitektur yang Dipilih: ST-GCN++ diperkenalkan sebagai baseline sederhana dengan performa kompetitif, yang menunjukkan pentingnya *good practices* dibandingkan penambahan kompleksitas arsitektur secara berlebihan (Duan, Wang, et al., 2022). Temuan tersebut mendukung pemilihan ST-GCN++ sebagai model klasifikasi pada penelitian ini. Penelitian pose estimation berfrekuensi tinggi berbasis event camera tetap relevan sebagai preseden untuk efisiensi HPE, tetapi MoveNet hanya diposisikan sebagai baseline awal dalam penelitian ini (Goyal et al., 2023). Pipeline akhir menggunakan YOLO11l-Pose karena eksperimen

menunjukkan bahwa kebutuhan sistem tidak hanya mencakup latency rendah, tetapi juga kestabilan skeleton karakter target dalam kondisi *close contact*, *overlap*, dan *side switch*.

- Konteks eSports dan Gaming: Penggunaan Vision Transformer untuk deteksi adegan dan pelacakan otomatis telah diteliti dengan tujuan meningkatkan pengalaman penonton esports (Joo et al., 2025). Penelitian tersebut memberikan preseden bagi penggunaan CV untuk mendukung kualitas penyajian informasi dalam siaran esports

1. Efficient Skeleton-Based Action Recognition for Real-Time Embedded Systems (Noor et al., 2024)

- **Field of Establishment:** Real-Time Computer Vision, Embedded Systems, Skeleton-based Action Recognition

- **Problem:** Mengembangkan model pengenalan aksi berbasis skeleton yang dapat bekerja real-time pada perangkat embedded dengan sumber daya komputasi terbatas sambil mempertahankan akurasi tinggi

- **Solution:** Implementasi model 2D-CNN ringan yang memproses heatmap skeleton dari koordinat keypoint, menggunakan YOLOv8s-pose yang telah dipruning dan dikuantisasi untuk estimasi pose yang efisien

- **Contribution:** Arsitektur dengan parameter <1 juta yang mencapai akurasi superior dibanding baseline dengan kecepatan inferensi tinggi, membuktikan feasibilitas HAR real-time pada embedded devices

- **Evaluation:** Pengujian pada dataset standar menunjukkan peningkatan akurasi dan kecepatan inferensi simultan dibanding PoseConv3D dan metode baseline lainnya

2. Revisiting Skeleton-Based Action Recognition (Duan et al., 2022 - CVPR)

- **Field of Establishment:** Deep Learning, Skeleton-based Human Action Recognition, Spatio-Temporal Analysis

- **Problem:** Mengatasi keterbatasan metode GCN konvensional (ST-GCN) dalam hal sensitivitas terhadap noise pose detection, kesulitan handling multi-person, dan kompleksitas komputasi

- **Solution:** PoseConv3D yang merepresentasikan data skeleton sebagai 3D volumetric heatmap dan menggunakan 3D-CNN untuk ekstraksi fitur spatio-temporal, menghindari representasi graf yang kompleks

- **Contribution:** Demonstrasi bahwa pendekatan CNN

berbasis volume dapat mengungguli metode GCN untuk skeleton HAR, dengan robustness superior terhadap error pose detection dan kemampuan multi-person native

- **Evaluation:** Mencapai state-of-the-art performance pada 5 dari 6 benchmark skeleton recognition (NTU, Kinetics) dan SOTA pada 8 dari 8 benchmark multimodal dengan fusion RGB+skeletal

3. PYSKL: Towards Good Practices for Skeleton Action Recognition (Duan et al., 2022 - ACM MM)

- **Field of Establishment:** Machine Learning Tools, Benchmarking, Reproducible Research in Skeleton HAR

- **Problem:** Inkonsistensi dalam praktik eksperimen antar berbagai implementasi skeleton HAR, kesulitan reproduksi hasil, dan kurangnya standarisasi dalam preprocessing dan evaluation

- **Solution:** Framework PyTorch terpadu yang mengintegrasikan berbagai algoritma (GCN & CNN) dengan good practices standar, memperkenalkan ST-GCN++ sebagai baseline sederhana namun efektif

- **Contribution:** Standarisasi praktik terbaik dalam skeleton HAR research, penyediaan model pre-trained dan benchmark scripts, serta demonstrasi bahwa simplicity dengan good practices mengalahkan architectural complexity

- **Evaluation:** ST-GCN++ mencapai competitive performance hingga SOTA pada NTU-120 dengan kompleksitas minimal, membuktikan pentingnya augmentasi dan preprocessing dibanding arsitektur rumit

4. Skeleton-based Graph Neural Network for Human Action Recognition: A Review (Feng et al., 2022)

- **Field of Establishment:** Survey Research, Graph Neural Networks, Human Action Recognition

- **Problem:** Perkembangan pesat metode GNN untuk skeleton HAR sulit diikuti, kurangnya systematic overview tentang tren terkini dan future directions dalam skeleton-

based recognition

- **Solution:** Comprehensive literature survey metode GNN-based HAR (2020-2022), klasifikasi arsitektur terkini, analisis representasi graf, dan identifikasi tantangan riset masa depan

- **Contribution:** Roadmap sistematis untuk riset skeleton HAR, identifikasi gap penelitian, dan konfirmasi viabilitas real-time pose detection menggunakan OpenPose, PoseNet, dan NuiTrack

- **Evaluation:** Analisis komparatif hasil SOTA pada dataset standar, sintesis tren metodologi, dan proyeksi arah riset future work dalam skeleton-based recognition

5. Human Action Recognition Based on Skeleton Information and Multi-Feature Fusion (Wang et al., 2023)

- **Field of Establishment:** Computer Vision, Exercise Recognition, Multi-modal Feature Fusion

- **Problem:** Mengembangkan sistem pengenalan gerakan (khususnya exercise/fitness) yang menggabungkan multiple feature sources untuk mencapai akurasi tinggi dengan computational cost rendah

- **Solution:** OpenPose yang dioptimalkan dengan Oct-MobileNet (octave convolution + attention mechanism) untuk pose detection ringan, dikombinasikan dengan fusion fitur spasial-geometrik dan temporal melalui sliding window

- **Contribution:** Arsitektur lightweight yang mampu mengklasifikasi dan mengevaluasi kualitas gerakan kompleks (BaDuanJin exercises) dengan latency rendah, demonstrasi efektivitas multi-feature fusion

- **Evaluation:** Pengujian pada dataset BaDuanJin menunjukkan akurasi tinggi dalam pengenalan dan penilaian kualitas gerakan dengan maintaining real-time performance

6. MoveEnet: Online High-Frequency Human Pose Estimation With an Event Camera (Goyal et al., 2023)

- **Field of Establishment:** Human Pose Estimation,

Event-based Vision, Real-Time Processing

- **Problem:** Mengembangkan sistem estimasi pose ultra-high-frequency menggunakan event camera, dengan relevansi pada validasi efisiensi model MoveNet untuk aplikasi HPE real-time

- **Solution:** Penggunaan arsitektur MoveNet-Lightning sebagai backbone HPE karena sifatnya yang ringan dan mendukung inferensi berfrekuensi tinggi dalam sistem berbasis event camera.

- **Contribution:** Validasi dan dokumentasi comprehensive tentang MoveNet-Lightning sebagai solusi HPE tercepat untuk single-person pose estimation dengan akurasi tinggi

- **Evaluation:** Sistem dilaporkan mencapai frekuensi hingga 100 Hz pada perangkat dengan GPU dan sekitar 20 Hz pada perangkat CPU-only, dengan akurasi 89% pada evaluasi yang dilaporkan.

7. Are Spatial-Temporal Graph Convolution Networks for HAR Over-Parameterized? (Xie et al., 2025)

- **Field of Establishment:** Model Complexity Analysis, Graph Convolutional Networks, Efficient Deep Learning

- **Problem:** Menganalisis apakah kompleksitas model ST-GCN modern memberikan improvement yang signifikan dibanding cost komputasinya, questioning trend toward architectural complexity

- **Solution:** Systematic evaluation berbagai GCN architectures dalam controlled setting, memperkenalkan ST-GCN++ sebagai simplified baseline dengan focus pada good practices

- **Contribution:** Revelation bahwa architectural complexity memberikan marginal gains (<2% pada NTU-60), sementara data augmentation dan good practices memberikan impact signifikan

- **Evaluation:** Comprehensive comparison FLOPs, parameters, dan accuracy across multiple SOTA models

menunjukkan ST-GCN++ mencapai competitive performance dengan complexity jauh lebih rendah

8. Improving esports viewing experience through hierarchical scene detection and tracking (Joo et al., 2025)

- **Field of Establishment:** eSports Technology, Computer Vision in Gaming, Automated Broadcasting

- **Problem:** Meningkatkan viewer experience dalam eSports melalui automated camera control yang intelligent, mengatasi masalah scene transition yang abrupt dan tracking yang tidak stabil

- **Solution:** Hierarchical system combining Vision Transformer untuk high-accuracy scene detection dengan lightweight tracking system untuk smooth transitions dan camera stability

- **Contribution:** Automated observer system yang significantly superior dibanding conventional approaches dalam user preference studies, dengan maintain >30 FPS untuk real-time broadcasting

- **Evaluation:** User studies menunjukkan clear preference untuk hierarchical system, dengan quantitative metrics IoU untuk scene accuracy dan qualitative assessment untuk viewing experience

9. A Lightweight Skeleton-Based 3D-CNN for Real-Time Fall Detection and Action Recognition (Noor & Park, 2023)

- **Field of Establishment:** Healthcare AI, Real-Time Safety Systems, Skeleton-based Recognition

- **Problem:** Mengembangkan sistem deteksi jatuh dan pengenalan aksi yang lightweight namun robust terhadap occlusion dan challenging real-world conditions

- **Solution:** Pipeline terintegrasi dengan dataset augmentation untuk pose yang challenging + lightweight 3D-CNN yang memproses temporal sequence keypoint skeleton untuk dual-task classification

- **Contribution:** Demonstrasi successful integration

pose estimation dengan action recognition dalam single lightweight model untuk safety-critical applications

- **Evaluation:** Superior performance dibanding baseline methods pada real-world video data untuk fall detection dan general action recognition dengan computational efficiency

10. 3D Skeleton-Based Action Recognition: A Review (Liu et al., 2025)

- **Field of Establishment:** Survey Research, 3D Human Action Recognition, Computer Vision Pipeline Design

- **Problem:** Kurangnya comprehensive overview tentang end-to-end pipeline dalam 3D skeleton-based action recognition, dengan focus pada practical implementation aspects

- **Solution:** Systematic survey covering seluruh pipeline stages: data acquisition, preprocessing, representation methods, augmentation techniques, hingga arsitektur spatio-temporal modeling

- **Contribution:** Holistic framework untuk understanding critical components dalam skeleton HAR systems, emphasis pada importance of preprocessing dan data representation choices

- **Evaluation:** Comprehensive analysis of state-of-the-art methods across different pipeline stages dengan practical guidance untuk implementation decisions

2.3.2 Relevansi untuk Penelitian Tekken 8

1. **Architecture Selection:** Penelitian menunjukkan bahwa model sederhana seperti ST-GCN++ dengan *good practices* dapat mencapai performa kompetitif dengan kompleksitas yang lebih rendah. Temuan tersebut relevan bagi pengembangan sistem yang mempertimbangkan efisiensi komputasi (Duan, Wang, et al., 2022; Xie et al., 2025)

2. MoveNet-Lightning tetap relevan sebagai baseline awal untuk pose estimation cepat, tetapi pipeline akhir penelitian ini menggunakan YOLO11l-Pose karena eksperimen menunjukkan kebutuhan utama sistem bukan hanya latency rendah, melainkan kestabilan skeleton terhadap karakter target dalam kondisi close contact, overlap, dan side switch. Dengan demikian, kontribusi studi MoveNet diposisikan sebagai pembanding awal, sedangkan YOLO11l-Pose menjadi model pose utama yang digunakan pada implementasi akhir (Goyal et al., 2023; Jo & Kim, 2022).
3. Multi-Feature Integration: Konsep fusion fitur dari Wang et al. dapat diadaptasi untuk menggabungkan skeleton recognition dengan input annotation (Bryan: f, b+2) (L. Wang et al., 2023).
4. Real-Time Constraints: Noor et al. mendemonstrasikan implementasi skeleton-based recognition pada embedded systems dengan parameter <1M, memberikan benchmark untuk optimization (Noor et al., 2024).
5. eSports Context: Joo et al. memberikan wawasan tentang aplikasi computer vision dalam gaming context, khususnya untuk meningkatkan pengalaman penonton (Joo et al., 2025).

2.3.3 Kesimpulan Strategis

Literatur menunjukkan tren menuju simplification dengan optimization dalam skeleton-based action recognition (Duan, Wang, et al., 2022; Noor et al., 2024; Xie et al., 2025). Untuk implementasi Tekken 8, hasil studi pustaka dan eksperimen awal mengarah pada pipeline modular yang memisahkan deteksi pemain, tracking identitas, pose estimation, dan action recognition. Kombinasi YOLO detector, OC-SORT, ROI crop, YOLO11l-Pose, dan ST-GCN++ dipilih karena memberikan keseimbangan yang lebih sesuai antara stabilitas skeleton, kecepatan inference, dan kebutuhan anotasi visual pada video fighting game.

BAB III Metodologi Penelitian

3.1 Gambaran Umum Objek Penelitian



Gambar 3.1: Metodologi Penelitian

Objek penelitian ini berfokus pada *gameplay* dari permainan video **Tekken 8**. Tekken 8 adalah *fighting game* 3D yang memiliki dinamika pergerakan karakter yang sangat cepat (*fast-paced*) dan kompleks. Setiap karakter memiliki ratusan *moveset* (kumpulan gerakan) yang unik, yang terdiri dari serangan *high*, *mid*, *low*, bantingan (*throw*), serta kombinasi serangan (*combo*).

Dalam konteks kompetitif maupun pembelajaran bagi pemain pemula, identifikasi gerakan lawan secara *real-time* menjadi tantangan utama. Berdasarkan observasi dan pengalaman pribadi penulis, penonton atau pemain sering mengalami kesulitan kognitif dalam mengenali *input* gerakan spesifik yang dieksekusi oleh pemain profesional. Saat ini, proses identifikasi tersebut masih dilakukan secara manual, yaitu dengan mengandalkan hafalan visual atau melihat tayangan ulang (*replay*) untuk mencocokkan gerakan dengan *frame data*.

Sistem yang dikembangkan dalam penelitian ini bertujuan untuk mengotomatisasi proses identifikasi manual tersebut menggunakan pendekatan *Computer Vision* dan *Human Action Recognition* (HAR). Sistem akan menganalisis video *gameplay practice mode* untuk mengenali aksi dari dua karakter yang bertarung dan menampilkan anotasi *input* secara *real-time*.

Objek penelitian ini adalah rekaman *gameplay* Tekken 8 dalam kondisi *practice mode* yang menampilkan Bryan Fury sebagai karakter target. Data penelitian dibatasi pada 16 gerakan terpilih yang direkam secara terkontrol dan dianotasi secara manual. Pemilihan *practice mode* bertujuan untuk menjaga

konsistensi visual, posisi karakter, serta kemunculan gerakan, sehingga setiap aksi dapat diamati, dipotong, dan diberi label dengan lebih terukur. Data tersebut kemudian digunakan untuk mengevaluasi pipeline visual-only yang mencakup deteksi pemain, tracking, ROI crop, pose estimation, pembentukan skeleton, klasifikasi gerakan, dan overlay anotasi.

3.2 Teknik Pengumpulan Data

Untuk mendukung pembangunan sistem pengenalan aksi ini, teknik pengumpulan data dilakukan melalui tahapan eksperimental dan studi literatur yang mendalam.

3.2.1 Survei Pendahuluan (Preliminary Survey)

Sebelum pengembangan, dilakukan survei eksploratif untuk memvalidasi problem statement dan kebutuhan pengguna menggunakan pendekatan mixed-methods di tiga kanal:

- Google Form (N=21): kuesioner terstruktur dengan skala Likert untuk pengukuran kuantitatif.
- Reddit r/Tekken Poll (N=22): voting biner untuk triangulasi temuan.
- Reddit r/Tekken Discussion: pertanyaan terbuka untuk pendalaman kualitatif.

Tabel 3.1: Overview Pengumpulan Data Survei Multi-Platform

Platform	Metode Pengumpulan	Tipe Data	N	Periode Pengumpulan	Distribusi
Google Form	Kuesioner terstruktur dengan skala Likert 5-point	Kuantitatif (rating 1-5) + Kualitatif (open-ended)	21	26 Sept - 30 Okt 2025	WhatsApp groups, Discord Tekken Indonesia,

					Email kampus
Reddit r/Tekken (Poll)	Binary voting poll	Kuantitatif (Yes/No)	22	24 Nov 2025 - 1 Des 2025	Public post di subreddit r/Tekken (~150K members)
Reddit r/Tekken (Discussion)	Open discussion thread	Kualitatif (expert testimony & community responses)	2300+ views, 4 quality responses analyzed	17 Nov 2025	Public post di subreddit r/Tekken
TOTAL	Mixed-methods	Kuantitatif + Kualitatif	43	~1 bulan	Multi-channel distribution

Total responden unik: 43 orang, dengan convenience sampling dan distribusi snowball melalui WhatsApp, Discord, dan Reddit. Temuan menunjukkan konsistensi lintas kanal dan keselarasan antara data kuantitatif-kualitatif (validitas konvergen), sehingga memperkuat keyakinan awal terhadap problem statement. Evaluasi akhir akan menggunakan metrik teknis objektif, bukan survei subjektif. Survei dilakukan dengan prinsip voluntary participation dan anonymity, sesuai etika penelitian observasional.

3.2.2 Studi literatur dan Eksperimen Awal

Tahap eksperimen awal dilakukan untuk menentukan pipeline yang paling sesuai dengan karakteristik visual Tekken 8. Percobaan pertama

dilakukan menggunakan LSTM dan MediaPipe, tetapi pendekatan ini kurang memadai karena model harus dilatih dari awal dan dataset yang tersedia masih terbatas. Selain itu, variasi animasi game dan efek visual membuat ekstraksi fitur manual sulit dipertahankan secara stabil.

Eksperimen kemudian beralih ke MoveNet MultiPose karena model ini mampu mendeteksi beberapa pose dalam satu frame dengan 17 keypoint COCO. Namun, berdasarkan eksperimen awal, penggunaan keypoint dari lebih dari satu karakter sebagai input action recognition membuat model menerima representasi skeleton multi-person dalam satu window temporal. Dalam kondisi tersebut, pola yang dipelajari cenderung merepresentasikan hubungan dan perubahan posisi antar-skeleton sebagai satu kesatuan, bukan gerakan individu dari karakter target secara terpisah. Hal ini kurang sesuai dengan kebutuhan penelitian Tekken 8, karena kelas yang ingin dikenali adalah gerakan spesifik Bryan Fury, bukan interaksi dua karakter secara keseluruhan. Oleh karena itu, pipeline perlu memisahkan identitas pemain terlebih dahulu melalui deteksi, tracking, dan ROI crop sebelum skeleton karakter target dikirimkan ke model klasifikasi.

Untuk mengatasi masalah tersebut, pipeline dikembangkan menjadi modular. Sistem terlebih dahulu mendeteksi pemain menggunakan YOLO, kemudian mempertahankan identitas pemain menggunakan tracker. Setelah bounding box pemain diperoleh, sistem melakukan ROI crop agar pose estimation hanya berfokus pada karakter target. Pada tahap akhir eksperimen, MoveNet SinglePose digantikan oleh YOLO11l-Pose karena hasil keypoint lebih stabil pada banyak sequence dan model tersebut menghasilkan bounding box serta keypoint dalam satu output pose detection. Pipeline final yang digunakan adalah YOLO detector, OC-SORT tracking, ROI crop, YOLO11l-Pose, dan ST-GCN++.

3.2.3 Pengumpulan Dataset dan Anotasi (Custom Annotator)

Karena tidak tersedianya dataset publik untuk gerakan spesifik Tekken 8, penulis melakukan pengumpulan data primer dari rekaman gameplay practice mode. Practice mode digunakan sebagai sumber utama karena gerakan dapat direkam secara terkontrol, sehingga variasi visual, posisi karakter, dan kemunculan gerakan dapat dijaga lebih konsisten selama proses anotasi dan pembentukan dataset.

Penulis mengembangkan custom annotator untuk memberi label pada potongan video berdasarkan start_frame, end_frame, player, character, move, start_time, end_time, duration_frames, dan annotation_type. Format ini dipilih agar data anotasi dapat langsung dipetakan ke sequence skeleton dan digunakan untuk membangun window pelatihan ST-GCN++.

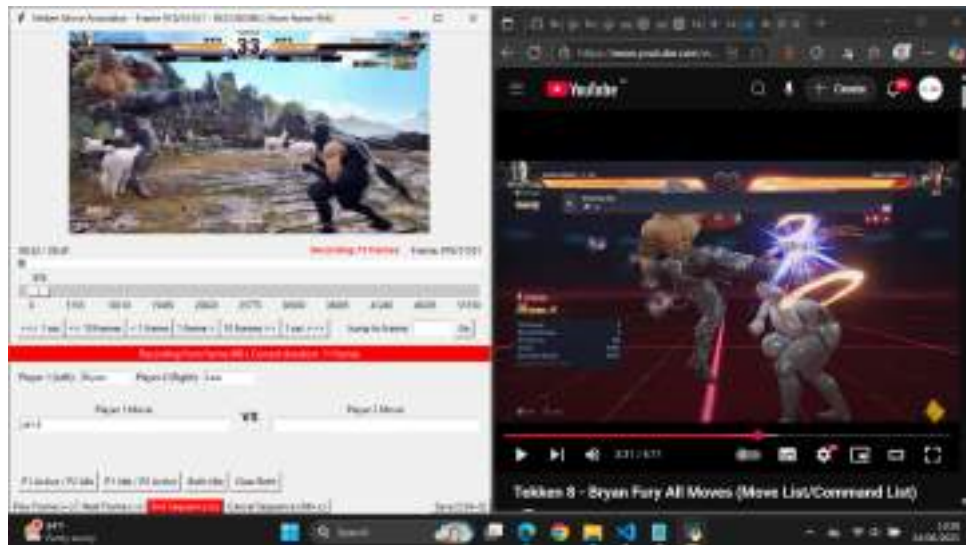
Dataset akhir tidak hanya berisi gerakan target, tetapi juga idle class. Penambahan idle class diperlukan karena pada inference video panjang terdapat banyak frame ketika karakter tidak sedang melakukan salah satu gerakan target. Tanpa idle class, model cenderung memaksa frame idle atau transisi ke salah satu label gerakan yang tersedia.

Rekaman sumber diakuisisi dengan target 60 FPS. Setelah proses penggabungan, video kerja memiliki frame rate 29,97 FPS. Frame rate tersebut dipertahankan secara konsisten selama anotasi, ekstraksi skeleton, pembentukan dataset, dan pengujian agar indeks frame, timestamp, serta rolling window tetap selaras. Dengan demikian, 60 FPS pada Bab I merujuk pada frame rate native Tekken 8, sedangkan 29,97 FPS merupakan frame rate video yang diproses oleh pipeline.

Contoh struktur data JSON yang dihasilkan dari proses anotasi adalah sebagai berikut:

```
"p2_sequence_151": {  
  "player": "player2",  
  "character": "Bryan",  
  "move": "f, b+2",  
  "start_frame": 7220,  
  "end_frame": 7227,  
  "start_time": 240.90733333333333,  
  "end_time": 241.14090000000002,  
  "duration_frames": 8,  
  "annotation_type": "single_character"  
},
```

Gambar 3.2: Contoh Anotasi Json



Gambar 3.3: Aplikasi Custom Annotator

Selain dataset anotasi gerakan untuk action recognition, penelitian ini juga menyusun dataset deteksi pemain untuk melatih YOLO detector. Data detector direkam secara mandiri dari gameplay Tekken 8 dalam kondisi practice mode dengan variasi lima arena dan kombinasi kostum yang berbeda. Rekaman dibuat dalam dua skenario orientasi pemain, yaitu 16 gerakan Bryan Fury sebagai Player 1 melawan Law sebagai Player 2, serta 16 gerakan Bryan Fury sebagai Player 2 melawan Law sebagai Player 1. Variasi ini digunakan agar detector tidak hanya belajar posisi karakter pada satu sisi layar, tetapi juga mengenali bentuk karakter pada perubahan arena, kostum, dan orientasi kiri-kanan.

Frame dari rekaman tersebut kemudian dianotasi secara manual menggunakan MakeSense.ai dan diekspor ke format YOLO bounding box dengan satu kelas, yaitu *fighter*. Dataset detector terdiri dari 511 frame yang dibagi menjadi 408 data train dan 103 data validation. Dari keseluruhan dataset, terdapat 493 frame berlabel dengan total 986 bounding box, sedangkan 18 frame memiliki label kosong. Dataset ini digunakan untuk melatih YOLO11m sebagai detector dalam pipeline, sehingga sistem dapat memperoleh bounding box karakter sebelum tahap tracking, ROI crop, pose estimation, dan klasifikasi gerakan.



Gambar 3.4: Anotasi manual dataset training YOLO11m

3.3 Analisis Sistem

Analisis sistem dilakukan terhadap alur kerja pengolahan data video menjadi prediksi gerakan. Berdasarkan evaluasi dari metode sebelumnya, arsitektur sistem saat ini dirancang untuk menangani masalah identifikasi karakter yang dinamis.

3.3.1 Gambaran Sistem (System Overview)

Sistem yang dikembangkan menggunakan arsitektur pipeline bertingkat. Tahap pertama adalah object detection untuk mendeteksi posisi pemain pada frame video. Hasil deteksi kemudian diproses oleh OC-SORT agar identitas pemain dapat dipertahankan meskipun terjadi side switch, gerakan cepat, atau oklusi singkat. Setelah identitas pemain diperoleh, sistem melakukan ROI crop berdasarkan bounding box target agar proses pose estimation lebih fokus pada karakter yang sedang dianalisis.

Pada tahap pose estimation, sistem menggunakan YOLO11l-Pose dengan imsz=384 untuk mengekstraksi keypoint COCO-17. Model ini dipilih karena memberikan keseimbangan terbaik antara stabilitas keypoint dan latency dibandingkan varian YOLO11s-Pose, YOLO11m-Pose, dan YOLO11x-Pose. Hasil keypoint kemudian disusun menjadi sequence skeleton dengan fixed 30-frame window dan diklasifikasikan menggunakan ST-GCN++. Output akhir sistem berupa overlay label gerakan pada video.

Alur pipeline final dimulai dari input video Tekken 8, YOLO detector, OC-SORT tracking, ROI crop, YOLO11l-Pose, pose candidate selection, confidence filtering, visual QA skeleton, fixed 30-frame window, ST-GCN++, dan overlay anotasi. Dengan memisahkan setiap tahap, kesalahan sistem dapat dianalisis secara modular sehingga kegagalan output akhir tidak langsung dianggap sebagai kegagalan classifier.

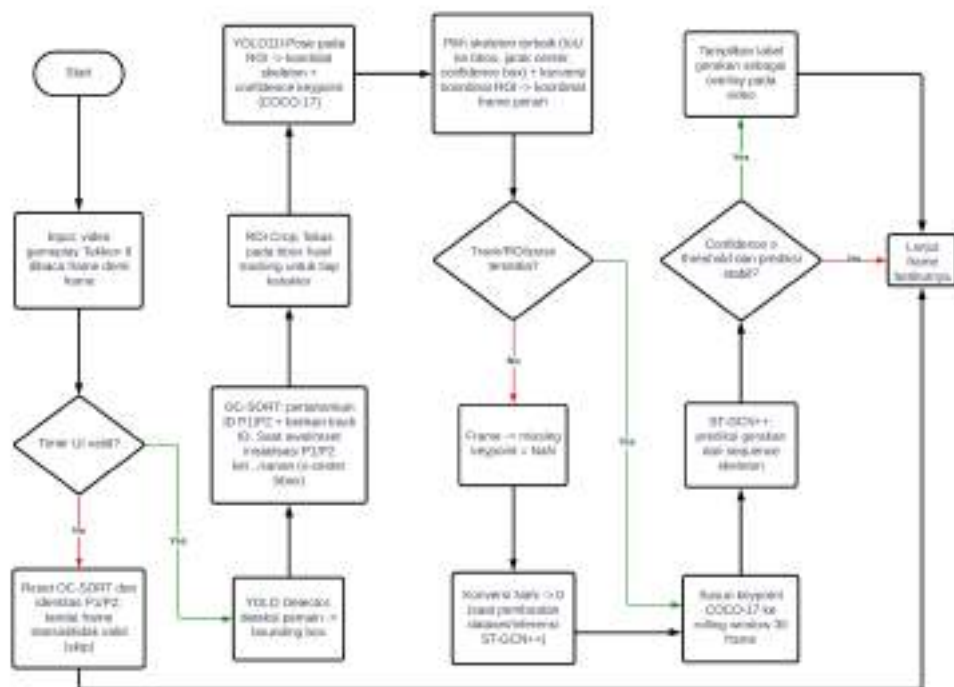
3.3.2 Analisis Fungsional (Functional Analysis)

- **Fungsi Deteksi:** sistem mampu mendeteksi bounding box karakter dari frame video dan membedakannya dari latar belakang, efek visual, serta elemen antarmuka game.
- **Fungsi Pelacakan Identitas:** sistem menyediakan mekanisme pelacakan untuk mempertahankan identitas Player 1 dan Player 2 antar-frame. Pada eksperimen awal, pelatihan detector dengan data video pertandingan menunjukkan potensi peningkatan stabilitas pada kondisi side switch, oklusi, throw, dan tailspin. Namun, karena evaluasi utama penelitian ini dibatasi pada rekaman practice mode, kemampuan tersebut tidak dijadikan klaim utama dan diposisikan sebagai arah pengembangan melalui perluasan dataset pertandingan nyata.
- **Fungsi ROI Crop:** sistem mengambil area karakter berdasarkan bounding box target agar pose estimation lebih fokus pada karakter yang dilacak dan mengurangi gangguan dari background.
- **Fungsi Pose Candidate Selection:** sistem memilih kandidat skeleton terbaik ketika YOLO-Pose menghasilkan lebih dari satu kandidat pose dalam ROI. Pemilihan dilakukan dengan menghitung skor gabungan yang mempertimbangkan IoU antara bounding box kandidat pose dan bounding box target hasil tracking, jarak center kedua bounding box, serta confidence pose. Fungsi ini bertujuan mengurangi risiko skeleton berpindah ke karakter lawan ketika kedua karakter saling berdekatan atau overlap.
- **Fungsi Validasi Kualitas Skeleton:** sistem menggunakan confidence keypoint, deteksi missing keypoint, dan evaluasi visual overlay untuk menilai kualitas skeleton. Pada pembentukan idle

class, sistem juga menggunakan threshold confidence dan nilai motion rendah untuk memilih window idle.

- **Fungsi Windowing Dataset:** sistem membentuk sequence 30 frame agar format data pelatihan sesuai dengan cara model digunakan pada rolling-window inference.
- **Fungsi Klasifikasi:** sistem memetakan urutan keypoint COCO-17 ke dalam label gerakan menggunakan ST-GCN++.
- **Fungsi Overlay Output:** sistem menampilkan label gerakan yang memiliki confidence cukup tinggi pada video hasil inference.

3.3.3 Diagram Aktivitas (Activity Diagram)



Gambar 3.5: Activity Diagram Proses Inferensi Pengenalan Gerakan Tekken 8 Berbasis Skeleton

1. Input berupa video gameplay Tekken 8 dibaca frame demi frame. Pada setiap frame, sistem memeriksa kondisi transisi permainan

melalui keberadaan elemen timer UI. Apabila timer UI tidak terdeteksi melebihi batas tertentu, frame dianggap berada pada kondisi transisi atau tidak valid, sehingga sistem melakukan reset terhadap OC-SORT dan identitas P1/P2.

2. Sistem menjalankan YOLO detector untuk memperoleh bounding box pemain. Hasil deteksi ini digunakan untuk menentukan posisi karakter pada setiap frame sebelum dilakukan proses pelacakan dan ekstraksi pose.
3. Bounding box diproses menggunakan OC-SORT untuk mempertahankan identitas P1 dan P2. OC-SORT memberikan track ID pada setiap karakter yang terdeteksi. Pada awal proses atau setelah terjadi reset, sistem menginisialisasi identitas P1 dan P2 berdasarkan urutan posisi karakter dari kiri ke kanan menggunakan nilai x-center bounding box.
4. Sistem melakukan ROI crop pada masing-masing karakter berdasarkan bounding box target. Bounding box hasil tracking digunakan sebagai area fokus agar proses pose estimation hanya dilakukan pada wilayah karakter yang relevan, sehingga gangguan dari area lain pada frame dapat dikurangi.
5. YOLO11-Pose dijalankan pada ROI untuk menghasilkan kandidat skeleton dan confidence keypoint. Model pose estimation menghasilkan keypoint COCO-17 pada masing-masing ROI beserta nilai confidence yang menunjukkan tingkat keyakinan terhadap titik-titik skeleton yang terdeteksi.
6. Sistem memilih kandidat skeleton terbaik berdasarkan IoU terhadap bounding box target, jarak center, dan confidence box. Setelah kandidat skeleton dipilih, koordinat keypoint yang awalnya berada pada ruang ROI dikonversi kembali ke koordinat frame penuh agar posisi skeleton sesuai dengan posisi karakter pada video asli.
7. Ketika track, ROI, atau pose tidak tersedia, sistem merepresentasikan frame tersebut sebagai missing keypoint dalam bentuk NaN. Nilai NaN digunakan sebagai penanda bahwa data keypoint pada frame tersebut tidak valid atau tidak tersedia. Pada tahap preprocessing dataset atau inference ST-GCN++, nilai missing tersebut dikonversi menjadi 0 agar dapat diproses oleh model.

8. Keypoint COCO-17 disusun menjadi rolling window berukuran 30 frame. Sequence skeleton dari beberapa frame berurutan dikumpulkan untuk membentuk input temporal, sehingga model dapat mempelajari pola gerakan berdasarkan perubahan pose dari waktu ke waktu.
9. ST-GCN++ melakukan prediksi gerakan berdasarkan sequence skeleton. Hasil prediksi kemudian diperiksa menggunakan confidence threshold dan stabilitas prediksi. Prediksi yang memenuhi kriteria tersebut ditampilkan sebagai overlay pada video, sedangkan prediksi yang belum memenuhi kriteria tidak ditampilkan dan sistem melanjutkan proses ke frame berikutnya.

3.4 Analisis Masalah yang Dihadapi

Selama proses pengembangan sistem, ditemukan beberapa masalah utama yang memengaruhi stabilitas pipeline. Masalah tersebut tidak hanya berada pada model klasifikasi, tetapi juga pada tahap deteksi pemain, tracking, ROI, pose estimation, dan pembentukan dataset skeleton.

Masalah interaksi skeleton gabungan muncul ketika dua karakter berada sangat dekat. Pada percobaan awal menggunakan MoveNet MultiPose, skeleton dua karakter dapat terbaca sebagai satu konteks interaksi sehingga model berisiko mempelajari hubungan antar karakter, bukan aksi individu dari karakter target. Solusi yang digunakan adalah memisahkan karakter melalui tracking dan ROI crop sebelum pose estimation.

Masalah pertukaran identitas terjadi karena karakter Tekken sering bertukar posisi kiri-kanan akibat throw, sidestep, combo, atau animasi berputar. Pendekatan berbasis posisi kiri-kanan saja tidak cukup aman. Pada percobaan awal, ByteTrack belum cukup stabil pada beberapa kondisi overlap, side switch, dan gerakan non-linear. Oleh karena itu, penelitian ini menggunakan OC-SORT untuk mempertahankan identitas pemain antar-frame karena pada pengujian awal memberikan hasil tracking yang lebih stabil untuk skenario tersebut. Selain itu, sistem menerapkan mekanisme reset saat transisi ronde terdeteksi melalui hilangnya timer, agar tracking lama tidak terus digunakan pada scene yang sudah berubah.

Masalah missing detection terjadi ketika bounding box hilang pada gerakan jongkok, gerakan cepat, overlap, akhir ronde, atau efek visual yang menutupi karakter. Dalam pipeline akhir, missing frame tidak selalu dipaksa

diisi dengan skeleton hasil interpolasi karena skeleton palsu dapat lebih merusak model dibanding data kosong yang ditandai dengan confidence 0.

Masalah ROI dan padding muncul karena bounding box yang terlalu ketat dapat memotong tangan atau kaki, sedangkan padding yang terlalu besar dapat memasukkan tubuh lawan. Karena itu, ROI harus cukup luas untuk menangkap tubuh target, tetapi tetap dibatasi agar tidak mencampur karakter lawan.

Masalah pose candidate selection muncul karena YOLO-Pose dapat menghasilkan lebih dari satu kandidat skeleton dalam satu ROI. Kandidat pose dipilih berdasarkan kombinasi IoU terhadap bounding box target, jarak center bounding box, dan confidence pose. Namun, metode ini masih memiliki keterbatasan ketika kandidat yang salah memiliki posisi center dekat dengan target atau confidence tinggi.

Masalah keterbatasan dataset muncul karena model awal hanya mengenal sedikit gerakan. Ketika inference dilakukan pada video panjang, frame idle, transisi, atau gerakan di luar kelas target dipaksa masuk ke salah satu label yang tersedia. Penambahan idle class dan perluasan dataset ke semua gerakan yang dianotasi menjadi solusi yang lebih realistis dibanding menggunakan unknown class yang terlalu luas.

3.5 Metode Pengolahan Data

Data skeleton yang digunakan dalam penelitian ini mengikuti format PoseDataset MMAction2. Keypoint dari YOLO-Pose disesuaikan menjadi format [M, T, V, C] untuk koordinat dan [M, T, V] untuk confidence score. Jumlah keypoint yang digunakan adalah COCO-17, sehingga konsistensi urutan titik, koordinat, dan confidence menjadi bagian penting dari proses pengolahan data.

Pada tahap ekstraksi, missing keypoint dipertahankan sebagai nilai kosong atau NaN. Ketika dataset atau inference membutuhkan format numerik, nilai tersebut dikonversi menjadi 0 dengan confidence 0. Pendekatan ini dipilih karena global interpolation dapat menciptakan skeleton palsu ketika karakter hilang, layar gelap, tracking gagal, atau tubuh tertutup.

Dataset pelatihan dibuat menggunakan fixed 30-frame window agar konsisten dengan mode inference yang membaca rolling window 30 frame terakhir. Selain window utama, temporal offset augmentation digunakan

agar model lebih toleran terhadap pergeseran waktu antara awal gerakan pada anotasi dan posisi window pada inference. Offset -3, 0, dan 6 dipilih karena masih mempertahankan informasi gerakan utama dan recovery tanpa terlalu banyak memasukkan konteks idle.

Idle class ditambahkan sebagai label untuk kondisi ketika karakter tidak sedang melakukan gerakan target. Penambahan ini penting karena pada video panjang, sebagian besar frame dapat berupa stance, transisi, atau jeda pendek. Dengan adanya idle class, model tidak perlu memaksa semua kondisi non-gerakan menjadi salah satu move target.

Konfigurasi akhir pembentukan dataset skeleton menggunakan `positive-window-size 30`, `positive-window-mode center`, `positive-offsets -3, 0, dan 6`, `idle-count 30`, `idle-window-size 30`, `idle-stride 15`, `exclude-margin 15`, serta `min-mean-score 0,35`. Konfigurasi tersebut digunakan untuk menjaga keseimbangan antara variasi temporal, kualitas skeleton, dan kebutuhan `rolling-window inference`.

Dataset skeleton akhir untuk pelatihan ST-GCN++ terdiri atas 510 window yang terbagi ke dalam 17 kelas, yaitu 16 kelas gerakan Bryan Fury dan satu idle class. Setiap kelas memiliki 30 window. Dataset dibagi menggunakan rasio 80:20 dengan seed 42 pada proses pembentukan split, sehingga diperoleh 408 window training dan 102 window validation. Setiap kelas menyumbang 24 window training dan 6 window validation. Variasi temporal yang berasal dari anotasi gerakan yang sama dikelompokkan agar tidak ditempatkan pada split yang berbeda.

Model ST-GCN++ dilatih menggunakan batch size 16. Dataset training diterapkan melalui RepeatDataset sebanyak lima kali pada setiap epoch. Optimizer yang digunakan adalah SGD dengan learning rate awal 0,1, momentum 0,9, dan weight decay 0,0005. Pada implementasinya, opsi Nesterov diaktifkan. Learning rate diatur menggunakan Cosine Annealing dengan `T_max 24` dan `eta_min 0`. Pelatihan dilakukan selama 24 epoch dengan evaluasi validation pada setiap epoch. Checkpoint terbaik dipilih berdasarkan nilai Top-1 validation tertinggi; pada konfigurasi setelah koreksi anotasi, checkpoint terbaik diperoleh pada epoch ke-22.

3.6 Metode Evaluasi

Evaluasi dilakukan secara bertahap pada pipeline akhir agar sumber kegagalan sistem dapat dianalisis dengan jelas. Output akhir sistem tidak hanya dipengaruhi oleh model klasifikasi ST-GCN++, tetapi juga oleh kualitas deteksi pemain, stabilitas tracking, hasil ROI crop, kualitas keypoint dari pose estimation, serta latency keseluruhan pipeline.

Evaluasi deteksi dan tracking dilakukan dengan mengamati kestabilan bounding box dan konsistensi identitas Player 1 dan Player 2 pada video gameplay. Evaluasi ini difokuskan pada apakah bounding box tetap mengikuti karakter target, apakah ID pemain tetap konsisten saat karakter saling mendekat atau overlap, serta apakah sistem dapat kembali stabil setelah terjadi missing detection atau transisi ronde.

Evaluasi pose estimation dilakukan melalui visualisasi overlay skeleton pada hasil video. Tujuannya adalah memastikan keypoint mengikuti tubuh karakter target di dalam ROI, tidak berpindah ke karakter lawan, dan tidak menghasilkan struktur skeleton yang terlalu rusak. Evaluasi ini penting karena ST-GCN++ menerima input berupa sequence keypoint, sehingga kesalahan pose dapat langsung memengaruhi hasil klasifikasi gerakan.

Evaluasi action recognition dilakukan menggunakan hasil validasi internal dari dataset skeleton yang dibentuk dari 16 gerakan terpilih Bryan Fury dan idle class. Dataset dibagi menjadi data train dan validation, kemudian model ST-GCN++ dievaluasi menggunakan metrik seperti akurasi top-1 dan top-5. Karena dataset masih berasal dari ruang lingkup video dan anotasi yang terbatas, hasil validasi diposisikan sebagai evaluasi internal, bukan sebagai klaim generalisasi penuh terhadap semua kondisi pertandingan Tekken 8.

Selain evaluasi kuantitatif, sistem juga dievaluasi secara end-to-end melalui video inference. Evaluasi ini dilakukan dengan menjalankan pipeline lengkap, yaitu object detection, OC-SORT tracking, ROI extraction, YOLO-Pose, rolling-window skeleton sequence, ST-GCN++ classification, dan overlay prediksi pada video gameplay. Hasil video inference digunakan untuk menilai apakah prediksi gerakan tampil pada waktu yang masuk akal, apakah overlay cukup stabil, dan apakah sistem dapat digunakan sebagai prototipe anotasi visual.

Evaluasi runtime dilakukan secara end-to-end untuk mengukur kelayakan pipeline sebagai sistem anotasi video. Ruang lingkup pengukuran dimulai ketika frame dibaca menggunakan OpenCV, dilanjutkan dengan deteksi dan tracking pemain, pose estimation pada ROI kedua pemain, inferensi ST-GCN++, rendering overlay, hingga frame keluaran selesai ditulis. Waktu inisialisasi dan pemuatan model tidak dimasukkan karena proses tersebut hanya dilakukan satu kali sebelum loop inference dimulai.

Pengujian dilakukan terhadap 330 frame berurutan. Tiga puluh frame pertama digunakan sebagai warm-up, sedangkan 300 frame berikutnya digunakan untuk menghitung nilai mean, median, dan persentil ke-95 (P95) dari setiap komponen pipeline. Throughput pipeline dihitung menggunakan persamaan berikut:

$$\text{Pipeline FPS} = 1000 / \text{mean end-to-end latency (ms)}$$

Target real-time ditetapkan berdasarkan frame rate video masukan, yaitu 29,97 FPS. Dengan demikian, sistem dinyatakan memenuhi target apabila throughput pipeline sama dengan atau lebih besar dari 29,97 FPS. Pendekatan ini digunakan agar evaluasi runtime mengacu pada karakteristik video yang benar-benar diproses, bukan pada frame rate native game sebesar 60 FPS.

Selain evaluasi internal terhadap model ST-GCN++, dilakukan evaluasi internal end-to-end dengan membandingkan hasil inferensi video terhadap ground truth anotasi. Evaluasi dibatasi pada split validation yang tersimpan dalam dataset MMAction2, yaitu 102 window yang terdiri atas 96 window gerakan dari 32 event dan 6 window idle. Sistem mencatat probabilitas seluruh kelas dan label Top-1 pada setiap langkah klasifikasi, yang dijalankan setiap tiga frame (predict-every=3). Label mentah kemudian difilter menggunakan confidence threshold 0,75, sedangkan label overlay baru diperbarui setelah kelas yang sama muncul pada empat langkah prediksi berturut-turut (stable-predictions=4); overlay tetap dirender pada setiap frame. Untuk evaluasi window, prediksi dicocokkan dengan titik akhir window validation menggunakan toleransi frame terdekat sebesar dua frame. Untuk evaluasi event, prediksi yang benar dicari sejak frame awal anotasi hingga 0, 5, 10, 15, 20, atau 30 frame setelah frame akhir anotasi; detection delay dihitung relatif terhadap frame akhir anotasi. Metrik yang digunakan meliputi Top-1, Top-5, cross-entropy loss, macro-F1, weighted-F1, prediction coverage, event-level accuracy, event recall, dan detection delay. Frame yang tidak memiliki anotasi eksplisit tidak langsung dianggap

sebagai idle. Karena evaluasi masih menggunakan rekaman yang sama dengan sumber pembentukan dataset, hasil ini diposisikan sebagai evaluasi internal end-to-end dan bukan pengujian generalisasi pada rekaman baru.

BAB IV ANALISIS DAN PEMBAHASAN

4.1 Implementasi Pipeline Sistem

Implementasi sistem dilakukan secara modular agar setiap komponen dapat diuji dan diperbaiki secara terpisah. Pipeline akhir yang digunakan adalah YOLO detector untuk mendeteksi pemain, OC-SORT untuk mempertahankan identitas P1 dan P2, ROI crop untuk memisahkan karakter target, YOLO11l-Pose untuk mengekstraksi keypoint, pose candidate selection, confidence filtering, dan evaluasi visual overlay untuk memeriksa konsistensi skeleton, ST-GCN++ untuk klasifikasi gerakan, dan overlay video untuk menampilkan hasil prediksi.

Keputusan menggunakan pipeline modular didasarkan pada temuan bahwa masalah utama sistem tidak hanya berada pada classifier. Pada beberapa kondisi, output yang salah dapat berasal dari bounding box yang hilang, ID tracker yang berubah, ROI yang terlalu lebar, skeleton yang berpindah ke lawan, atau window inference yang tidak sejajar dengan awal gerakan. Karena itu, evaluasi dilakukan dari tahap awal hingga akhir pipeline.



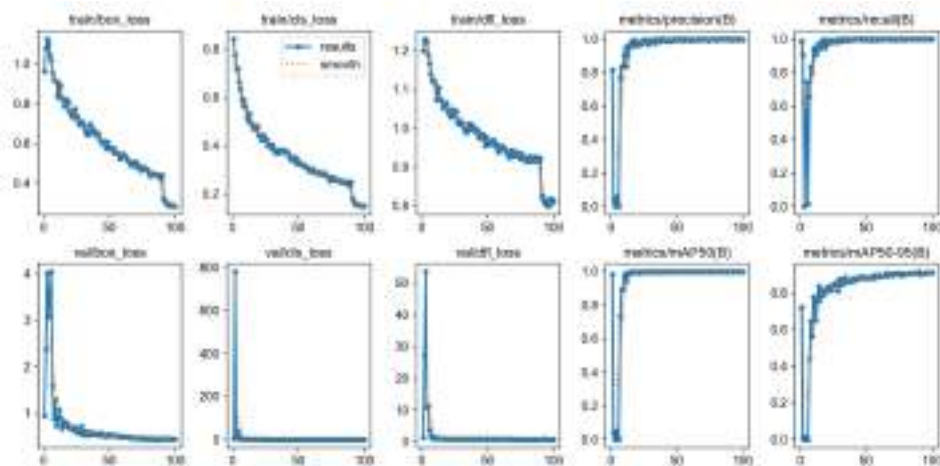
Gambar 4.1: Contoh deteksi pemain menggunakan bounding box



Gambar 4.2: Contoh ROI crop dan pose estimation pada karakter target

4.2 Analisis Deteksi Pemain dan Parameter YOLO

Hasil training YOLO11m pada dataset detector custom menunjukkan performa validasi yang tinggi. Konfigurasi terbaik diperoleh pada epoch 90 dengan precision 0,99981, recall 1,000, mAP50 0,995, dan mAP50-95 0,91381. Hasil ini menunjukkan bahwa detector mampu mengenali kelas fighter dengan baik pada variasi practice mode yang mencakup beberapa arena, kombinasi kostum, dan orientasi pemain. Namun, karena dataset detector tetap berasal dari skenario practice mode yang direkam secara terkontrol, hasil tersebut tidak diklaim sebagai generalisasi penuh terhadap video pertandingan nyata atau skenario turnamen profesional.



Gambar 4.3: Hasil training YOLOv11m

Eksperimen detector menunjukkan bahwa stabilitas bounding box lebih penting daripada sekadar kecepatan inference. Model yang terlalu ringan atau threshold yang terlalu longgar dapat menghasilkan weak detection, duplicate detection, dan bounding box yang kurang stabil pada kondisi close contact. Pada dataset practice-mode, kombinasi confidence 0,5 dan IoU 0,35 dipilih sebagai konfigurasi baseline karena secara visual menghasilkan bounding box yang lebih stabil dibandingkan konfigurasi confidence yang lebih rendah maupun IoU yang lebih tinggi. Konfigurasi ini membantu mengurangi noise pada tracking dan menjaga ROI tetap lebih konsisten terhadap karakter target.

Tabel 4.1: Perbandingan Parameter Confidence dan IoU YOLO

Kombinasi	Temuan	Keputusan
conf=0.35, iou=0.35	Lebih banyak weak detection, tetapi bounding box dan skeleton lebih mudah tidak stabil.	Tidak dipilih.
conf=0.5, iou=0.35	Bounding box lebih bersih, duplicate detection berkurang, dan skeleton lebih stabil saat close contact.	Dipilih sebagai baseline.
conf=0.5, iou=0.5	Hasil cukup mirip, tetapi overlapping box lebih sering bertahan saat dua pemain berdekatan.	Tidak dipilih sebagai konfigurasi akhir.



Gambar 4.4: Perbandingan visual parameter YOLO $conf=0,35/iou=0,35$ dan $conf=0,5/iou=0,35$

Model detector alternatif seperti RT-DETR atau RF-DETR dapat dibandingkan pada penelitian lanjutan, khususnya untuk mengevaluasi apakah pendekatan detector lain dapat meningkatkan stabilitas pada kondisi overlap berat. Namun, perbandingan tersebut perlu mempertimbangkan biaya komputasi, kebutuhan real-time, dan dampaknya terhadap pipeline secara keseluruhan.

4.3 Analisis Tracking Identitas

Raw tracker ID yang dihasilkan oleh algoritma tracking tidak selalu dapat dianggap sebagai identitas semantik absolut. ID numerik dari tracker hanya merepresentasikan track yang sedang dipertahankan oleh algoritma,

bukan pemahaman bahwa suatu objek pasti merupakan Player 1 atau Player 2 secara konseptual. Oleh karena itu, pada pipeline ini ID tracker digunakan sebagai bantuan untuk menjaga konsistensi bounding box antar-frame, sedangkan assignment awal Player 1 dan Player 2 dilakukan berdasarkan posisi kedua karakter ketika dua track valid terdeteksi.

Pada kondisi awal, dua bounding box karakter diurutkan berdasarkan posisi horizontal. Bounding box di sisi kiri diasosiasikan sebagai Player 1 dan bounding box di sisi kanan sebagai Player 2. Setelah ID awal ditentukan, sistem tidak lagi hanya bergantung pada posisi kiri-kanan setiap frame, tetapi mengikuti ID track dari OC-SORT agar identitas pemain lebih stabil ketika terjadi pergerakan, overlap, atau perubahan posisi sementara.

Pada eksperimen awal, ByteTrack dan OC-SORT sempat dibandingkan untuk menjaga identitas Player 1 dan Player 2. ByteTrack bekerja dengan melakukan asosiasi detection box berdasarkan IoU, termasuk memanfaatkan detection dengan confidence rendah agar track tidak mudah terputus. Pendekatan ini efektif pada banyak kasus object tracking, tetapi pada gameplay Tekken 8 terdapat banyak kondisi close contact, overlap karakter, side switch, throw, dan animasi cepat yang menyebabkan bounding box saling bertumpuk atau berubah bentuk secara drastis.

Pada kondisi tersebut, penggunaan weak detection dapat menjadi masalah karena detection dengan confidence rendah tidak selalu merepresentasikan karakter target yang benar. Ketika dua karakter saling overlap, ByteTrack lebih mudah mempertahankan atau mengasosiasikan track ke bounding box yang salah karena keputusan asosiasi sangat bergantung pada overlap antar bounding box. Akibatnya, ID pemain lebih rentan tertukar atau mengikuti karakter lawan.

OC-SORT memberikan hasil yang lebih stabil pada eksperimen ini karena asosiasi track tidak hanya bergantung pada posisi bounding box saat ini, tetapi juga mempertimbangkan konsistensi observasi dan arah pergerakan objek. Pendekatan ini lebih sesuai untuk kasus Tekken 8 yang memiliki gerakan cepat, perpindahan posisi mendadak, dan oklusi singkat. Oleh karena itu, pipeline akhir menggunakan OC-SORT untuk menjaga konsistensi identitas karakter sebelum ROI crop dan pose estimation dilakukan.

Namun, tracking tetap memiliki keterbatasan. Ketika bounding box target hilang, sistem tidak memaksa skeleton untuk tetap diisi, melainkan menandai frame tersebut sebagai missing. Selain itu, ketika transisi ronde terdeteksi melalui hilangnya timer, tracker di-reset agar ID lama tidak terbawa ke scene berikutnya.

4.4 Analisis ROI, Padding, dan Overlap

Kualitas ROI sangat memengaruhi hasil pose estimation. Jika bounding box terlalu ketat, tangan atau kaki dapat keluar dari crop sehingga model pose berpotensi menghasilkan keypoint yang tidak lengkap. Padding dapat membantu menangkap anggota tubuh yang keluar dari bounding box, tetapi padding yang terlalu besar juga berisiko memasukkan tubuh lawan ke dalam ROI.

Pada game fighting seperti Tekken, dua karakter sering berada sangat dekat atau saling bertumpuk. Ketika ROI satu karakter mencakup bagian tubuh karakter lain, skeleton dapat berpindah ke lawan atau keypoint anggota tubuh dapat tertukar. Karena itu, solusi yang lebih sesuai bukan hanya memperbesar crop, tetapi mengombinasikan tracking pemain, ROI yang tidak terlalu lebar, skeleton matching terhadap bounding box target, dan confidence filtering.

4.5 Analisis Pemilihan Model Pose

Beberapa model pose diuji, yaitu MoveNet SinglePose, MoveNet MultiPose, YOLO11s-Pose, YOLO11m-Pose, YOLO11l-Pose, dan YOLO11x-Pose. Pengujian dilakukan pada dua skenario utama: full frame dan ROI pair. Pada full frame, model melakukan satu kali inference terhadap layar penuh. Pada ROI pair, model melakukan dua kali inference, masing-masing untuk P1 dan P2.

ROI pair tidak selalu lebih cepat karena jumlah pemanggilan model menjadi dua kali. Jika ROI tetap di-resize ke `imgsz=384`, maka model tetap memproses tensor dengan ukuran input yang sama walaupun kontennya hanya bagian karakter. Namun, ROI tetap berguna secara visual karena membantu model lebih fokus pada karakter target dan mengurangi gangguan dari background.

Tabel 4.2: Perbandingan Latency Model Pose pada ROI Pair

Model	Median ROI Pair Time	Catatan
MoveNet SinglePose	48,47 ms	Baseline awal, tetapi mudah salah memilih skeleton jika ROI berisi dua karakter.
MoveNet MultiPose	55,78 ms	Dapat menghasilkan beberapa skeleton, tetapi tetap membutuhkan selection logic.
YOLO11s-Pose 384	22,76 ms	Paling cepat, tetapi keypoint masih kurang stabil.
YOLO11m-Pose 384	32,82 ms	Lebih stabil dari YOLO11s-Pose, tetapi belum sebaik YOLO11l-Pose pada beberapa sequence.
YOLO11l-Pose 384	36,65 ms	Trade-off terbaik antara stabilitas keypoint dan latency.
YOLO11x-Pose 384	66,48 ms	Kualitas baik, tetapi terlalu berat untuk kebutuhan cepat.

Berdasarkan benchmark dan observasi visual, YOLO11l-Pose dengan imgsz=384 dipilih sebagai model pose terbaik. Model ini bukan yang tercepat, tetapi memberikan keseimbangan terbaik antara stabilitas skeleton dan latency.



Gambar 4.5: Contoh kegagalan ekstraksi skeleton pada MoveNet



Gambar 4.6: Contoh skeleton YOLO-Pose pada sequence uji

4.6 Analisis Pose Candidate Selection dan Quality Gate

Hasil pengamatan menunjukkan bahwa YOLO-Pose menghasilkan bentuk skeleton yang lebih baik dibanding MoveNet SinglePose, tetapi pada beberapa kondisi close contact, kandidat skeleton YOLO-Pose dapat berpindah ke tubuh lawan. Hal ini terjadi karena YOLO-Pose menghasilkan beberapa kandidat pose dalam satu ROI, sehingga diperlukan mekanisme pose candidate selection.

Pada pipeline ini, kandidat pose dipilih berdasarkan kombinasi IoU terhadap bounding box target, jarak center bounding box, dan confidence pose. Namun, pendekatan ini masih memiliki keterbatasan ketika kandidat yang salah memiliki posisi center dekat dengan target atau confidence tinggi. Pengembangan selanjutnya dapat menambahkan penalti berdasarkan rasio luas bounding box agar kandidat pose yang terlalu kecil atau tidak proporsional terhadap target tidak mudah terpilih.

Visualisasi overlay digunakan sebagai quality assurance untuk melihat apakah skeleton benar-benar mengikuti karakter target. Kualitas skeleton tidak harus sempurna, tetapi harus konsisten terhadap karakter yang benar. ST-GCN++ masih dapat menoleransi sedikit jitter, beberapa keypoint tangan atau kaki yang kurang tepat, dan missing keypoint pendek. Sebaliknya, model sulit menoleransi skeleton yang berpindah ke karakter lawan, skeleton collapse dalam banyak frame, sequence terlalu pendek

dengan frame penting rusak, atau label gerakan benar tetapi pose berasal dari subjek yang salah.

4.7 Analisis Dataset, Windowing, dan Training ST-GCN++

Eksperimen awal ST-GCN++ sempat dilakukan hanya dengan 5 gerakan. Walaupun hasil validasi terlihat cukup baik, video inference panjang masih menghasilkan banyak kesalahan karena model hanya mengenal sedikit kelas. Frame idle, transisi, dan gerakan lain dipaksa masuk ke salah satu label yang tersedia. Kelas unknown sempat dicoba, tetapi hasilnya kurang baik karena unknown terlalu luas dan mencakup terlalu banyak kondisi berbeda.

Fixed 30-frame window digunakan agar training lebih sesuai dengan cara model dipakai pada inference. Pada inference, model membaca rolling window sepanjang 30 frame terakhir. Karena itu, dataset training juga dibuat dalam bentuk window 30 frame, bukan hanya potongan pendek sesuai panjang anotasi asli. Temporal offset augmentation menjadi salah satu temuan penting karena rolling window pada inference tidak selalu tepat berada di awal atau tengah gerakan.

Tabel 4.3: Perbandingan Eksperimen ST-GCN++

Versi	Setting Utama	Top-1	Catatan
v3	5 move awal	90,00%	Validasi baik, tetapi demo masih banyak salah.
v5	5 move + unknown	85,71%	Unknown terlalu dominan.
v8	Semua move + idle	76,47%	Lebih sesuai scope, tetapi belum stabil.
v10	Semua move + idle + offset -6,0,6	97,06%	Peningkatan besar, tetapi masih ada prediksi transisi.
v12	Tanpa global interpolation	97,06%	Lebih bersih dari v10.
v13 awal	Tanpa interpolation + offset -3,0,6	100,00%	Hasil validasi tertinggi sebelum koreksi anotasi SLS.1+2.

Versi	Setting Utama	Top-1	Catatan
v13 fix_sls	v13 dengan koreksi anotasi SLS.1+2	78,43%	Dipakai sebagai hasil setelah koreksi anotasi. Top-5 tetap 100,00%, tetapi Top-1 turun karena confusion antara gerakan stance.

Pada konfigurasi v13 awal, model memperoleh nilai validasi Top-1 sebesar 100,00%. Namun, setelah dilakukan evaluasi visual pada video inference, ditemukan adanya kesalahan anotasi pada salah satu sampel kelas SLS.1+2. Karena anotasi digunakan sebagai ground truth dalam proses pelatihan dan validasi, kesalahan label tersebut dapat memengaruhi interpretasi performa model. Oleh karena itu, anotasi diperbaiki dan model dilatih ulang menggunakan konfigurasi yang sama dengan v13.

Setelah koreksi anotasi, konfigurasi v13 fix_sls memperoleh nilai Top-1 sebesar 78,43% dan Top-5 sebesar 100,00%. Penurunan Top-1 menunjukkan bahwa setelah label diperbaiki, model masih mengalami confusion dalam memilih kelas utama, terutama pada gerakan stance seperti SLS, SWA, dan WS yang memiliki bentuk skeleton relatif mirip. Namun, nilai Top-5 sebesar 100,00% menunjukkan bahwa kelas yang benar masih berada dalam kandidat prediksi terbatas, sehingga kesalahan lebih banyak terjadi pada urutan ranking prediksi pertama, bukan karena model sepenuhnya gagal mengenali pola gerakan.

Dengan demikian, hasil v13 fix_sls lebih tepat digunakan sebagai hasil akhir setelah koreksi anotasi, sedangkan v13 awal diposisikan sebagai hasil sebelum audit anotasi. Temuan ini menunjukkan bahwa kualitas anotasi sangat memengaruhi hasil validasi, terutama pada dataset kecil dengan kelas gerakan yang memiliki kemiripan pose tinggi. Selain itu, temuan ini juga memperlihatkan bahwa evaluasi kuantitatif perlu selalu didampingi evaluasi visual pada video inference agar kesalahan label, wrong skeleton, atau prediksi yang tidak stabil dapat terdeteksi secara lebih jelas.

4.8 Analisis Inference, Threshold, dan Runtime

4.8.1 Konfigurasi Inference dan Evaluasi Runtime

Pada mode inference, sistem menggunakan rolling 30-frame window, yaitu model membaca 30 frame keypoint terakhir untuk menghasilkan prediksi gerakan. Untuk mengurangi beban komputasi, proses klasifikasi ST-GCN++ tidak dijalankan pada setiap frame, melainkan setiap beberapa frame menggunakan parameter predict-every. Pada konfigurasi akhir, nilai default predict-every=3 digunakan, sehingga action recognition dilakukan setiap 3 frame, sementara deteksi, tracking, pose estimation, dan pembaruan rolling window tetap berjalan setiap frame. Untuk menstabilkan overlay, stable-predictions=4 digunakan sehingga label tampilan baru diperbarui setelah kelas yang sama muncul pada empat langkah prediksi berturut-turut.

Action confidence threshold memengaruhi kebersihan tampilan demo. Threshold 0,55 menampilkan lebih banyak prediksi, tetapi beberapa prediksi medium-confidence terlihat lebih noisy. Threshold 0,75 membuat tampilan lebih bersih karena prediksi yang kurang yakin tidak ditampilkan. Pada hasil akhir, threshold 0,75 dipilih karena untuk demonstrasi dan visualisasi sistem, prediksi yang lebih sedikit tetapi lebih yakin lebih mudah dibaca daripada banyak prediksi yang noisy.

Pengujian runtime end-to-end dilakukan pada MacBook Pro dengan chip Apple M5 10-core dan unified memory 24 GB yang menjalankan macOS 26.5.1. YOLO11m dan YOLO11l-Pose dijalankan menggunakan backend MPS, sedangkan ST-GCN++ dijalankan pada CPU. Video uji memiliki frame rate 29,97 FPS sehingga nilai tersebut digunakan sebagai target real-time.

Benchmark dijalankan pada 330 frame berurutan. Tiga puluh frame pertama digunakan sebagai warm-up, sedangkan 300 frame berikutnya digunakan untuk menghitung statistik waktu pemrosesan. Hasil pengukuran setiap komponen pipeline ditunjukkan pada Tabel 4.4.

Tabel 4.4: Hasil Pengukuran Latency End-to-End Pipeline

Komponen pipeline	Mean (ms)	Median (ms)	P95 (ms)
-------------------	-----------	-------------	----------

Pembacaan frame	2,22	2,87	3,07
YOLO detector dan OC-SORT	25,45	25,28	26,00
YOLO11l-Pose pada dua ROI	41,34	37,79	54,17
ST-GCN++	6,29	0,00	19,17
Rendering overlay	2,44	2,45	2,59
Penulisan output	3,14	3,21	3,80
End-to-end	80,89	72,30	96,50



Gambar 4.7: Visualisasi end-to-end latency, pipeline FPS, target FPS, dan perangkat pengujian pada video inference.

Hasil pengujian menunjukkan bahwa komponen dengan waktu pemrosesan terbesar adalah pose estimation pada dua ROI, dengan mean latency sebesar 41,34 ms, diikuti oleh YOLO detector dan OC-SORT sebesar 25,45 ms. Hal ini menunjukkan bahwa optimasi throughput

terutama perlu diarahkan pada model pose, detector, ukuran input, dan frekuensi pemrosesan setiap modul.

Mean end-to-end latency sebesar 80,89 ms menghasilkan throughput sebesar 12,36 FPS. Nilai tersebut setara dengan sekitar 0,41 kali target video sebesar 29,97 FPS. Dengan demikian, konfigurasi akhir belum memenuhi target real-time terhadap frame rate video masukan. Sistem telah mampu menjalankan inference dan menghasilkan visualisasi secara kontinu, tetapi istilah real-time dalam penelitian ini perlu dibatasi sebagai demonstrasi inferensi langsung dan berurutan, bukan kemampuan memproses seluruh 29,97 frame dalam satu detik.

Nilai median latency ST-GCN++ sebesar 0,00 ms (tercatat 0,00079 ms pada data mentah) terjadi karena proses klasifikasi dijalankan setiap tiga frame menggunakan parameter `predict_every=3`. Pada frame yang tidak menjalankan klasifikasi, rolling window tetap diperbarui, tetapi tidak terdapat biaya inferensi ST-GCN++. Data lengkap hasil benchmark disimpan dalam berkas `Bukti projek/e2e_benchmark_normal_power.json`.

Overlay video menampilkan rolling mean 60 frame untuk total latency end-to-end, throughput pipeline, target FPS, rasio terhadap target, serta waktu pemrosesan setiap komponen utama. Status MEETS TARGET hanya ditampilkan apabila throughput pipeline memenuhi target, sedangkan konfigurasi di bawah target ditandai dengan status BELOW TARGET.



Gambar 4.8: Contoh output overlay anotasi pada video inference

4.8.2 Evaluasi Ketepatan End-to-End terhadap Ground Truth

Tabel 4.5: Hasil Evaluasi Internal End-to-End

Metrik	Hasil
Window validation	102
Window berhasil dicocokkan	100
Prediction coverage	98,04%
Top-1 pada window yang cocok	75,00%
Strict Top-1 termasuk missing prediction	73,53%
Top-5 pada window yang cocok	99,00%
Macro-F1	73,06%
Weighted-F1	73,02%
Cross-entropy loss	0,7026
Accepted prediction coverage pada window yang cocok	74,00%
Accuracy pada accepted prediction (74 window)	91,89%
Event-level Top-1	78,13%
Event-level Macro-F1	76,88%
Event-level Top-5	100,00%

Dari 102 window validation, sebanyak 100 window berhasil dipasangkan dengan prediksi inference, sedangkan dua window idle tidak memiliki prediction record yang cukup dekat dengan titik akhir window. Oleh karena itu, prediction coverage mencapai 98,04%. Akurasi Top-1 pada window yang berhasil dicocokkan adalah 75,00%, sedangkan strict Top-1 dengan missing prediction dianggap sebagai kesalahan adalah 73,53%. Nilai Top-5 mencapai 99,00% dengan macro-F1 sebesar 73,06% dan cross-entropy loss sebesar 0,7026.

Penerapan action confidence threshold 0,75 menghasilkan accepted prediction coverage sebesar 74,00%. Namun, prediksi yang berhasil melewati threshold memiliki akurasi sebesar 91,89%. Hasil ini menunjukkan adanya trade-off antara cakupan prediksi dan keandalan label: threshold mengurangi jumlah label yang ditampilkan, tetapi meningkatkan kemungkinan bahwa label yang ditampilkan benar.

Ketika probabilitas dari beberapa temporal offset milik event yang sama dirata-ratakan, event-level Top-1 mencapai 78,13%, macro-F1 sebesar 76,88%, dan Top-5 sebesar 100,00%. Kesalahan terutama ditemukan pada kelas dengan pola skeleton dan fase transisi yang serupa, khususnya SWA.3 dan WS.1, yang memperoleh F1 sebesar 0,00 pada evaluasi window. Temuan tersebut konsisten dengan confusion antar-gerakan yang memiliki bentuk skeleton dan fase transisi serupa.

Tabel 4.6: Event Recall Berdasarkan Toleransi Setelah Akhir Anotasi

Toleransi	Recall label mentah setelah threshold	Recall label stabil pada overlay
0 frame	28,13%	0,00%
+5 frame	62,50%	6,25%
+10 frame	78,13%	31,25%
+15 frame	81,25%	50,00%
+20 frame	81,25%	59,38%
+30 frame	81,25%	59,38%

Pada rentang evaluasi sejak awal anotasi hingga +20 frame setelah akhir anotasi, label mentah setelah confidence threshold 0,75 mendeteksi 26 dari 32 event (recall 81,25%), sedangkan label stabil yang ditampilkan pada overlay mendeteksi 19 dari 32 event (recall 59,38%). Median detection delay label mentah adalah dua frame atau sekitar 0,07 detik, sedangkan median detection delay label stabil adalah sepuluh frame atau sekitar 0,33 detik pada video 29,97 FPS. Perbedaan ini terutama disebabkan oleh stable-predictions=4, yang mensyaratkan kelas yang sama muncul pada empat langkah prediksi berturut-turut sebelum label tampilan diperbarui. Dengan demikian, stabilisasi membuat tampilan lebih konsisten, tetapi menambah keterlambatan dan dapat menyebabkan beberapa gerakan pendek selesai sebelum label stabil muncul.

4.9 Pembahasan Umum

Hasil eksperimen menunjukkan bahwa keberhasilan sistem anotasi moveset tidak dapat ditentukan hanya dari pemilihan classifier. ST-GCN++ dapat bekerja baik ketika sequence skeleton yang diberikan konsisten, tetapi performa akhir sangat dipengaruhi oleh kualitas tahap sebelumnya. Dengan kata lain, error pada detector, tracker, ROI, atau pose estimation dapat

mengalir hingga tahap klasifikasi dan menghasilkan label gerakan yang salah.

Temuan paling penting dari penelitian ini adalah bahwa wrong skeleton lebih berbahaya daripada missing skeleton. Missing skeleton masih dapat ditandai sebagai data kosong dengan confidence 0, sedangkan skeleton yang berasal dari karakter lawan akan memberi sinyal gerakan yang salah kepada model. Oleh karena itu, strategi konservatif pada tracking, pose candidate selection, confidence filtering, dan visual QA lebih sesuai untuk prototipe anotasi visual Tekken 8.

Secara keseluruhan, kombinasi YOLO detector, OC-SORT, ROI crop, YOLO11l-Pose, no global interpolation, all moves + idle class, fixed 30-frame window, temporal offset -3,0,6, ST-GCN++, dan action confidence threshold 0,75 menjadi konfigurasi akhir yang paling seimbang pada penelitian ini.

BAB V KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan evaluasi, penelitian ini berhasil merancang prototipe sistem anotasi gerakan Tekken 8 berbasis skeleton-based action recognition untuk mengenali 16 gerakan terpilih karakter Bryan Fury. Sistem dibangun menggunakan pendekatan visual-only sehingga tidak memerlukan akses internal game, memory reading, maupun modifikasi game.

Pipeline akhir yang digunakan terdiri dari YOLO detector, OC-SORT tracking, ROI crop, YOLO11l-Pose, pose candidate selection, confidence filtering, fixed 30-frame window, dan ST-GCN++. Pembagian pipeline secara modular membantu sistem mempertahankan identitas karakter target dan memudahkan analisis sumber kegagalan ketika output prediksi tidak sesuai.

Dataset skeleton yang berhasil disusun terdiri atas 510 window berdurasi 30 frame dalam 17 kelas, yaitu 16 kelas gerakan dan satu kelas idle. Dataset tersebut dibagi menjadi 408 window training dan 102 window validation menggunakan seed 42, dengan variasi temporal yang berasal dari anotasi yang sama ditempatkan pada split yang sama.

Eksperimen menunjukkan bahwa kualitas skeleton merupakan faktor yang sangat menentukan performa action recognition. YOLO11l-Pose dipilih karena memberikan keseimbangan terbaik antara stabilitas keypoint dan latency dibandingkan model pose lain yang diuji. Selain itu, penambahan idle class, penghapusan global keypoint interpolation, dan temporal offset augmentation -3,0,6 membantu membuat model lebih sesuai dengan kondisi inference video panjang.

Pada eksperimen ST-GCN++, konfigurasi v13 awal sempat memperoleh Top-1 sebesar 100,00% pada validasi internal. Namun, setelah evaluasi visual pada video inference, ditemukan kesalahan anotasi pada salah satu sampel kelas SLS.1+2. Setelah anotasi diperbaiki dan model dilatih ulang dengan konfigurasi yang sama, konfigurasi v13 fix_sls memperoleh Top-1 sebesar 78,43% dan Top-5 sebesar 100,00%. Oleh karena itu, hasil v13 fix_sls digunakan sebagai hasil akhir yang lebih valid, sedangkan nilai 100,00% pada v13 awal diposisikan sebagai hasil sebelum koreksi anotasi.

Hasil tersebut menunjukkan bahwa sistem sudah mampu memasukkan kelas gerakan yang benar ke dalam kandidat prediksi teratas, tetapi masih mengalami confusion pada ranking pertama, terutama pada kelas gerakan stance yang memiliki bentuk skeleton mirip. Dengan demikian, penelitian ini menunjukkan bahwa keberhasilan sistem anotasi moveset tidak hanya ditentukan oleh classifier ST-GCN++, tetapi juga oleh kualitas deteksi, stabilitas tracking, ketepatan ROI, kualitas pose estimation, konsistensi anotasi, dan evaluasi visual terhadap output inference.

Pada evaluasi internal end-to-end terhadap split validation, pipeline memperoleh strict Top-1 sebesar 73,53% dan event-level Top-1 sebesar 78,13%. Dengan toleransi hingga +20 frame setelah akhir anotasi, recall label mentah setelah threshold 0,75 mencapai 81,25%, sedangkan recall label stabil yang ditampilkan pada overlay mencapai 59,38%. Median detection delay masing-masing adalah dua frame atau sekitar 0,07 detik dan sepuluh frame atau sekitar 0,33 detik. Hasil ini menunjukkan bahwa stabilisasi membuat tampilan lebih konsisten, tetapi menurunkan cakupan deteksi dan menambah keterlambatan.

Dari sisi runtime, pipeline memperoleh mean end-to-end latency sebesar 80,89 ms dan throughput sebesar 12,36 FPS. Nilai tersebut setara dengan sekitar 0,41 kali target frame rate video masukan sebesar 29,97 FPS. Dengan demikian, pipeline belum memenuhi target real-time untuk memproses seluruh frame video pada kecepatan aslinya. Meskipun demikian, sistem mampu menjalankan inferensi secara kontinu dan menghasilkan overlay anotasi secara berurutan. Oleh karena itu, hasil penelitian ini lebih tepat diposisikan sebagai prototipe anotasi visual, bukan sebagai sistem real-time penuh. Optimasi lebih lanjut terutama diperlukan pada tahap pose estimation, object detection, dan tracking yang memiliki biaya komputasi terbesar.

Evaluasi modular terhadap tahap deteksi, tracking, dan pose estimation menunjukkan bahwa detector YOLO11m memperoleh precision 0,99981, recall 1,000, mAP50 0,995, dan mAP50-95 0,91381. Sementara itu, stabilitas tracking identitas dan kualitas keypoint dievaluasi secara kualitatif melalui overlay visual, terutama pada kondisi close contact, overlap, dan side switch.

5.2 Keterbatasan Penelitian

Penelitian ini masih memiliki beberapa keterbatasan. Pertama, dataset masih terbatas pada karakter Bryan Fury dan 16 gerakan terpilih, sehingga sistem belum dapat diklaim mengenali seluruh moveset Tekken 8 atau seluruh karakter. Kedua, sistem belum memiliki character recognition otomatis, sehingga label dan pipeline masih diasumsikan sesuai dengan karakter target yang digunakan dalam dataset.

Ketiga, kualitas keypoint masih bergantung pada YOLO11l-Pose. Pada kondisi close contact, overlap berat, crouch, jump, efek visual besar, atau oklusi, skeleton masih dapat patah atau berpindah ke tubuh lawan. Keempat, pengujian utama masih menggunakan video yang sudah direkam dan belum sepenuhnya berupa deployment real-time langsung dari layar pertandingan.

Kelima, hasil validasi internal masih dipengaruhi oleh keterbatasan ukuran dataset dan kemiripan antar-kelas gerakan. Penurunan Top-1 dari v13 awal ke v13 fix_sls setelah koreksi anotasi menunjukkan bahwa kualitas label sangat berpengaruh terhadap interpretasi performa model. Evaluasi end-to-end juga masih menggunakan rekaman yang sama dengan sumber pembentukan dataset, sehingga hasilnya belum membuktikan generalisasi pada sesi rekaman baru. Selain itu, anotasi tidak menyediakan label background/ignore secara kontinu pada seluruh frame; karena itu, precision event untuk keseluruhan video belum dapat dihitung secara valid. Oleh karena itu, seluruh hasil tersebut perlu dibaca sebagai evaluasi internal terhadap kelayakan pipeline, bukan sebagai klaim generalisasi penuh terhadap seluruh kondisi pertandingan Tekken 8.

Keenam, pipeline belum memenuhi target throughput real-time. Benchmark terhadap 300 frame pengukuran setelah 30 frame warm-up menghasilkan mean end-to-end latency sebesar 80,89 ms atau throughput sebesar 12,36 FPS, dibandingkan target video masukan sebesar 29,97 FPS. Pengujian masih menggunakan video yang telah direkam dan mencakup pembacaan frame, deteksi dan tracking, pose estimation, klasifikasi ST-GCN++, rendering overlay, serta penulisan video keluaran. Oleh karena itu, hasil tersebut merepresentasikan pemrosesan video end-to-end dan belum sepenuhnya menggambarkan deployment langsung dari layar game. Pengujian live-screen dan optimasi komponen detector serta pose estimation masih diperlukan sebelum sistem dapat digunakan untuk anotasi pertandingan secara real-time.

Ketujuh, evaluasi tracking dan kualitas pose pada penelitian ini masih dilakukan melalui pengamatan visual dan belum menggunakan metrik kuantitatif khusus untuk mengukur kestabilan identitas maupun ketepatan keypoint.

5.3 Saran Pengembangan

Pengembangan selanjutnya dapat diarahkan pada perluasan dataset ke lebih banyak karakter, lebih banyak gerakan, variasi arena, variasi costume, dan kondisi pertandingan nyata. Dengan dataset yang lebih luas, sistem dapat diuji pada variasi visual yang lebih representatif terhadap pertandingan Tekken 8 sebenarnya.

Pada sisi pose estimation, penelitian lanjutan dapat menambahkan penalti berdasarkan rasio luas bounding box pada pose candidate selection agar kandidat pose yang terlalu kecil atau tidak proporsional terhadap target tidak mudah terpilih. Segmentation juga dapat dipertimbangkan untuk meningkatkan kualitas ROI, meskipun pendekatan tersebut membutuhkan annotation mask, inference yang lebih berat, dan integrasi yang lebih kompleks.

Model detector alternatif seperti RT-DETR atau RF-DETR dapat dibandingkan pada penelitian lanjutan, khususnya untuk mengevaluasi apakah pendekatan detector lain dapat meningkatkan stabilitas pada kondisi overlap berat. Namun, perbandingan tersebut perlu mempertimbangkan biaya komputasi, kebutuhan real-time, dan dampaknya terhadap pipeline secara keseluruhan. Selain itu, mekanisme unknown-move rejection dan real-time move spotting dapat dikembangkan agar sistem tidak hanya mengenali window gerakan yang telah disiapkan, tetapi juga mendeteksi awal dan akhir gerakan secara otomatis.

Pada sisi evaluasi, split dataset berdasarkan video atau sesi rekaman yang berbeda perlu dilakukan agar metrik akurasi lebih mencerminkan kemampuan generalisasi. Evaluasi live-screen inference juga penting untuk mengukur latency end-to-end, stabilitas overlay, dan kelayakan sistem pada skenario penggunaan nyata. Selain itu, audit anotasi perlu dijadikan bagian eksplisit dari workflow dataset karena hasil penelitian menunjukkan bahwa kesalahan label kecil dapat berdampak besar pada interpretasi performa model.

Penelitian selanjutnya dapat menambahkan character recognition otomatis agar pemilihan karakter, daftar gerakan, dan model klasifikasi

tidak lagi bergantung pada penentuan karakter target secara manual. Optimasi runtime juga perlu diprioritaskan pada pose estimation, detector, dan tracking karena komponen-komponen tersebut menyumbang sebagian besar waktu pemrosesan pipeline.

5.4 Implikasi Penelitian

Secara praktis, penelitian ini menunjukkan bahwa anotasi visual berbasis computer vision berpotensi membantu menjembatani kesenjangan antara animasi game dan pemahaman teknis penonton. Overlay gerakan dapat berfungsi seperti subtitle mekanik yang membantu penonton memahami apa yang sedang dilakukan pemain tanpa harus menghafal seluruh moveset terlebih dahulu.

Secara akademik, penelitian ini memperlihatkan bahwa penerapan skeleton-based action recognition pada video game fighting memiliki tantangan yang berbeda dari human action recognition konvensional. Gerakan cepat, close contact, side switch, efek visual, dan overlap karakter membuat kualitas detection, tracking, ROI, dan pose estimation sama pentingnya dengan arsitektur classifier. Dengan demikian, kontribusi penelitian ini berada pada rancangan pipeline, dataset awal, analisis failure mode, dan evaluasi modular untuk konteks fighting game.

DAFTAR PUSTAKA

- Baxter, K. A., Sachdeva, N., & Baker, S. (2025). The Application of Cognitive Load Theory to the Design of Health and Behavior Change Programs: Principles and Recommendations. *Health Education & Behavior*, 52(4), 469–477. <https://doi.org/10.1177/10901981251327185>
- Cao, J., Pang, J., Weng, X., Khirodkar, R., & Kitani, K. (2023). *Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking* (arXiv:2203.14360). arXiv. <https://doi.org/10.48550/arXiv.2203.14360>
- Cao, Z., Simon, T., Wei, S.-E., & Sheikh, Y. (2017). *Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields* (arXiv:1611.08050). arXiv. <https://doi.org/10.48550/arXiv.1611.08050>
- Demetrio, A. (2021, April 24). I wanna make a fighting game! A practical guide for beginners—Part 7. *Medium*. <https://andrea-jens.medium.com/i-wanna-make-a-fighting-game-a-practical-guide-for-beginners-part-7-56f32f706a46>
- Duan, H., Wang, J., Chen, K., & Lin, D. (2022). *PYSKL: Towards Good Practices for Skeleton Action Recognition* (arXiv:2205.09443). arXiv. <https://doi.org/10.48550/arXiv.2205.09443>
- Duan, H., Zhao, Y., Chen, K., Lin, D., & Dai, B. (2022). Revisiting Skeleton-based Action Recognition. *2022 IEEE/CVF Conference on*

Computer Vision and Pattern Recognition (CVPR), 2959–2968.
<https://doi.org/10.1109/CVPR52688.2022.00298>

Esports Charts. (2023). *EVO Japan 2023 Tekken 7 / Statistics*. Esports Charts. <https://escharts.com/tournaments/tekken/evolution-championship-series-japan-2023-t7>

Esports Charts. (2024, May 9). EVO Japan 2024 set regional series records for Street Fighter, Tekken and Guilty Gear. *Esports Charts*. <https://escharts.com/news/evo-japan-2024-series-records>

Esports Charts. (2025, August 19). Esports World Cup 2025 ends as Tekken’s most-watched event ever. *Esports Charts*. <https://escharts.com/news/esports-world-cup-2025-tekken>

Fang, H.-S., Xie, S., Tai, Y.-W., & Lu, C. (2018). *RMPE: Regional Multi-person Pose Estimation* (arXiv:1612.00137). arXiv. <https://doi.org/10.48550/arXiv.1612.00137>

Feng, M., & Meunier, J. (2022). Skeleton Graph-Neural-Network-Based Human Action Recognition: A Survey. *Sensors*, 22(6), 2091. <https://doi.org/10.3390/s22062091>

Goyal, G., Di Pietro, F., Carissimi, N., Glover, A., & Bartolozzi, C. (2023). MoveEnet: Online High-Frequency Human Pose Estimation with an Event Camera. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 4024–4033. <https://doi.org/10.1109/CVPRW59228.2023.00420>

- Gülsoylu, E., Abdelhalim, A., Boztas, D. K., Grasse, O., Jahn, C., Frintrop, S., & Edinger, J. (2025). *Automatic Intermodal Loading Unit Identification using Computer Vision: A Scoping Review* (arXiv:2509.17707). arXiv. <https://doi.org/10.48550/arXiv.2509.17707>
- Jo, B., & Kim, S. (2022). Comparative Analysis of OpenPose, PoseNet, and MoveNet Models for Pose Estimation in Mobile Devices. *Traitement Du Signal*, 39(1), 119–124. <https://doi.org/10.18280/ts.390111>
- Jocher, G., & Qiu, J. (2024). *Ultralytics YOLO11* [Computer software]. <https://github.com/ultralytics/ultralytics>
- Joo, H.-T., Lee, S.-H., Chung, I., & Kim, K.-J. (2025). Improving esports viewing experience through hierarchical scene detection and tracking. *Scientific Reports*, 15(1), 9803. <https://doi.org/10.1038/s41598-025-93692-0>
- Kennedy, J. (2022, May 9). How To Read Frame Data In Fighting Games. *The Gamer*. <https://www.thegamer.com/fighting-games-frame-data-explained/>
- Khan, A., Sohail, A., Zahoora, U., & Qureshi, A. S. (2020). A Survey of the Recent Architectures of Deep Convolutional Neural Networks. *Artificial Intelligence Review*, 53(8), 5455–5516. <https://doi.org/10.1007/s10462-020-09825-6>
- Khanam, R., & Hussain, M. (2024). *YOLOv11: An Overview of the Key Architectural Enhancements* (arXiv:2410.17725). arXiv. <https://doi.org/10.48550/arXiv.2410.17725>

- Kim, J., Kim, K., Kim, H., & Li, S. (2025). Public perception of esports: An examination of esports image and genre differences. *Sport Management Review*, 28(1), 97–120. <https://doi.org/10.1080/14413523.2024.2398836>
- Li, Q., Chen, K., Wang, Q., Wang, F., & Deng, W. (2025). Enhanced YOLOv11N: A Method for Potato Peel Damage Detection. *Food Science & Nutrition*, 13(7), e70576. <https://doi.org/10.1002/fsn3.70576>
- Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C. L., & Dollár, P. (2015). *Microsoft COCO: Common Objects in Context* (arXiv:1405.0312). arXiv. <https://doi.org/10.48550/arXiv.1405.0312>
- Liu, M., Liu, H., Hu, Q., Ren, B., Yuan, J., Lin, J., & Wen, J. (2025). *3D Skeleton-Based Action Recognition: A Review* (arXiv:2506.00915). arXiv. <https://doi.org/10.48550/arXiv.2506.00915>
- Ma, A. (2024, August 29). Is the Execution Barrier of Fighting Games Too High? *SUPERJUMP*. <https://www.superjumpmagazine.com/is-the-execution-barrier-of-fighting-games-too-high/>
- Macey, J., Mattinen, T., & Hamari, J. (2025). Paying, Playing, and Spectating: Relationships Between Digital Game Play and Spectating Esports. *Games and Culture*, 15554120251376073. <https://doi.org/10.1177/15554120251376073>
- Noor, N., Jametoni, F., Kim, J., Hong, H., & Park, I. K. (2024). Efficient Skeleton-Based Action Recognition for Real-Time Embedded Systems. *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition*

Workshops (CVPRW), 5889–5897.

<https://doi.org/10.1109/CVPRW63382.2024.00596>

Noor, N., & Park, I. K. (2023). A Lightweight Skeleton-Based 3D-CNN for Real-Time Fall Detection and Action Recognition. *2023 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2171–2180. <https://doi.org/10.1109/ICCVW60793.2023.00232>

Phalke, D. A., Kotipalli, V., Ranjan, P., Pawar, Y., & Bharat, P. (2025). Artificial Intelligence in Fitness: Pose Estimation and Movement Correction. *Cureus Journal of Computer Science*. <https://doi.org/10.7759/s44389-024-00747-w>

Sweller, J., Van Merriënboer, J. J. G., & Paas, F. (2019). Cognitive Architecture and Instructional Design: 20 Years Later. *Educational Psychology Review*, 31(2), 261–292. <https://doi.org/10.1007/s10648-019-09465-5>

TensorFlow. (2024, March 9). *MoveNet: Ultra fast and accurate pose detection model*. Tensorflow. <https://www.tensorflow.org/hub/tutorials/movenet>

Toshpulatov, M., Lee, W., Lee, S., & Haghighian Roudsari, A. (2022). Human pose, hand and mesh estimation using deep learning: A survey. *The Journal of Supercomputing*, 78(6), 7616–7654. <https://doi.org/10.1007/s11227-021-04184-7>

- Wang, C., & Yan, J. (2023). A Comprehensive Survey of RGB-Based and Skeleton-Based Human Action Recognition. *IEEE Access*, 11, 53880–53898. <https://doi.org/10.1109/ACCESS.2023.3282311>
- Wang, L., Su, B., Liu, Q., Gao, R., Zhang, J., & Wang, G. (2023). Human Action Recognition Based on Skeleton Information and Multi-Feature Fusion. *Electronics*, 12(17), 3702. <https://doi.org/10.3390/electronics12173702>
- Wavu Wiki contributors. (2024, August 5). *Frame*. Wavu Wiki. <https://wavu.wiki/t/Frame>
- Wavu Wiki contributors. (2025, November 11). *Bryan movelist*. Wavu Wiki. https://wavu.wiki/t/Bryan_movelist
- Xie, J., Zhao, Y., Meng, Y., Zhao, H., Nguyen, A., & Zheng, Y. (2025). *Are Spatial-Temporal Graph Convolution Networks for Human Action Recognition Over-Parameterized?* (arXiv:2505.10679). arXiv. <https://doi.org/10.48550/arXiv.2505.10679>
- Yan, S., Xiong, Y., & Lin, D. (2018). Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). <https://doi.org/10.1609/aaai.v32i1.12328>
- Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). *ByteTrack: Multi-Object Tracking by Associating Every Detection Box* (arXiv:2110.06864). arXiv. <https://doi.org/10.48550/arXiv.2110.06864>

LAMPIRAN

Lampiran 1 Custom Anotator untuk tekken (anotator_5.py):

```
import cv2
import json
import os
from tkinter import *
from PIL import Image, ImageTk
from tkinter import filedialog

class Annotator:

    def __init__(self, video_path=None, output_file=None):

        # Allow selecting a video file if none provided

        if video_path is None:

            video_path = filedialog.askopenfilename(title="Select Video",

                                                    filetype=[("Video files",
                                                                    "*.mp4 *.mkv *.webm")])

            if not video_path:

                print("No video selected. Exiting.")

                return

        # Allow selecting/creating an output file if none provided

        if output_file is None:

            output_file = filedialog.asksaveasfilename(title="Save
Annotations As",

                                                        defaultextension=".json",

                                                        filetype=[("JSON
files", "*.json")])

            if not output_file:

                output_file = os.path.splitext(video_path)[0] +
                "_annotations.json"
```

```

self.video_path = video_path

self.output_file = output_file

# Initialize video capture
self.cap = cv2.VideoCapture(video_path)

if not self.cap.isOpened():
    print(f"Error: Could not open video file {video_path}")
    return

# Get video properties
self.total_frames = int(self.cap.get(cv2.CAP_PROP_FRAME_COUNT))
self.fps = self.cap.get(cv2.CAP_PROP_FPS)
self.sequence_start = None
self.annotations = {}

# Load existing annotations if file exists
if os.path.exists(output_file):
    with open(output_file, "r") as f:
        self.annotations = json.load(f)

# Jump to the latest annotated frame if available
if self.annotations:
    self.current_frame = max(seq["end_frame"] for seq in
self.annotations.values())
else:
    self.current_frame = 0

# Set up GUI
self.root = Tk()

```

```

self.root.title("Tekken Move Annotator")

# Display frame

self.display_frame = Frame(self.root)
self.display_frame.pack(side="top", fill="both", expand=True)

self.canvas = Canvas(self.display_frame, width=480, height=270)
self.canvas.pack()

# Slider frame with better controls

slider_frame = Frame(self.root)
slider_frame.pack(fill="x", padx=10, pady=5)

# Add time display and frame info

time_frame = Frame(slider_frame)
time_frame.pack(fill="x", pady=2)

self.time_label = Label(time_frame, text="00:00 / 00:00")
self.time_label.pack(side="left")

# Frame counter and current sequence duration

self.frame_info_label = Label(time_frame, text="Frame: 0/0")
self.frame_info_label.pack(side="right")

# Current sequence frame count (when recording)

self.sequence_frames_label = Label(time_frame, text="", fg="red",
font=("Arial", 8, "bold"))
self.sequence_frames_label.pack(side="right", padx=10)

SLIDER_WIDTH = 600

```

```

        self.mark_canvas = Canvas(slider_frame, width=SLIDER_WIDTH,
height=10, bg="white", highlightthickness=0)

        self.mark_canvas.pack(fill="x", padx=0, pady=(0, 2))

        self.frame_slider = Scale(slider_frame, from_=0,
to=self.total_frames-1,

                                orient="horizontal", length=SLIDER_WIDTH,

                                command=self.update_frame,

                                resolution=1,
tickinterval=self.total_frames//10)

        self.frame_slider.pack(fill="x", padx=0)

        # Jump controls with frame input

        jump_frame = Frame(slider_frame)

        jump_frame.pack(fill="x", pady=5)

        Button(jump_frame, text="<<< 1 sec", command=lambda:
self.jump_frames(-int(self.fps))).pack(side="left")

        Button(jump_frame, text="<< 10 frames", command=lambda:
self.jump_frames(-10)).pack(side="left")

        Button(jump_frame, text="< 1 frame", command=lambda:
self.jump_frames(-1)).pack(side="left")

        Button(jump_frame, text="1 frame >", command=lambda:
self.jump_frames(1)).pack(side="left")

        Button(jump_frame, text="10 frames >>", command=lambda:
self.jump_frames(10)).pack(side="left")

        Button(jump_frame, text="1 sec >>>", command=lambda:
self.jump_frames(int(self.fps))).pack(side="left")

        # Frame jump input

        jump_input_frame = Frame(jump_frame)

        jump_input_frame.pack(side="right", padx=10)

        Label(jump_input_frame, text="Jump to frame:").pack(side="left")

        self.jump_entry = Entry(jump_input_frame, width=8)

```

```

self.jump_entry.pack(side="left", padx=2)

self.jump_entry.bind('<Return>', self.jump_to_frame)

Button(jump_input_frame, text="Go",
command=self.jump_to_frame).pack(side="left", padx=2)

# Status indicator

self.status_frame = Frame(self.root, height=30)

self.status_frame.pack(side="top", fill="x")

self.status_label = Label(self.status_frame, text="Ready",
fg="black", bg="lightgray")

self.status_label.pack(fill="x")

# ...existing code...

# Dual-character input UI

self.input_frame = Frame(self.root)

self.input_frame.pack(side="top", fill="x", padx=10, pady=5)

# Character setup frame

char_setup_frame = Frame(self.input_frame)

char_setup_frame.pack(fill="x", pady=5)

Label(char_setup_frame, text="Player 1 (Left):").pack(side="left")

self.p1_char_entry = Entry(char_setup_frame, width=10)

self.p1_char_entry.pack(side="left", padx=5)

Label(char_setup_frame, text="Player 2 (Right):").pack(side="left")

self.p2_char_entry = Entry(char_setup_frame, width=10)

self.p2_char_entry.pack(side="left", padx=5)

```

```

# Annotation input frame
annotation_frame = Frame(self.input_frame)
annotation_frame.pack(fill="x", pady=5)

# Player 1 annotation
p1_frame = Frame(annotation_frame)
p1_frame.pack(side="left", fill="x", expand=True, padx=5)

Label(p1_frame, text="Player 1 Move:").pack()
self.p1_move_entry = Entry(p1_frame, width=15)
self.p1_move_entry.pack(fill="x")

# Bind key release to check for previous annotations
self.p1_move_entry.bind('<KeyRelease>', self.check_move_history)

# VS separator
vs_frame = Frame(annotation_frame)
vs_frame.pack(side="left", padx=10)

Label(vs_frame, text="VS", font=("Arial", 12,
"bold")).pack(pady=20)

# Player 2 annotation
p2_frame = Frame(annotation_frame)
p2_frame.pack(side="left", fill="x", expand=True, padx=5)

Label(p2_frame, text="Player 2 Move:").pack()
self.p2_move_entry = Entry(p2_frame, width=15)
self.p2_move_entry.pack(fill="x")

# Bind key release to check for previous annotations
self.p2_move_entry.bind('<KeyRelease>', self.check_move_history)

```

```

# Move history display frame

history_frame = Frame(self.input_frame)

history_frame.pack(fill="x", pady=5)

self.history_label = Label(history_frame, text="", fg="blue",
font=("Arial", 8),

                                wraplength=800, justify="left")

self.history_label.pack(fill="x")

# Quick buttons for common states

quick_frame = Frame(self.input_frame)

quick_frame.pack(fill="x", pady=5)

Button(quick_frame, text="P1 Active / P2 Idle",
        command=lambda: self.set_quick_state("active",
"idle")).pack(side="left", padx=2)

Button(quick_frame, text="P1 Idle / P2 Active",
        command=lambda: self.set_quick_state("idle",
"active")).pack(side="left", padx=2)

Button(quick_frame, text="Both Idle",
        command=lambda: self.set_quick_state("idle",
"idle")).pack(side="left", padx=2)

Button(quick_frame, text="Clear Both",
        command=lambda: self.set_quick_state("",
"")).pack(side="left", padx=2)

# Controls

self.btn_frame = Frame(self.root)

self.btn_frame.pack(side="bottom", fill="x")

Button(self.btn_frame, text="Prev Frame (←)",
command=self.prev_frame).pack(side="left")

```

```

        Button(self.btn_frame, text="Next Frame (→)",
command=self.next_frame).pack(side="left")

        self.start_btn = Button(self.btn_frame, text="Start Sequence
(Control+s)", command=self.start_sequence)

        self.start_btn.pack(side="left")

        self.cancel_btn = Button(self.btn_frame, text="Cancel Sequence
(Control+c)", command=self.cancel_sequence)

        self.cancel_btn.pack(side="left")

        Button(self.btn_frame, text="Save (Command+S)",
command=self.save_annotations).pack(side="right")

# Key bindings
self.root.bind('<Left>', lambda e: self.prev_frame())
self.root.bind('<Right>', lambda e: self.next_frame())
self.root.bind('<Shift-Left>', lambda e: self.jump_frames(-10))
self.root.bind('<Shift-Right>', lambda e: self.jump_frames(10))
self.root.bind('<Control-s>', lambda e: self.start_sequence())
self.root.bind('<Control-c>', lambda e: self.cancel_sequence())
self.root.bind('<Command-s>', lambda e: self.save_annotations())

# Load the first frame
self.draw_annotation_marks()

self.update_frame(self.current_frame)

self.root.protocol("WM_DELETE_WINDOW", self.on_close)

self.root.mainloop()

def jump_to_frame(self, event=None):
    """Jump to a specific frame number entered by user"""
    try:
        # Use 0-based indexing directly
        target_frame = int(self.jump_entry.get())

```

```

        target_frame = max(0, min(target_frame, self.total_frames - 1))

        self.update_frame(target_frame)

        self.jump_entry.delete(0, END) # Clear the input

    except ValueError:

        self.status_label.config(text="Invalid      frame      number!",
bg="orange", fg="black")

    def check_move_history(self, event=None):

        """Check if the current move combination has been annotated
before"""

        p1_move = self.p1_move_entry.get().strip().lower()
        p2_move = self.p2_move_entry.get().strip().lower()

        if not p1_move and not p2_move:

            self.history_label.config(text="")

            return

        # Find matching annotations

        matches = []

        for seq_id, seq_data in self.annotations.items():

            p1_annotated = seq_data["move"].lower() if
seq_data.get("player") == "player1" else ""

            p2_annotated = seq_data["move"].lower() if
seq_data.get("player") == "player2" else ""

            p1_match = p1_move and (p1_move == p1_annotated)

            p2_match = p2_move and (p2_move == p2_annotated)

            if p1_match or p2_match:

                duration = seq_data["duration_frames"]

                # Use 0-based frame numbers

```

```

        start_frame = seq_data["start_frame"]
        end_frame = seq_data["end_frame"] + 1

        matches.append({
            'id': seq_id,
            'p1_move': seq_data["player1"]["move"],
            'p2_move': seq_data["player2"]["move"],
            'duration': duration,
            'start': start_frame,
            'end': end_frame
        })

    if matches:
        history_text = "Previous annotations: "
        history_items = []
        for match in matches[:3]: # Show only first 3 matches to avoid
clutter
            history_items.append(f"{match['p1_move']} vs {match['p2_move']} ({match['duration']} frames, {match['start']}-{match['end']})")

        history_text += " | ".join(history_items)

        if len(matches) > 3:
            history_text += f" ... and {len(matches) - 3} more"

        self.history_label.config(text=history_text)
    else:
        self.history_label.config(text="")

    def set_quick_state(self, p1_state, p2_state):
        """Set quick states for both players"""
        self.p1_move_entry.delete(0, END)

```

```

self.p1_move_entry.insert(0, p1_state)

self.p2_move_entry.delete(0, END)

self.p2_move_entry.insert(0, p2_state)

# Trigger history check after setting states
self.check_move_history()

def update_frame(self, frame_index=None):
    if not self.cap.isOpened():
        return

    if frame_index is not None:
        self.current_frame = int(float(frame_index))

    self.cap.set(cv2.CAP_PROP_POS_FRAMES, self.current_frame)
    ret, frame = self.cap.read()

    if not ret:
        return

    # Convert frame to PIL Image and resize
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    img = Image.fromarray(frame)
    img = img.resize((480, 270), Image.LANCZOS)

    # Convert to PhotoImage and display
    self.photo = ImageTk.PhotoImage(image=img)
    self.canvas.create_image(0, 0, image=self.photo, anchor="nw")

    # Update slider to match current position

```

```

self.frame_slider.set(self.current_frame)

# Update time display

current_time = self.current_frame / self.fps

total_time = self.total_frames / self.fps

self.time_label.config(text=f"{self.format_time(current_time)} / {self.format_time(total_time)}")

# Update frame info - use 0-based frame numbers

self.frame_info_label.config(text=f"Frame: {self.current_frame}/{self.total_frames-1}")

# Update sequence frame count if recording

if self.sequence_start is not None:
    sequence_duration = self.current_frame - self.sequence_start +
1
    self.sequence_frames_label.config(text=f"Recording: {sequence_duration} frames")
else:
    self.sequence_frames_label.config(text="")

# Update title with recording status - use 0-based frame numbers

title = f"Tekken Move Annotator - Frame {self.current_frame}/{self.total_frames-1}"

if self.sequence_start is not None:
    title += f" - RECORDING (from frame {self.sequence_start})"
self.root.title(title)

# Update status based on recording state

if self.sequence_start is not None:
    sequence_duration = self.current_frame - self.sequence_start +
1

```

```

        self.status_label.config(text=f"Recording from frame
{self.sequence_start} | Current duration: {sequence_duration} frames",
bg="red", fg="white")

    else:

        self.status_label.config(text="Ready", bg="lightgray",
fg="black")

def format_time(self, seconds):

    """Convert seconds to MM:SS format"""

    minutes = int(seconds // 60)

    seconds = int(seconds % 60)

    return f"{minutes:02d}:{seconds:02d}"

def jump_frames(self, offset):

    """Jump a specific number of frames forward or backward"""

    target_frame = self.current_frame + offset

    target_frame = max(0, min(target_frame, self.total_frames - 1))

    self.update_frame(target_frame)

def prev_frame(self, event=None):

    if self.current_frame > 0:

        self.current_frame -= 1

        self.update_frame()

def next_frame(self, event=None):

    if self.current_frame < self.total_frames - 1:

        self.current_frame += 1

        self.update_frame()

def on_close(self):

    """Handle window close event"""

```

```

        if self.cap.isOpened():
            self.cap.release()

        self.root.destroy()

    def start_sequence(self, event=None):
        if self.sequence_start is None:
            self.sequence_start = self.current_frame

            self.start_btn.config(bg="red", fg="white", text="End Sequence
(s)")

            self.update_frame()

        else:
            self.end_sequence()

    def cancel_sequence(self, event=None):
        self.sequence_start = None

        self.start_btn.config(bg="SystemButtonFace", fg="black",
text="Start Sequence (s)")

        self.update_frame()

    def draw_annotation_marks(self):
        """Draw marks on the canvas for annotated sequences."""
        self.mark_canvas.delete("all")

        width = int(self.mark_canvas['width'])

        for seq in self.annotations.values():
            start = seq["start_frame"]
            end = seq["end_frame"]

            # Map frame to canvas position
            x1 = int(start / (self.total_frames - 1) * width)
            x2 = int(end / (self.total_frames - 1) * width)

            # Draw a small rectangle or line for the sequence

```

```

        self.mark_canvas.create_rectangle(x1, 0, x2, 10,
fill="#4caf50", outline="")

def end_sequence(self):

    p1_character = self.p1_char_entry.get().strip()
    p2_character = self.p2_char_entry.get().strip()
    p1_move = self.p1_move_entry.get().strip()
    p2_move = self.p2_move_entry.get().strip()

    saved_any = False

    status_msgs = []

    if p1_character and p1_move:

        sequence_id = f"p1_sequence_{len(self.annotations) + 1}"
        self.annotations[sequence_id] = {

            "player": "player1",

            "character": p1_character,

            "move": p1_move,

            "start_frame": self.sequence_start,

            "end_frame": self.current_frame,

            "start_time": self.sequence_start / self.fps,

            "end_time": self.current_frame / self.fps,

            "duration_frames": self.current_frame - self.sequence_start
+ 1,

            "annotation_type": "single_character"

        }

        status_msgs.append(f"{p1_character}: {p1_move} (frames
{self.sequence_start}-{self.current_frame})")

        saved_any = True

    if p2_character and p2_move:

```

```

sequence_id = f"p2_sequence_{len(self.annotations) + 1}"
self.annotations[sequence_id] = {
    "player": "player2",
    "character": p2_character,
    "move": p2_move,
    "start_frame": self.sequence_start,
    "end_frame": self.current_frame,
    "start_time": self.sequence_start / self.fps,
    "end_time": self.current_frame / self.fps,
    "duration_frames": self.current_frame - self.sequence_start
+ 1,
    "annotation_type": "single_character"
}

status_msgs.append(f"{p2_character}: {p2_move} (frames
{self.sequence_start}-{self.current_frame})")

saved_any = True

if saved_any:
    self.status_label.config(text=" | ".join(status_msgs),
bg="green", fg="white")

    self.p1_move_entry.delete(0, END)

    self.p2_move_entry.delete(0, END)

else:
    self.status_label.config(text="Error: Enter character and move
for at least one player!", bg="orange", fg="black")

    self.sequence_start = None

    self.start_btn.config(bg="SystemButtonFace", fg="black",
text="Start Sequence (s)")

    self.draw_annotation_marks()

    self.update_frame()

```

```

def save_annotations(self, event=None):
    with open(self.output_file, "w") as f:
        json.dump(self.annotations, f, indent=2)

        self.status_label.config(text=f"Annotations saved to {self.output_file}",
                                  bg="blue", fg="white")

        self.draw_annotation_marks()

        print(f"Annotations saved to {self.output_file}")

if __name__ == "__main__":
    Annotator()

    #Annotator("move_list/bryan/frames",
    "move_list/bryan/bryan_movelist.json")

```


Lampiran 2 Script pembuatan dataset dan training model YOLO detector (fighter_dataset_creation.ipynb):

```
# Libraries Import

import os, cv2, json, glob

import numpy as np

import matplotlib.pyplot as plt

from collections import defaultdict

import tensorflow as tf

import tensorflow_hub as hub

import torch

from ultralytics import YOLO

#Local Base Path

base_path = ""

# === Dataset Paths ===

data_path = os.path.join(base_path, "practice_videos")

video_path = os.path.join(data_path, "Bryan_LR_Complete.mp4")

# Load Object detector

if getattr(torch.backends, "mps", None) and torch.backends.mps.is_available():

    train_device = "mps"

elif torch.cuda.is_available():

    train_device = "cuda"

else:

    train_device = "cpu"

print(f"Selected training device: {train_device}")

# yolo = YOLO("runs/detect/twt/weights/best.pt")

yolo = YOLO("yolo11m.pt")

yolo.to(train_device)
```

```

os.makedirs(output_dir, exist_ok=True)

# Create Fighter Dataset
def yolo_detect_batch(frame_batch, original_width, original_height):
    # Dynamic minimum size (5% of the image dimensions)
    min_w = original_width * 0.05
    min_h = original_height * 0.05

    # Run batch inference! (Process multiple frames at once)
    results = yolo(frame_batch, conf=0.15, iou=0.75, max_det=4, classes=[0],
        verbose=False)

    batch_filtered_boxes = []

    for result in results:
        if result.bboxes is None or len(result.bboxes) == 0:
            batch_filtered_boxes.append([])
            continue

        boxes = result.bboxes.xyxy.cpu().numpy().astype(int)
        filtered_boxes = []

        for box in boxes:
            x1, y1, x2, y2 = box
            w = x2 - x1
            h = y2 - y1

            # Use the dynamic thresholds instead of hardcoded 200
            if w > min_w and h > min_h:
                filtered_boxes.append(box)

```

```

        batch_filtered_boxes.append(np.array(filtered_boxes))

    return batch_filtered_boxes

def pad_box(h, w, box, padding=25):
    x1, y1, x2, y2 = box

    x1 = max(0, x1 - padding)
    y1 = max(0, y1 - padding)
    x2 = min(w, x2 + padding)
    y2 = min(h, y2 + padding)

    return np.array([x1, y1, x2, y2], dtype=int)

def xyxy_to_yolo_normalized(box, img_width, img_height):
    x1, y1, x2, y2 = box

    # Calculate width and height
    w = x2 - x1
    h = y2 - y1

    # Calculate center x and center y
    x_center = x1 + (w / 2)
    y_center = y1 + (h / 2)

    # Normalize by image dimensions
    x_center_n = x_center / img_width
    y_center_n = y_center / img_height
    w_n = w / img_width
    h_n = h / img_height

```

```

    return [x_center_n, y_center_n, w_n, h_n]

def process_video_batched(video_path, frame_dir, label_dir, frame_step=15,
batch_size=16):

    video_name = os.path.splitext(os.path.basename(video_path))[0]

    cap = cv2.VideoCapture(video_path)

    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    frame_batch = []

    frame_info_batch = [] # To keep track of frame numbers

    # Read the first frame to get dimensions

    cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

    ret, test_frame = cap.read()

    if ret:

        original_height, original_width = test_frame.shape[:2]

    else:

        return

    for frame_count in range(0, total_frames, frame_step):

        cap.set(cv2.CAP_PROP_POS_FRAMES, frame_count)

        ret, frame_bgr = cap.read()

        if not ret: break

        frame_batch.append(frame_bgr)

        frame_info_batch.append(frame_count)

    # When the batch is full, or we hit the end of the video

```

```

        if len(frame_batch) == batch_size or frame_count + frame_step >=
total_frames:

            # Detect on the entire batch at once

            batch_boxes = yolo_detect_batch(frame_batch, original_width,
original_height)

            for i, box_array in enumerate(batch_boxes):

                curr_frame_num = frame_info_batch[i]

                curr_frame_img = frame_batch[i]

                base_filename = f"{video_name}_frame_{curr_frame_num:05d}"

                image_path = os.path.join(frame_dir,
f"{base_filename}.jpg")

                label_path = os.path.join(label_dir,
f"{base_filename}.txt")

                # SAVE WITH COMPRESSION TO SAVE SSD SPACE

                cv2.imwrite(image_path, curr_frame_img,
[cv2.IMWRITE_JPEG_QUALITY, 90])

                with open(label_path, "w") as f:

                    if len(box_array) > 0:

                        padded_boxes = box_array

                        for box in padded_boxes:

                            norm_box = xyxy_to_yolo_normalized(box,
original_width, original_height)

                            x_c, y_c, w, h = [max(0.0, min(1.0, val)) for
val in norm_box] # Clamp

                            f.write(f"0 {x_c:.6f} {y_c:.6f} {w:.6f}
{h:.6f}\n")

                # Clear the lists for the next batch

```

```

        frame_batch = []

        frame_info_batch = []

    cap.release()

input_size = 192

frame_dir = os.path.join(base_path, "fighter_dataset/images")
label_dir = os.path.join(base_path, "fighter_dataset/labels")

os.makedirs(frame_dir, exist_ok=True)
os.makedirs(label_dir, exist_ok=True)
process_video_batched(video_path, frame_dir, label_dir, 15)

# Train Val Split

import os
import shutil
import random
import yaml

# Adjust base_path if needed
base_path = ""

dataset_dir = os.path.join(base_path, "fighter_dataset") # Change to your
dataset directory

# Source directories from your previous step
src_images_dir = os.path.join(dataset_dir, "images")

src_labels_dir = os.path.join(dataset_dir, "labels") # Note: YOLO prefers
'labels' plural, we will fix this below

# YOLO standard split directories

```

```

train_images_dir = os.path.join(dataset_dir, "images", "train")
val_images_dir = os.path.join(dataset_dir, "images", "val")
train_labels_dir = os.path.join(dataset_dir, "labels", "train")
val_labels_dir = os.path.join(dataset_dir, "labels", "val")

# Ensure target directories exist

for dir_path in [train_images_dir, val_images_dir, train_labels_dir,
val_labels_dir]:

    os.makedirs(dir_path, exist_ok=True)

# Get all image files and shuffle them for a random split
image_files = [f for f in os.listdir(src_images_dir) if f.endswith('.jpg')]
random.shuffle(image_files)

# Define split ratio (80% Train, 20% Val)
split_ratio = 0.8
split_index = int(len(image_files) * split_ratio)

train_files = image_files[:split_index]
val_files = image_files[split_index:]

def populate_split(file_list, dest_img_dir, dest_lbl_dir):

    for img_file in file_list:

        # Source paths

        src_img = os.path.join(src_images_dir, img_file)

        lbl_file = img_file.replace('.jpg', '.txt')
        src_lbl = os.path.join(src_labels_dir, lbl_file)

        # Destination paths

```

```

dest_img = os.path.join(dest_img_dir, img_file)

dest_lbl = os.path.join(dest_lbl_dir, lbl_file)


# Move files

if os.path.exists(src_img):

    shutil.move(src_img, dest_img)

if os.path.exists(src_lbl):

    shutil.move(src_lbl, dest_lbl)


# Execute the split
populate_split(train_files, train_images_dir, train_labels_dir)
populate_split(val_files, val_images_dir, val_labels_dir)


if os.path.exists(src_labels_dir) and not os.listdir(src_labels_dir):

    os.rmdir(src_labels_dir)


# Generate the data.yaml file required by YOLO
yaml_content = {

    'path': os.path.abspath(dataset_dir),

    'train': 'images/train',

    'val': 'images/val',

    'nc': 1,

    'names': {0: 'fighter'}

}


yaml_path = os.path.join(dataset_dir, 'data.yaml')

with open(yaml_path, 'w') as f:

    yaml.dump(yaml_content, f, sort_keys=False)

```

```

print(f"✅ Split Complete! {len(train_files)} Training | {len(val_files)} Validation")

print(f"✅ data.yaml generated at: {yaml_path}")

# Yolo Training
yolo.train(
    data=os.path.join(base_path, 'fighter_dataset/data.yaml'),
    epochs=100,
    imgsz=640,
    batch=16,          # Increased to utilize 24GB RAM and stabilize
                        # gradients
    device=train_device,
    patience=20,        # Give it a bit more room to breathe
    workers=16,         # M-series chips handle 8 workers beautifully
    half=False,         # Disabled to prevent MPS gradient stalling
    cache=False,
    cos_lr=True,        # Use Cosine Learning Rate to gently unstick plateaus

    # --- TAILORED AUGMENTATIONS ---
   fliplr=0.5,          # MUST KEEP: Simulates Player 1 side vs Player 2 side
    translate=0.1,      # KEEP: Simulates camera panning left/right
    scale=0.3,          # KEEP: Handles dynamic zoom-ins during combos/rage
                        # arts
    hsv_h=0.015,        # KEEP: Alters hue to help generalize across the 9
                        # different maps
    hsv_s=0.7,          # KEEP: Color saturation variation
    hsv_v=0.4           # KEEP: Brightness variation

```

Lampiran 3 Konfigurasi dataset YOLO (fighter_dataset/data.yaml):

```
path: /Users/julianyang/Mine/Kuliah/SMT 6/CV/UAS/fighter_dataset
train: images/train
val: images/val
nc: 1
names:
  0: fighter
```

Lampiran 4 Pipeline lengkap object detection, tracking, ROI cropping, pose estimation, penyimpanan data ke dalam bentuk .npy, hingga training STGCN++ (Tekken_recognition_yoloPose_final_cleaned.ipynb):

```
# Libraries Import and Base Path initialization

import os, cv2, json, math, pickle, random

import numpy as np

import matplotlib.pyplot as plt

from collections import defaultdict

from ultralytics import YOLO

from boxmot.trackers.ocsort.ocsort import OcSort

import torch


from mmengine.config import Config
from mmengine.registry import MODELS
from mmengine.runner import load_checkpoint
from mmaction.apis import init_recognizer


# # This function will now work correctly because we are running from the
# cloned directory

from mmaction.utils import register_all_modules

register_all_modules(init_default_scope=True) # We set the scope manually
later


#Local Base Path

base_path = ""


# === Dataset Paths ===

data_path = os.path.join(base_path, "practice_videos")

video_path = os.path.join(data_path, "Bryan_LR_Complete.mp4")
```

```

annotation_path = os.path.join(data_path, "Bryan_LR_Complete.json")

labels_path = os.path.join(data_path, "move_labels.json") # move class
labels

filtered_labels_path = os.path.join(data_path,
"move_labels_filtered.json") # selected/remapped move labels

skeleton_dataset = os.path.join(data_path, "skeleton_dataset.pkl") #
STGCN++ dataset

output_dir = os.path.join(data_path, "frames")

kp_dir = os.path.join(data_path)

# === Load YOLO detection and pose models ===
yolo = YOLO("runs/detect/practice_m/weights/best.pt")
yolo_pose = YOLO("yolo11l-pose.pt")
yolo.to("mps")
yolo_pose.to("mps")

# YOLO and ByteTrack character tracking
def make_tracker():
    return OcSort(
        half=True,
        device="mps",
        max_age=90,
        min_hits=2,
        iou_threshold=0.15,
        det_thresh=0.20,
    )

tracker = make_tracker()

def yolo_detect(image):

```

```

result = yolo(image, conf=0.5, iou=0.35, classes=[0])[0]

# the results of yolo.predict contains list of object per frame it
detects, for example image will have 1 object in the list

# while video will have as many object in it as the video frames

# we will access the first object as it is an image

# Play with conf(minimum conf to be detected) and

# iou (how much the boxes can overlap to be considered the same object)
if result.bboxes is None or len(result.bboxes) == 0:

    return np.empty((0, 6), dtype=np.float32)

xyxy = result.bboxes.xyxy.cpu().numpy() # convert boxes x1, y1, x2, y2
of selected object to numpy, then to int

conf = result.bboxes.conf.cpu().numpy().reshape(-1, 1) # convert boxes
confidence of selected object to numpy

cls = result.bboxes.cls.cpu().numpy().reshape(-1, 1) # convert boxes
class of selected object to numpy

detections = np.hstack((xyxy, conf, cls))

# stack boxes, conf, cls horizontally (it only accepts tuple so we
encapsulate it with double ())

return detections

def ocsort_tracking(detections, image):

    if detections.shape[0] == 0:

        return None

    tracks = tracker.update(detections, image)

    if tracks is None or len(tracks) == 0:

        return None

    ids = tracks[:, 4].astype(int).reshape(-1, 1)

```

```

boxes = tracks[:, :4].astype(int)

id_box_array = np.hstack((ids, boxes))

return id_box_array

def pad_box(h, w, box, padding=0):
    # box: [id, x1, y1, x2, y2]
    _, x1, y1, x2, y2 = box
    x1 = max(0, x1 - padding)
    y1 = max(0, y1 - padding)
    x2 = min(w, x2 + padding)
    y2 = min(h, y2 + padding)

    return np.array([_, x1, y1, x2, y2], dtype=int)

def crop_roi(frame, box):
    # box: [id, x1, y1, x2, y2]
    _, x1, y1, x2, y2 = box
    return frame[y1:y2, x1:x2]

    # Matrix (Memory) Coordinates: NumPy thinks in (row, column).
    # Because images are processed top-to-bottom, a row corresponds to
    # the vertical y position, and a column corresponds to the horizontal x
    # position.

def is_timer_missing(frame, edge_threshold=50):
    """
    Checks the Tekken timer UI using Edge Detection.
    Returns True if the sharp metallic borders of the numbers are missing.
    """
    y1, y2 = 26, 86

```

```

x1, x2 = 600, 680

timer_roi = frame[y1:y2, x1:x2]

gray = cv2.cvtColor(timer_roi, cv2.COLOR_BGR2GRAY)

# cv2.Canny highlights sharp transitions.
# The silver border of the font will light up brilliantly here.
edges = cv2.Canny(gray, 100, 200)

# Count how many 'edge' pixels exist in that small box
edge_count = cv2.countNonZero(edges)

# If the count drops below the threshold, the timer is gone.
return edge_count < edge_threshold

# YOLO Pose Function
def empty_keypoints():
    return np.full((17, 3), np.nan, dtype=np.float32)

def box_iou_xyxy(box, boxes):
    box = np.asarray(box, dtype=np.float32)
    boxes = np.asarray(boxes, dtype=np.float32)

    if boxes.size == 0:
        return np.array([], dtype=np.float32)

    x1 = np.maximum(box[0], boxes[:, 0])
    y1 = np.maximum(box[1], boxes[:, 1])
    x2 = np.minimum(box[2], boxes[:, 2])
    y2 = np.minimum(box[3], boxes[:, 3])

```

```

inter = np.maximum(0, x2 - x1) * np.maximum(0, y2 - y1)

box_area = max(0, box[2] - box[0]) * max(0, box[3] - box[1])

boxes_area = np.maximum(0, boxes[:, 2] - boxes[:, 0]) * np.maximum(0,
boxes[:, 3] - boxes[:, 1])

union = box_area + boxes_area - inter

return np.divide(inter, union, out=np.zeros_like(inter,
dtype=np.float32), where=union > 0)

def choose_pose_candidate(boxes_xyxy, box_scores, target_box=None):

    if boxes_xyxy.shape[0] == 0:

        return 0

    if target_box is None:

        return int(np.argmax(box_scores))

    target_box = np.asarray(target_box, dtype=np.float32)

    ious = box_iou_xyxy(target_box, boxes_xyxy)

    target_center = np.array([(target_box[0] + target_box[2]) / 2,
(target_box[1] + target_box[3]) / 2], dtype=np.float32)

    candidate_centers = np.column_stack(((boxes_xyxy[:, 0] + boxes_xyxy[:,
2]) / 2, (boxes_xyxy[:, 1] + boxes_xyxy[:, 3]) / 2))

    distances = np.linalg.norm(candidate_centers - target_center, axis=1)

    target_diag = max(1.0, np.linalg.norm([target_box[2] - target_box[0],
target_box[3] - target_box[1]]))

    center_score = 1.0 - np.clip(distances / target_diag, 0.0, 1.0)

    match_score = (2.0 * ious) + center_score + (0.25 * box_scores)

    return int(np.argmax(match_score))

def track_box_to_roi_box(track_box, crop_box):

    _, tx1, ty1, tx2, ty2 = track_box

```

```

_, cx1, cy1, _, _ = crop_box

return np.array([tx1 - cx1, ty1 - cy1, tx2 - cx1, ty2 - cy1],
dtype=np.float32)

def run_yolopose(image, input_size=192, conf=0.25, iou=0.35,
target_box=None):

    if image is None or image.size == 0:

        return empty_keypoints()

    roi_height, roi_width = image.shape[:2]

    if roi_height == 0 or roi_width == 0:

        return empty_keypoints()

    result = yolo_pose(image, imgsiz=input_size, conf=conf, iou=iou,
verbose=False)[0]

    if result.keypoints is None or len(result.keypoints) == 0:

        return empty_keypoints()

    xy = result.keypoints.xy.cpu().numpy()

    kp_conf = result.keypoints.conf

    if kp_conf is None:

        scores = np.ones(xy.shape[:2], dtype=np.float32)

    else:

        scores = kp_conf.cpu().numpy()

    if xy.shape[0] == 0:

        return empty_keypoints()

    if result.bboxes is not None and len(result.bboxes) > 0:

        boxes_xyxy = result.bboxes.xyxy.cpu().numpy()

        box_scores = result.bboxes.conf.cpu().numpy()

```

```

        best_idx = choose_pose_candidate(boxes_xyxy, box_scores,
target_box)

        best_idx = min(best_idx, xy.shape[0] - 1)

    else:

        mean_scores = np.nanmean(scores, axis=1)

        best_idx = 0 if np.all(np.isnan(mean_scores)) else
int(np.nanargmax(mean_scores))

    keypoints = empty_keypoints()

    points_xy = xy[best_idx]

    point_scores = scores[best_idx]

    num_points = min(17, points_xy.shape[0])

    keypoints[:num_points, 0] = points_xy[:num_points, 1] / roi_height
    keypoints[:num_points, 1] = points_xy[:num_points, 0] / roi_width
    keypoints[:num_points, 2] = point_scores[:num_points]

    return keypoints

def denormalize_points(points, original_height, original_width,
input_size=None):
    """
    Converts normalized YOLO-pose ROI keypoints back to ROI pixel
coordinates.
    """
    y, x, c = points

    if np.isnan(y) or np.isnan(x):
        return (np.nan, np.nan, c)

    y_abs = y * original_height
    x_abs = x * original_width

    return (y_abs, x_abs, c)

```

```

def normalize_points_to_full_frame(kp_array, box, full_height,
full_width):
    """
    Normalize array of keypoints to full frame size
    """
    _, x1, y1, x2, y2 = box
    roi_height = y2-y1
    roi_width = x2-x1

    kp_full = []
    for kp in kp_array:
        y_roi, x_roi, c = denormalize_points(kp, roi_height, roi_width)
        if np.isnan(y_roi) or np.isnan(x_roi):
            kp_full.append([np.nan, np.nan, c])
            continue

        y_full = (y_roi + y1) / full_height
        x_full = (x_roi + x1) / full_width
        kp_full.append([y_full, x_full, c])

    return np.array(kp_full)

def draw_keypoints(frame, keypoints, color, full_height, full_width,
score_threshold=0.0):
    if keypoints is None:
        return

    for kp in keypoints:

```

```

        y, x, c = kp

        if not np.isfinite(y) or not np.isfinite(x):

            continue

        if np.isfinite(c) and c < score_threshold:

            continue

        y_px = int(np.clip(y * full_height, 0, full_height - 1))

        x_px = int(np.clip(x * full_width, 0, full_width - 1))

        cv2.circle(frame, (x_px, y_px), 3, color, thickness=2,
lineType=cv2.LINE_AA)

def interpolate_points(player_kp):

    player_kp = np.array(player_kp)

    for kp in range(player_kp.shape[1]):

        for coord in range(player_kp.shape[2]):

            data = player_kp[:, kp, coord]

            nans = np.isnan(data) # nans mask example: [true, false, true]
based on the positions

            if np.any(~nans):

                data[nans] = np.interp(np.flatnonzero(nans),
np.flatnonzero(~nans), data[~nans])

                player_kp[:, kp, coord] = data

                print(data)

    return player_kp

# YOLO Pose Extraction

input_size = 384

cap = cv2.VideoCapture(video_path)

print(video_path, os.path.exists(video_path))

player1_kp = []

player2_kp = []

```

```

player1_track = [] # [id, x1, y1, x2, y2] or NaNs per frame
player2_track = [] # [id, x1, y1, x2, y2] or NaNs per frame
other_track = []   # [id, x1, y1, x2, y2] or NaNs per frame

player_set = False

player1_id, player2_id = None, None

player1_box, player2_box, other_box = None, None, None

player1_kp_full, player2_kp_full, other_kp_full = None, None, None

frame_count = 0

missing_timer_frames = 0

buffer_limit = 10 # 10 frames ignores brief juggles, but catches the 15-
frame practice reset

tracking_active = True

cinematic_frames = 0

cinematic_buffer = 5 # Wait 5 frames to confirm a zoom

while cap.isOpened(): # read every single frame of the video
    ret, frame_bgr = cap.read()
    if not ret:
        print("End of video")
        break

    original_height, original_width = frame_bgr.shape[:2]

    # --- 1. CHECK THE TIMER ---
    if is_timer_missing(frame_bgr):
        missing_timer_frames += 1
    else:
        missing_timer_frames = 0

```

```

        tracking_active = True # Timer is clearly visible, tracking is safe

# --- HANDLE THE RESET STATE ---

if missing_timer_frames > buffer_limit:

    # Only print and wipe memory the FIRST time we cross the threshold

    if tracking_active:

        print(f"Scene transition detected at frame {frame_count}.
Pausing tracking...")

        tracking_active = False

        tracker = make_tracker()

        # last_p1_box = None

        # last_p2_box = None

        # If using an OC-SORT instance, destroy/re-init it here.

        player_set = False

    # We are in a reset state (black screen or waiting for fade-in).

    # Append empty frames to keep your temporal arrays perfectly aligned
for the pipeline.

    player1_kp.append(np.full((17, 3), np.nan))
    player2_kp.append(np.full((17, 3), np.nan))
    player1_track.append(np.full(5, np.nan, dtype=np.float32))
    player2_track.append(np.full(5, np.nan, dtype=np.float32))
    other_track.append(np.full(5, np.nan, dtype=np.float32))

    frame_count += 1

    # Skip the rest of the loop entirely. Do not run YOLO.

    continue

```

```

# --- 2. THE CINEMATIC CHECK ---

# Run your raw YOLO detection ONCE per frame

detections = yolo_detect(frame_bgr)

id_box_array = ocsort_tracking(detections, frame_bgr) # 2d array
containing id_box from p1 and 2

if id_box_array is None or id_box_array.size == 0:

    player1_kp.append(np.full((17, 3), np.nan))

    player2_kp.append(np.full((17, 3), np.nan))

    player1_track.append(np.full(5, np.nan, dtype=np.float32))
    player2_track.append(np.full(5, np.nan, dtype=np.float32))
    other_track.append(np.full(5, np.nan, dtype=np.float32))

    frame_count += 1

    continue

if id_box_array is not None and id_box_array.shape[0] >= 2 and not
player_set:

    centers_x = (id_box_array[:,1] + id_box_array[:,3]) / 2

    order = np.argsort(centers_x) # left -> right

    player1_id = int(id_box_array[order[0], 0])

    player2_id = int(id_box_array[order[1], 0])

    player_set = True

p1_exist = any(id_box_array[:, 0] == player1_id)
p2_exist = any(id_box_array[:, 0] == player2_id)

other_mask = ~np.isin(id_box_array[:, 0], [player1_id, player2_id])

```

```

has_other = np.any(other_mask)

player1_track_frame = np.full(5, np.nan, dtype=np.float32)
player2_track_frame = np.full(5, np.nan, dtype=np.float32)
other_track_frame = np.full(5, np.nan, dtype=np.float32)

if p1_exist:
    player1_box = id_box_array[id_box_array[:, 0] == player1_id][0]
    player1_track_frame = player1_box.astype(np.float32)
    # print("p1 box", player1_box)

    player1_box_padded = pad_box(original_height, original_width,
player1_box)

    player1_roi = crop_roi(frame_bgr, player1_box_padded)

    player1_target_box = track_box_to_roi_box(player1_box,
player1_box_padded)

    player1_kp_raw = run_yolopose(player1_roi, input_size,
target_box=player1_target_box)

    player1_kp_full = normalize_points_to_full_frame(player1_kp_raw,
player1_box_padded, original_height, original_width)

    player1_kp.append(player1_kp_full)

else:
    player1_box = None
    player1_box_padded = None
    player1_kp_full = None
    player1_kp.append(np.full((17, 3), np.nan))

if p2_exist:
    player2_box = id_box_array[id_box_array[:, 0] == player2_id][0]
    player2_track_frame = player2_box.astype(np.float32)

    player2_box_padded = pad_box(original_height, original_width,
player2_box)

    player2_roi = crop_roi(frame_bgr, player2_box_padded)

```

```

        player2_target_box = track_box_to_roi_box(player2_box,
player2_box_padded)

        player2_kp_raw = run_yolopose(player2_roi, input_size,
target_box=player2_target_box)

        player2_kp_full = normalize_points_to_full_frame(player2_kp_raw,
player2_box_padded, original_height, original_width)

        player2_kp.append(player2_kp_full)
    else:

        player2_box = None

        player2_box_padded = None

        player2_kp_full = None

        player2_kp.append(np.full((17, 3), np.nan))

    if has_other:

        other_box = id_box_array[other_mask][0]

        other_track_frame = other_box.astype(np.float32)

        other_box_padded = pad_box(original_height, original_width,
other_box)

        other_roi = crop_roi(frame_bgr, other_box_padded)

        other_target_box = track_box_to_roi_box(other_box,
other_box_padded)

        other_kp_raw = run_yolopose(other_roi, input_size,
target_box=other_target_box)

        other_kp_full = normalize_points_to_full_frame(other_kp_raw,
other_box_padded, original_height, original_width)
    else:

        other_box = None

        other_box_padded = None

        other_kp_full = None

    # Visualization: draw player 1 (green) and player 2 (red)

    color_p1 = (0, 255, 0) # green (B, G, R)

```

```

color_p2 = (0, 0, 255) # red

# Boxes + IDs

if player1_box is not None:

    obj_id, x1, y1, x2, y2 = player1_box_padded

    cv2.rectangle(frame_bgr, (x1, y1), (x2, y2), color_p1, thickness=2,
lineType=cv2.LINE_AA)

    cv2.putText(frame_bgr, "ID: " + str(int(obj_id)), (x1, y1+20),
cv2.FONT_HERSHEY_SIMPLEX,
                    fontScale=1,          color=color_p1,          thickness=1,
lineType=cv2.LINE_AA)

if player2_box is not None:

    obj_id, x1, y1, x2, y2 = player2_box_padded

    cv2.rectangle(frame_bgr, (x1, y1), (x2, y2), color_p2, thickness=2,
lineType=cv2.LINE_AA)

    cv2.putText(frame_bgr, "ID: " + str(int(obj_id)), (x1, y1+20),
cv2.FONT_HERSHEY_SIMPLEX,
                    fontScale=1,          color=color_p2,          thickness=1,
lineType=cv2.LINE_AA)

if other_box is not None:

    obj_id, x1, y1, x2, y2 = other_box_padded

    cv2.rectangle(frame_bgr, (x1, y1), (x2, y2), (255, 0, 0),
thickness=2, lineType=cv2.LINE_AA)

    cv2.putText(frame_bgr, "ID: " + str(int(obj_id)), (x1, y1+20),
cv2.FONT_HERSHEY_SIMPLEX,
                    fontScale=1,          color=color_p2,          thickness=1,
lineType=cv2.LINE_AA)

# Keypoints

draw_keypoints(frame_bgr, player1_kp_full, color_p1, original_height,
original_width)

draw_keypoints(frame_bgr, player2_kp_full, color_p2, original_height,
original_width)

```

```

player1_track.append(player1_track_frame)
player2_track.append(player2_track_frame)
other_track.append(other_track_frame)

cv2.imshow('Process feed', frame_bgr)
if cv2.waitKey(25) & 0xFF == ord('q'):
    break

frame_count += 1

cap.release()

player1_kp = np.asarray(player1_kp, dtype=np.float32)
player2_kp = np.asarray(player2_kp, dtype=np.float32)
player1_track = np.asarray(player1_track, dtype=np.float32)
player2_track = np.asarray(player2_track, dtype=np.float32)
other_track = np.asarray(other_track, dtype=np.float32)

np.save(os.path.join(kp_dir, "player1_kp"), player1_kp)
np.save(os.path.join(kp_dir, "player2_kp"), player2_kp)
np.save(os.path.join(kp_dir, "player1_track"), player1_track)
np.save(os.path.join(kp_dir, "player2_track"), player2_track)
np.save(os.path.join(kp_dir, "other_track"), other_track)

```

```
# Train STGCN++ all moves
```

```
%cd "/Users/julianyang/Mine/Kuliah/SMT 6/CV/UAS"
```

```
# Build dataset from every annotated move plus a small idle class.
```

```
!.venv/bin/python make_all_moves_idle_dataset.py \
```

```

--positive-window-size 30 \
--positive-window-mode center \
--positive-offsets=-3,0,6 \
--idle-count 30 \
--idle-window-size 30 \
--idle-stride 15 \
--exclude-margin 15 \
--min-mean-score 0.35

%cd "/Users/julianyang/Mine/Kuliah/SMT 6/CV/UAS/mmaction2"

!MPLCONFIGDIR=/private/tmp/matplotlib \
PYTORCH_ENABLE_MPS_FALLBACK=1 \
CUDA_VISIBLE_DEVICES=-1 \
../.venv/bin/python tools/train.py \
    configs/skeleton/stgcnpp/tekken_stgcn_all_moves_idle.py \
    --work-dir
work_dirs/tekken_stgcn_v13_all_moves_idle_no_interp_offsets_m3_0_6_rerun

# Run Inference
%cd "/Users/julianyang/Mine/Kuliah/SMT 6/CV/UAS"

!.venv/bin/python tekken_video_inference.py \
    --video practice_videos/Bryan_LR_Complete.mp4 \
    --output
practice_videos/tekken_stgcn_demo_v13_no_interp_offsets_m3_0_6_0.75.mp4 \
    --config
mmaction2/configs/skeleton/stgcnpp/tekken_stgcn_all_moves_idle.py \
    --labels practice_videos/move_labels_all_moves_idle.json \
    --checkpoint
mmaction2/work_dirs/tekken_stgcn_v13_fix_sls_annotation/best_acc_top1_epoch_22.pth \

```

```
--pose-model yolo11l-pose.pt \  
--pose-imsz 384 \  
--hide-labels idle \  
--predict-every 3 \  
--action-conf 0.75 \  
--stable-predictions 4 \  
--kp-interpolation none
```

Test inference

```
%cd "/Users/julianyang/Mine/Kuliah/SMT 6/CV/UAS"  
  
!.venv/bin/python tekken_video_inference.py \  
--video practice_videos/Bryan_LR_Complete.mp4 \  
--output "Bukti projek/e2e_benchmark_normal_power.mp4" \  
--metrics-json "Bukti projek/e2e_benchmark_normal_power.json" \  
--max-frames 330 \  
--metrics-warmup 30 \  
--metrics-window 60 \  
--device mps \  
--action-device cpu \  
--realtime-target-fps 0 \  
--device-label "MacBook Pro, Apple M5, 24 GB"
```

Lampiran 5 Membentuk dataset STGCN++ (make_all_moves_idle_dataset.py):

```
"""Build an all-annotated-moves + idle ST-GCN dataset.

Positive clips come directly from Bryan_LR_Complete.json and the full
player keypoint arrays. Idle clips are sampled only from low-motion windows
that do not overlap any annotated move for that player.
"""

from __future__ import annotations

import argparse
import json
import pickle
import random

from collections import Counter, defaultdict
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parent
DATA_DIR = ROOT / "practice_videos"

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Create an all-moves + idle
skeleton dataset.")

    parser.add_argument("--annotations", type=Path, default=DATA_DIR /
"Bryan_LR_Complete.json")
```

```

    parser.add_argument("--player1-kp", type=Path, default=DATA_DIR /
"player1_kp.npy")

    parser.add_argument("--player2-kp", type=Path, default=DATA_DIR /
"player2_kp.npy")

    parser.add_argument("--output-dataset", type=Path, default=DATA_DIR /
"skeleton_dataset_all_moves_idle.pkl")

    parser.add_argument("--output-labels", type=Path, default=DATA_DIR /
"move_labels_all_moves_idle.json")

    parser.add_argument(
        "--positive-window-size",
        type=int,
        default=30,
        help="If >0, use a fixed-size context window around each annotated
move instead of raw start/end frames.",
    )

    parser.add_argument(
        "--positive-window-mode",
        choices=("center", "end", "start"),
        default="center",
        help="How to align fixed positive windows when --positive-window-
size is used.",
    )

    parser.add_argument(
        "--positive-offsets",
        default="-3,0,6",
        help="Comma-separated frame offsets for fixed positive windows, e.g.
-6,0,6.",
    )

    parser.add_argument("--idle-label", default="idle")

    parser.add_argument("--idle-count", type=int, default=30)

    parser.add_argument("--idle-window-size", type=int, default=30)

    parser.add_argument("--idle-stride", type=int, default=15)

    parser.add_argument("--exclude-margin", type=int, default=15)

```

```

parser.add_argument("--min-mean-score", type=float, default=0.35)
parser.add_argument("--idle-pool-multiplier", type=int, default=5)
parser.add_argument("--val-ratio", type=float, default=0.2)
parser.add_argument("--seed", type=int, default=42)
parser.add_argument("--img-height", type=int, default=720)
parser.add_argument("--img-width", type=int, default=1280)
return parser.parse_args()

def require_path(path: Path, label: str) -> None:
    if not path.exists():
        raise FileNotFoundError(f"{label} not found: {path}")

def load_json(path: Path):
    with path.open("r", encoding="utf-8") as f:
        return json.load(f)

def keypoints_to_annotation(
    name: str,
    label: int,
    keypoints_yxc: np.ndarray,
    img_shape: tuple[int, int],
    source_frame_dir: str | None = None,
) -> dict:
    coords_xy = keypoints_yxc[:, :, [1, 0]].astype(np.float32)
    scores = keypoints_yxc[:, :, 2].astype(np.float32)
    coords_xy = np.nan_to_num(coords_xy, nan=0.0, posinf=0.0, neginf=0.0)

```

```

scores = np.nan_to_num(scores, nan=0.0, posinf=0.0, neginf=0.0)

ann = {
    "frame_dir": name,
    "label": label,
    "img_shape": img_shape,
    "original_shape": img_shape,
    "total_frames": int(keypoints_yxc.shape[0]),
    "keypoint": coords_xy[None],
    "keypoint_score": scores[None],
}

if source_frame_dir is not None:
    ann["source_frame_dir"] = source_frame_dir
return ann

def fixed_window_bounds(
    start: int,
    end: int,
    total_frames: int,
    window_size: int,
    mode: str,
    offset: int = 0,
) -> tuple[int, int]:
    if window_size <= 0:
        return start, end

    if window_size >= total_frames:
        return 0, total_frames

```

```

if mode == "start":
    win_start = start

elif mode == "end":
    win_start = end - window_size

else:
    center = (start + end) // 2
    win_start = center - (window_size // 2)

win_start += offset

win_start = max(0, min(win_start, total_frames - window_size))

return win_start, win_start + window_size


def parse_offsets(offsets: str) -> list[int]:
    parsed = []
    for item in offsets.split(","):
        item = item.strip()
        if not item:
            continue
        parsed.append(int(item))
    return parsed or [0]


def ordered_label_map(full_annotations: dict, idle_label: str) -> dict[str, int]:
    labels: dict[str, int] = {}
    for item in full_annotations.values():
        label = f"{item['character']} {item['move']}"
        if label not in labels:
            labels[label] = len(labels)

```

```

        labels[idle_label] = len(labels)

    return labels

def build_move_block_masks(
    full_annotations: dict,
    total_frames: int,
    margin: int,
) -> dict[str, np.ndarray]:
    blocked = {
        "player1": np.zeros(total_frames, dtype=bool),
        "player2": np.zeros(total_frames, dtype=bool),
    }

    for item in full_annotations.values():
        player = item["player"]

        if player not in blocked:
            continue

        start = max(0, int(item["start_frame"]) - margin)
        end = min(total_frames - 1, int(item["end_frame"]) + margin)

        blocked[player][start : end + 1] = True

    return blocked

def window_motion(keypoints_yxc: np.ndarray) -> float:
    coords = keypoints_yxc[:, :, :2]
    scores = keypoints_yxc[:, :, 2]

    valid = np.isfinite(coords).all(axis=2) & np.isfinite(scores) & (scores
> 0.3)

```

```

    diffs = np.linalg.norm(np.diff(np.nan_to_num(coords, nan=0.0), axis=0),
axis=2)

    valid_pairs = valid[1:] & valid[:-1]

    if not valid_pairs.any():

        return float("inf")

    return float(np.nanmean(diffs[valid_pairs]))

def collect_idle_candidates(
    player_name: str,
    keypoints: np.ndarray,
    blocked: np.ndarray,
    window_size: int,
    stride: int,
    min_mean_score: float,
) -> list[tuple[str, int, int, float, float]]:
    candidates = []
    total_frames = keypoints.shape[0]

    for start in range(0, total_frames - window_size + 1, stride):
        end = start + window_size

        if blocked[start:end].any():
            continue

        window = keypoints[start:end]
        coords = window[:, :, :2]
        scores = window[:, :, 2]

        finite_ratio = float(np.isfinite(coords).mean())
        mean_score = float(np.nanmean(scores))

        if finite_ratio < 0.95 or mean_score < min_mean_score:

```

```

        continue

    motion = window_motion(window)

    if not np.isfinite(motion):
        continue

    candidates.append((player_name, start, end, mean_score, motion))

return candidates

def split_by_class(
    annotations: list[dict],
    val_ratio: float,
    rng: random.Random,
) -> dict[str, list[str]]:
    by_label_group: dict[int, dict[str, list[str]]] = defaultdict(lambda: defaultdict(list))

    for ann in annotations:
        group = ann.get("source_frame_dir", ann["frame_dir"])
        by_label_group[int(ann["label"])] [group].append(ann["frame_dir"])

    train: list[str] = []
    val: list[str] = []

    for grouped_names in by_label_group.values():
        groups = list(grouped_names.items())
        rng.shuffle(groups)

        val_count = max(1, round(len(groups) * val_ratio)) if len(groups) >
1 else 0

        for _, names in groups[:val_count]:
            val.extend(names)

```

```

        for _, names in groups[val_count:]:
            train.extend(names)

    rng.shuffle(train)
    rng.shuffle(val)
    return {"train": train, "val": val}

def main() -> None:
    args = parse_args()
    for path, label in [
        (args.annotations, "Annotation JSON"),
        (args.player1_kp, "Player 1 keypoints"),
        (args.player2_kp, "Player 2 keypoints"),
    ]:
        require_path(path, label)

    rng = random.Random(args.seed)
    full_annotations = load_json(args.annotations)
    label_map = ordered_label_map(full_annotations, args.idle_label)
    idle_label_id = label_map[args.idle_label]
    img_shape = (args.img_height, args.img_width)
    positive_offsets = parse_offsets(args.positive_offsets)

    player_keypoints = {
        "player1": np.load(args.player1_kp).astype(np.float32),
        "player2": np.load(args.player2_kp).astype(np.float32),
    }
    total_frames = min(kp.shape[0] for kp in player_keypoints.values())

```

```

output_annotations: list[dict] = []

for name, item in full_annotations.items():

    player = item["player"]

    if player not in player_keypoints:

        continue

    raw_start = int(item["start_frame"])
    raw_end = int(item["end_frame"]) + 1

    label = f"{item['character']} {item['move']}"

    seen_windows = set()

    offsets = positive_offsets if args.positive_window_size > 0 else [0]

    for offset in offsets:

        start, end = fixed_window_bounds(

            raw_start,

            raw_end,

            player_keypoints[player].shape[0],

            args.positive_window_size,

            args.positive_window_mode,

            offset,

        )

        if start < 0 or end > player_keypoints[player].shape[0] or end
<= start:

            continue

        if (start, end) in seen_windows:

            continue

        seen_windows.add((start, end))

    frame_dir = name

    if args.positive_window_size > 0:

```

```

frame_dir = f"{name}_off{offset:+d}_w{start:05d}_{end -
1:05d}"

output_annotations.append(
    keypoints_to_annotation(
        frame_dir,
        label_map[label],
        player_keypoints[player][start:end],
        img_shape,
        source_frame_dir=name,
    )
)

blocked = build_move_block_masks(full_annotations, total_frames,
args.exclude_margin)

idle_candidates = []

for player_name, keypoints in player_keypoints.items():
    idle_candidates.extend(
        collect_idle_candidates(
            player_name,
            keypoints,
            blocked[player_name],
            args.idle_window_size,
            args.idle_stride,
            args.min_mean_score,
        )
    )

idle_candidates.sort(key=lambda item: item[4])

pool_size = max(args.idle_count, args.idle_count *
args.idle_pool_multiplier)

idle_pool = idle_candidates[:pool_size]

```

```

rng.shuffle(idle_pool)

chosen_idle = idle_pool[: args.idle_count]

if len(chosen_idle) < args.idle_count:
    print(f"Warning: requested {args.idle_count} idle clips, found
{len(chosen_idle)}.")

for idx, (player_name, start, end, _, motion) in enumerate(chosen_idle,
start=1):
    name = f"{player_name}_idle_{idx:03d}_{start:05d}_{end
1:05d}_m{motion:.4f}"

    output_annotations.append(
        keypoints_to_annotation(
            name,
            idle_label_id,
            player_keypoints[player_name][start:end],
            img_shape,
            source_frame_dir=name,
        )
    )

dataset = {
    "split": split_by_class(output_annotations, args.val_ratio, rng),
    "annotations": output_annotations,
}

args.output_dataset.parent.mkdir(parents=True, exist_ok=True)
with args.output_dataset.open("wb") as f:
    pickle.dump(dataset, f)

with args.output_labels.open("w", encoding="utf-8") as f:
    json.dump(label_map, f, indent=2)

```

```

        f.write("\n")

    id_to_label = {idx: label for label, idx in label_map.items()}
    counts = Counter(int(ann["label"]) for ann in output_annotations)
    print(f"Saved dataset: {args.output_dataset}")
    print(f"Saved labels: {args.output_labels}")
    print(f"Annotated clips: {len(output_annotations) - len(chosen_idle)}")
    print(f"Idle candidates: {len(idle_candidates)}")
    print(f"Idle chosen: {len(chosen_idle)}")

    print(f"Train/val sizes:
{len(dataset['split']['train'])}/{len(dataset['split']['val'])}")
    print("Class counts:")

    for label_id in sorted(counts):
        print(f"        {label_id:2d}: {id_to_label[label_id]} =
{counts[label_id]}")

if __name__ == "__main__":
    main()

```


Lampiran 6 Konfigurasi training STGCN++ (tekken_stgcn_all_moves_idle.py):

```
_base_ = '../_base_/default_runtime.py'

import json
import os

data_dir = '/Users/julianyang/Mine/Kuliah/SMT 6/CV/UAS/practice_videos'
ann_file = os.path.join(data_dir, 'skeleton_dataset_all_moves_idle.pkl')
label_file = os.path.join(data_dir, 'move_labels_all_moves_idle.json')

with open(label_file) as label_file_obj:
    num_classes = len(json.load(label_file_obj))
del label_file_obj

model = dict(
    type='RecognizerGCN',
    backbone=dict(
        type='STGCN',
        gcn_adaptive='init',
        gcn_with_res=True,
        tcn_type='mstcn',
        graph_cfg=dict(layout='coco', mode='spatial')),
    cls_head=dict(
        type='GCNHead',
        num_classes=num_classes,
        in_channels=256,
        topk=()))

dataset_type = 'PoseDataset'
```

```

train_pipeline = [
    dict(type='PreNormalize2D'),
    dict(type='GenSkeFeat', dataset='coco', feats=['j']),
    dict(type='UniformSampleFrames', clip_len=30),
    dict(type='PoseDecode'),
    dict(type='FormatGCNInput', num_person=1),
    dict(type='PackActionInputs')
]

val_pipeline = [
    dict(type='PreNormalize2D'),
    dict(type='GenSkeFeat', dataset='coco', feats=['j']),
    dict(
        type='UniformSampleFrames', clip_len=30, num_clips=1,
test_mode=True),
    dict(type='PoseDecode'),
    dict(type='FormatGCNInput', num_person=1),
    dict(type='PackActionInputs')
]

test_pipeline = [
    dict(type='PreNormalize2D'),
    dict(type='GenSkeFeat', dataset='coco', feats=['j']),
    dict(
        type='UniformSampleFrames', clip_len=30, num_clips=1,
        test_mode=True),
    dict(type='PoseDecode'),
    dict(type='FormatGCNInput', num_person=1),
    dict(type='PackActionInputs')
]

train_dataloader = dict(

```

```

        batch_size=16,
        num_workers=0,
        persistent_workers=False,
        sampler=dict(type='DefaultSampler', shuffle=True),
        dataset=dict(
            type='RepeatDataset',
            times=5,
            dataset=dict(
                type=dataset_type,
                ann_file=ann_file,
                pipeline=train_pipeline,
                split='train'))))
val_dataloader = dict(
    batch_size=16,
    num_workers=0,
    persistent_workers=False,
    sampler=dict(type='DefaultSampler', shuffle=False),
    dataset=dict(
        type=dataset_type,
        ann_file=ann_file,
        pipeline=val_pipeline,
        split='val',
        test_mode=True))
test_dataloader = dict(
    batch_size=1,
    num_workers=0,
    persistent_workers=False,
    sampler=dict(type='DefaultSampler', shuffle=False),
    dataset=dict(

```

```

        type=dataset_type,
        ann_file=ann_file,
        pipeline=test_pipeline,
        split='val',
        test_mode=True))

val_evaluator = [dict(type='AccMetric')]
test_evaluator = val_evaluator

train_cfg = dict(
    type='EpochBasedTrainLoop', max_epochs=24, val_begin=1, val_interval=1)
val_cfg = dict(type='ValLoop')
test_cfg = dict(type='TestLoop')

param_scheduler = [
    dict(
        type='CosineAnnealingLR',
        eta_min=0,
        T_max=24,
        by_epoch=True,
        convert_to_iter_based=True)
]

optim_wrapper = dict(
    optimizer=dict(
        type='SGD', lr=0.1, momentum=0.9, weight_decay=0.0005,
        nesterov=True))

default_hooks = dict(checkpoint=dict(interval=1, save_best='auto'),
                      logger=dict(interval=100))

```

```
auto_scale_lr = dict(enable=False, base_batch_size=128)
```

Lampiran 7 Script end-to-end Inference beserta evaluasi
(tekken_video_inference.py):

```
"""Run Tekken player tracking, YOLO-pose keypoints, and ST-GCN move
inference.

This script is intentionally close to the notebook pipeline:

YOLO detector -> OC-SORT player IDs -> cropped ROI YOLO-pose -> rolling
ST-GCN++.

It loads the MMAAction checkpoint directly, so exporting the classifier is
not
needed for normal local testing.
"""

from __future__ import annotations

import argparse
import json
import os
import platform
import subprocess
import sys
import time

from collections import deque
from dataclasses import dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from typing import Optional

os.environ.setdefault("MPLCONFIGDIR", "/private/tmp/matplotlib")
os.environ.setdefault("PYTORCH_ENABLE_MPS_FALLBACK", "1")
os.environ.setdefault("TORCH_FORCE_NO_WEIGHTS_ONLY_LOAD", "1")
```

```

Path(os.environ["MPLCONFIGDIR"]).mkdir(parents=True, exist_ok=True)

import cv2
import numpy as np
import torch
from ultralytics import YOLO

ROOT = Path(__file__).resolve().parent
MMACTION_DIR = ROOT / "mmaction2"
if str(MMACTION_DIR) not in sys.path:
    sys.path.insert(0, str(MMACTION_DIR))

from boxmot.trackers.ocsort.ocsort import OcSort # noqa: E402
from mmaction.apis import init_recognizer # noqa: E402
from mmengine.dataset import Compose, pseudo_collate # noqa: E402
from mmengine.registry import init_default_scope # noqa: E402

COCO_EDGES = (
    (5, 7), (7, 9),
    (6, 8), (8, 10),
    (5, 6),
    (5, 11), (6, 12),
    (11, 12),
    (11, 13), (13, 15),
    (12, 14), (14, 16),
    (0, 1), (0, 2), (1, 3), (2, 4),
)

```

```

@dataclass
class Prediction:
    label: str = "warming up"
    confidence: float = 0.0
    raw_label: str = ""

@dataclass
class PlayerState:
    name: str
    color: tuple[int, int, int]
    track_id: Optional[int] = None
    keypoints: deque[np.ndarray] = field(default_factory=deque)
    prediction: Prediction = field(default_factory=Prediction)
    pending_prediction: Prediction = field(default_factory=Prediction)
    pending_count: int = 0
    missing_frames: int = 0

@dataclass
class PipelineMetrics:
    """Collect rolling and run-level end-to-end pipeline measurements."""

    window_size: int = 60
    warmup_frames: int = 30
    stages: tuple[str, ...] = (
        "input",

```

```

        "det",

        "pose",

        "action",

        "render",

        "output",

        "e2e",

    )

    rolling: dict[str, deque[float]] = field(init=False)
    samples: dict[str, list[float]] = field(init=False)
    completed_frames: int = 0

    def __post_init__(self) -> None:

        self.window_size = max(1, self.window_size)

        self.warmup_frames = max(0, self.warmup_frames)

        self.rolling = {

            stage: deque(maxlen=self.window_size) for stage in self.stages

        }

        self.samples = {stage: [] for stage in self.stages}

    def record(self, timings_ms: dict[str, float]) -> None:

        """Record one completed frame, excluding configurable warm-up
frames."""

        self.completed_frames += 1

        if self.completed_frames <= self.warmup_frames:

            return

        for stage in self.stages:

            value = float(timings_ms.get(stage, 0.0))

            self.rolling[stage].append(value)

            self.samples[stage].append(value)

```

```

@property
def measured_frames(self) -> int:
    return len(self.samples["e2e"])

def snapshot(self) -> dict[str, float | int | bool]:
    """Return rolling values for the on-video performance panel."""
    if not self.rolling["e2e"]:
        return {
            "ready": False,
            "warmup_remaining": max(
                0, self.warmup_frames - self.completed_frames
            ),
        }

    values: dict[str, float | int | bool] = {"ready": True}
    for stage in self.stages:
        values[stage] = float(np.mean(self.rolling[stage]))
    values["fps"] = 1000.0 / max(float(values["e2e"]), 1e-9)
    values["window_frames"] = len(self.rolling["e2e"])
    return values

def summary(self) -> dict[str, object]:
    """Return aggregate latency percentiles and pipeline throughput."""
    if not self.samples["e2e"]:
        return {
            "measured_frames": 0,
            "note": "No post-warm-up frames were measured.",
        }

```

```

        summary: dict[str, object] = {
            "measured_frames": self.measured_frames,
            "warmup_frames": min(self.completed_frames,
self.warmup_frames),
            "latency_ms": {},
        }

        latency = summary["latency_ms"]
        assert isinstance(latency, dict)

        for stage in self.stages:
            values = np.asarray(self.samples[stage], dtype=np.float64)
            latency[stage] = {
                "mean": float(np.mean(values)),
                "median": float(np.median(values)),
                "p95": float(np.percentile(values, 95)),
                "std": float(np.std(values)),
            }

        e2e_mean_ms = float(np.mean(self.samples["e2e"]))
        summary["pipeline_fps"] = 1000.0 / max(e2e_mean_ms, 1e-9)

        return summary

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Tekken YOLO-pose + ST-GCN++ video inference demo."
    )
    parser.add_argument(
        "--video",
        type=Path,

```

```

        default=ROOT / "practice_videos" / "Bryan_LR_Complete.mp4",
        help="Input video path.",
    )
    parser.add_argument(
        "--output",
        type=Path,
        default=ROOT / "practice_videos" /
"tekken_stgc_n_demo_v13_no_interp_offsets_m3_0_6_0.75.mp4",
        help="Output visualization video path.",
    )
    parser.add_argument(
        "--detector",
        type=Path,
        default=ROOT / "runs" / "detect" / "practice_m" / "weights" /
"best.pt",
        help="YOLO player detector weights.",
    )
    parser.add_argument(
        "--pose-model",
        type=Path,
        default=ROOT / "yolo11l-pose.pt",
        help="YOLO pose weights. Use yolo11m-pose.pt for faster or yolo11x-
pose.pt for stronger.",
    )
    parser.add_argument(
        "--config",
        type=Path,
        default=ROOT
/ "mmaction2"
/ "configs"
/ "skeleton"

```

```

        / "stgcnpp"

        / "tekken_stgcn_all_moves_idle.py",

        help="MMAction2 ST-GCN++ all-moves + idle config.",
    )

    parser.add_argument(
        "--checkpoint",
        type=Path,
        default=ROOT

        / "mmaction2"

        / "work_dirs"

        / "tekken_stgcn_v13_all_moves_idle_no_interp_offsets_m3_0_6"

        / "best_acc_top1_epoch_16.pth",
        help="Trained ST-GCN++ checkpoint.",
    )

    parser.add_argument(
        "--labels",
        type=Path,
        default=ROOT / "practice_videos" /
"move_labels_all_moves_idle.json",
        help="Label map JSON used by the all-moves + idle dataset.",
    )

    parser.add_argument("--device", default="mps", help="YOLO device, e.g.
mps/cpu/cuda:0.")

    parser.add_argument(
        "--action-device",
        default="cpu",
        help="ST-GCN device. CPU is usually enough and avoids MPS checkpoint
quirks.",
    )

    parser.add_argument("--det-conf", type=float, default=0.50,
help="Detector confidence.")

```

```

    parser.add_argument("--det-iou", type=float, default=0.35,
help="Detector NMS IoU.")

    parser.add_argument("--pose-conf", type=float, default=0.25, help="Pose
confidence.")

    parser.add_argument("--pose-iou", type=float, default=0.35, help="Pose
NMS IoU.")

    parser.add_argument("--pose-imsz", type=int, default=384, help="YOLO-
pose image size.")

    parser.add_argument(
        "--padding",
        type=int,
        default=0,
        help="Pixels added around each tracked box before pose extraction.",
    )

    parser.add_argument(
        "--window-size",
        type=int,
        default=30,
        help="Rolling keypoint window length sent to ST-GCN++.",
    )

    parser.add_argument(
        "--kp-interpolation",
        choices=("window", "none"),
        default="none",
        help=(
            "How to handle missing keypoints before ST-GCN++. "
            "'window' keeps the v10 behavior by interpolating inside the
current rolling window; "
            "'none' zero-fills missing keypoints for the v12/v13 no-
interpolation setup."
        ),
    )

```

```

parser.add_argument(
    "--predict-every",
    type=int,
    default=3,
    help="Run ST-GCN once every N frames per player.",
)

parser.add_argument(
    "--action-conf",
    type=float,
    default=0.75,
    help="Display 'uncertain' when the top action probability is below this.",
)

parser.add_argument(
    "--stable-predictions",
    type=int,
    default=4,
    help="Only update the visible action after the same label appears this many prediction steps in a row.",
)

parser.add_argument(
    "--clear-missing-frames",
    type=int,
    default=8,
    help="Clear a player's keypoint buffer/prediction after this many consecutive missing tracked boxes.",
)

parser.add_argument(
    "--min-box-area-ratio",
    type=float,
    default=0.015,

```

```

        help="Prefer tracked boxes above this frame-area ratio when choosing
initial players.",
    )

    parser.add_argument(
        "--lost-reset-frames",
        type=int,
        default=45,
        help="Re-select player IDs if both assigned players disappear this
long.",
    )

    parser.add_argument(
        "--character-name",
        default="",
        help="Optional character name shown in the large prediction panels.
Empty means show P1/P2.",
    )

    parser.add_argument(
        "--strip-label-prefix",
        default="Bryan ",
        help="Prefix removed from displayed move labels. Use '' to show full
class names.",
    )

    parser.add_argument(
        "--hide-labels",
        default="idle",
        help="Comma-separated labels to hide in the large panels, e.g. idle
or idle,unknown.",
    )

    parser.add_argument(
        "--panel-alpha",
        type=float,
        default=0.58,

```

```

        help="Opacity of the bottom prediction panels.",
    )
    parser.add_argument(
        "--classify-players",
        choices=("both", "p1", "p2"),
        default="both",
        help="Choose which tracked player gets ST-GCN move predictions.",
    )
    parser.add_argument(
        "--metrics-window",
        type=int,
        default=60,
        help="Completed-frame rolling window used by the latency/FPS overlay.",
    )
    parser.add_argument(
        "--metrics-warmup",
        type=int,
        default=30,
        help="Initial frames excluded from the aggregate runtime result.",
    )
    parser.add_argument(
        "--metrics-json",
        type=Path,
        default=None,
        help=(
            "Runtime summary JSON path. By default, write "
            "<output_stem>_metrics.json next to the output video."
        ),
    )
)

```

```

parser.add_argument(
    "--device-label",
    default="",
    help=(
        "Optional device text for the overlay/report, e.g. "
        "'MacBook Pro, Apple M5, 24 GB'. Empty means auto-detect."
    ),
)

parser.add_argument(
    "--realtime-target-fps",
    type=float,
    default=0.0,
    help="FPS target used by the real-time status. 0 uses the input-
video FPS.",
)

parser.add_argument("--max-frames", type=int, default=0, help="Stop
after N frames. 0 means full video.")

parser.add_argument("--show", action="store_true", help="Show a live
OpenCV window.")

return parser.parse_args()

def require_path(path: Path, label: str) -> None:
    if not path.exists():
        raise FileNotFoundError(f"{label} not found: {path}")

def validate_device(device: str, label: str) -> None:
    """Prevent a requested accelerator from being reported after CPU
    fallback."""
    normalized = device.lower().strip()

```

```

        if normalized == "mps" and not torch.backends.mps.is_available():
            raise RuntimeError(
                f"{label} requested MPS, but torch.backends.mps.is_available()
is False. "
                "Run where Apple MPS access is available or explicitly choose
CPU."
            )
        if normalized.startswith("cuda") and not torch.cuda.is_available():
            raise RuntimeError(
                f"{label} requested {device}, but CUDA is not available. "
                "Choose an available device explicitly."
            )

def load_labels(path: Path) -> dict[int, str]:
    with path.open("r", encoding="utf-8") as f:
        label_map = json.load(f)

    id_to_label: dict[int, str] = {}
    for key, value in label_map.items():
        if isinstance(value, int):
            id_to_label[value] = str(key)
        else:
            id_to_label[int(key)] = str(value)
    return dict(sorted(id_to_label.items()))

def make_tracker(device: str) -> OcSort:
    return OcSort(
        half=device != "cpu",

```

```

        device=device,

        max_age=90,

        min_hits=2,

        iou_threshold=0.15,

        det_thresh=0.20,

    )

def empty_keypoints() -> np.ndarray:

    return np.full((17, 3), np.nan, dtype=np.float32)

def box_iou_xyxy(box: np.ndarray, boxes: np.ndarray) -> np.ndarray:

    box = np.asarray(box, dtype=np.float32)

    boxes = np.asarray(boxes, dtype=np.float32)

    if boxes.size == 0:

        return np.array([], dtype=np.float32)

    x1 = np.maximum(box[0], boxes[:, 0])
    y1 = np.maximum(box[1], boxes[:, 1])
    x2 = np.minimum(box[2], boxes[:, 2])
    y2 = np.minimum(box[3], boxes[:, 3])

    inter = np.maximum(0, x2 - x1) * np.maximum(0, y2 - y1)

    box_area = max(0, box[2] - box[0]) * max(0, box[3] - box[1])
    boxes_area = np.maximum(0, boxes[:, 2] - boxes[:, 0]) * np.maximum(
        0, boxes[:, 3] - boxes[:, 1]
    )

    union = box_area + boxes_area - inter

```

```

        return np.divide(inter, union, out=np.zeros_like(inter,
dtype=np.float32), where=union > 0)

def choose_pose_candidate(
    boxes_xyxy: np.ndarray, box_scores: np.ndarray, target_box:
Optional[np.ndarray] = None
) -> int:
    if boxes_xyxy.shape[0] == 0:
        return 0

    if target_box is None:
        return int(np.argmax(box_scores))

    target_box = np.asarray(target_box, dtype=np.float32)
    ious = box_iou_xyxy(target_box, boxes_xyxy)

    target_center = np.array(
        [(target_box[0] + target_box[2]) / 2, (target_box[1] +
target_box[3]) / 2],
        dtype=np.float32,
    )

    candidate_centers = np.column_stack(
        ((boxes_xyxy[:, 0] + boxes_xyxy[:, 2]) / 2, (boxes_xyxy[:, 1] +
boxes_xyxy[:, 3]) / 2)
    )

    distances = np.linalg.norm(candidate_centers - target_center, axis=1)

    target_diag = max(
        1.0, np.linalg.norm([target_box[2] - target_box[0], target_box[3] -
target_box[1]])
    )

    center_score = 1.0 - np.clip(distances / target_diag, 0.0, 1.0)

```

```

match_score = (2.0 * ious) + center_score + (0.25 * box_scores)

return int(np.argmax(match_score))

def yolo_detect(model: YOLO, frame: np.ndarray, conf: float, iou: float,
device: str) -> np.ndarray:

    result = model(frame, conf=conf, iou=iou, classes=[0], device=device,
verbose=False)[0]

    if result.bboxes is None or len(result.bboxes) == 0:

        return np.empty((0, 6), dtype=np.float32)

    xyxy = result.bboxes.xyxy.cpu().numpy()
    scores = result.bboxes.conf.cpu().numpy().reshape(-1, 1)
    cls = result.bboxes.cls.cpu().numpy().reshape(-1, 1)

    return np.hstack((xyxy, scores, cls)).astype(np.float32)

def ocsort_tracking(tracker: OcSort, detections: np.ndarray, frame:
np.ndarray) -> np.ndarray:

    if detections.shape[0] == 0:

        return np.empty((0, 5), dtype=np.float32)

    tracks = tracker.update(detections, frame)

    if tracks is None or len(tracks) == 0:

        return np.empty((0, 5), dtype=np.float32)

    ids = tracks.id.astype(np.float32).reshape(-1, 1)
    boxes = tracks.xyxy.astype(np.float32)

    return np.hstack((ids, boxes))

```

```

def pad_box(frame_h: int, frame_w: int, box: np.ndarray, padding: int) ->
np.ndarray:

    obj_id, x1, y1, x2, y2 = box

    x1 = max(0, int(x1) - padding)
    y1 = max(0, int(y1) - padding)
    x2 = min(frame_w, int(x2) + padding)
    y2 = min(frame_h, int(y2) + padding)

    return np.array([obj_id, x1, y1, x2, y2], dtype=np.float32)


def crop_roi(frame: np.ndarray, box: np.ndarray) -> np.ndarray:

    _, x1, y1, x2, y2 = box.astype(int)

    return frame[y1:y2, x1:x2]


def track_box_to_roi_box(track_box: np.ndarray, crop_box: np.ndarray) ->
np.ndarray:

    _, tx1, ty1, tx2, ty2 = track_box

    _, cx1, cy1, _, _ = crop_box

    return np.array([tx1 - cx1, ty1 - cy1, tx2 - cx1, ty2 - cy1],
dtype=np.float32)


def run_yolopose(
    model: YOLO,
    image: np.ndarray,
    input_size: int,
    conf: float,
    iou: float,
    device: str,
    target_box: Optional[np.ndarray] = None,

```

```

) -> np.ndarray:

    if image is None or image.size == 0:

        return empty_keypoints()

    roi_height, roi_width = image.shape[:2]
    if roi_height == 0 or roi_width == 0:

        return empty_keypoints()

    result = model(
        image,
        imsz=input_size,
        conf=conf,
        iou=iou,
        device=device,
        verbose=False,
    )[0]

    if result.keypoints is None or len(result.keypoints) == 0:

        return empty_keypoints()

    xy = result.keypoints.xy.cpu().numpy()
    kp_conf = result.keypoints.conf

    if kp_conf is None:

        scores = np.ones(xy.shape[:2], dtype=np.float32)
    else:

        scores = kp_conf.cpu().numpy()

    if xy.shape[0] == 0:

        return empty_keypoints()

```

```

    if result.bboxes is not None and len(result.bboxes) > 0:

        boxes_xyxy = result.bboxes.xyxy.cpu().numpy()

        box_scores = result.bboxes.conf.cpu().numpy()

        best_idx = choose_pose_candidate(boxes_xyxy, box_scores,
target_box)

        best_idx = min(best_idx, xy.shape[0] - 1)

    else:

        mean_scores = np.nanmean(scores, axis=1)

        best_idx = 0 if np.all(np.isnan(mean_scores)) else
int(np.nanargmax(mean_scores))

    keypoints = empty_keypoints()
    points_xy = xy[best_idx]
    point_scores = scores[best_idx]
    num_points = min(17, points_xy.shape[0])

    keypoints[:num_points, 0] = points_xy[:num_points, 1] / roi_height
    keypoints[:num_points, 1] = points_xy[:num_points, 0] / roi_width
    keypoints[:num_points, 2] = point_scores[:num_points]

    return keypoints

def normalize_points_to_full_frame(
    kp_array: np.ndarray, box: np.ndarray, full_height: int, full_width: int
) -> np.ndarray:
    _, x1, y1, x2, y2 = box

    roi_height = max(1.0, y2 - y1)
    roi_width = max(1.0, x2 - x1)

    kp_full = empty_keypoints()

```

```

    for i, (y_norm, x_norm, score) in enumerate(kp_array):
        if not np.isfinite(y_norm) or not np.isfinite(x_norm):
            continue

        y_full = ((y_norm * roi_height) + y1) / full_height
        x_full = ((x_norm * roi_width) + x1) / full_width
        kp_full[i] = [y_full, x_full, score]
    return kp_full

def interpolate_window(kp_window: deque[np.ndarray]) -> np.ndarray:
    window = np.asarray(kp_window, dtype=np.float32).copy()

    if window.ndim != 3:
        raise ValueError(f"Expected keypoint window shape (T, 17, 3), got {window.shape}")

    frame_idx = np.arange(window.shape[0])

    for joint in range(window.shape[1]):
        for coord in range(2):
            data = window[:, joint, coord]
            mask = np.isfinite(data)

            if mask.any():
                window[:, joint, coord] = np.interp(frame_idx,
frame_idx[mask], data[mask])
            else:
                window[:, joint, coord] = 0.0

    score = window[:, joint, 2]
    score[~np.isfinite(score)] = 0.0
    window[:, joint, 2] = score

```

```

    return window

def zero_fill_window(kp_window: deque[np.ndarray]) -> np.ndarray:
    window = np.asarray(kp_window, dtype=np.float32).copy()
    if window.ndim != 3:
        raise ValueError(f"Expected keypoint window shape (T, 17, 3), got {window.shape}")
    return np.nan_to_num(window, nan=0.0, posinf=0.0, neginf=0.0)

def prepare_action_window(kp_window: deque[np.ndarray], kp_interpolation:
str) -> np.ndarray:
    if kp_interpolation == "window":
        # v10 behavior: fill missing coordinates only inside the current 30-
        # frame window.
        return interpolate_window(kp_window)

    # v12 behavior: preserve missing keypoints as missing, then encode them
    # as zero + zero score.
    # window = interpolate_window(kp_window)
    return zero_fill_window(kp_window)

def build_action_sample(
    kp_window: deque[np.ndarray],
    frame_h: int,
    frame_w: int,
    kp_interpolation: str,
) -> dict:
    window = prepare_action_window(kp_window, kp_interpolation)

```

```

keypoint_xy = window[:, :, [1, 0]][None].astype(np.float32)
keypoint_score = window[:, :, 2][None].astype(np.float32)

return {
    "frame_dir": "realtime_window",
    "label": -1,
    "img_shape": (frame_h, frame_w),
    "original_shape": (frame_h, frame_w),
    "total_frames": keypoint_xy.shape[1],
    "keypoint": keypoint_xy,
    "keypoint_score": keypoint_score,
}

def init_action_model(config: Path, checkpoint: Path, device: str):
    model = init_recognizer(str(config), str(checkpoint), device=device)
    init_default_scope(model.cfg.get("default_scope", "mmaction"))
    pipeline = Compose(model.cfg.test_pipeline)
    model.eval()
    return model, pipeline

def softmax_if_needed(scores: np.ndarray) -> np.ndarray:
    scores = scores.astype(np.float32)
    if np.all(scores >= 0) and np.isclose(scores.sum(), 1.0, atol=1e-3):
        return scores
    scores = scores - np.max(scores)
    exp_scores = np.exp(scores)
    return exp_scores / np.maximum(exp_scores.sum(), 1e-12)

```

```

def predict_action(
    model,
    pipeline: Compose,
    kp_window: deque[np.ndarray],
    frame_h: int,
    frame_w: int,
    id_to_label: dict[int, str],
    confidence_threshold: float,
    kp_interpolation: str,
) -> Prediction:
    sample = build_action_sample(kp_window, frame_h, frame_w,
    kp_interpolation)

    data = pipeline(sample)
    data = pseudo_collate([data])

    with torch.no_grad():
        result = model.test_step(data)[0]

    scores = result.pred_score.detach().cpu().numpy()
    probs = softmax_if_needed(scores)
    pred_idx = int(np.argmax(probs))
    raw_label = id_to_label.get(pred_idx, f"class_{pred_idx}")
    confidence = float(probs[pred_idx])
    label = raw_label if confidence >= confidence_threshold else "uncertain"

    return Prediction(label=label, confidence=confidence,
    raw_label=raw_label)

def should_classify_player(player: PlayerState, mode: str) -> bool:

```

```

    return mode == "both" or (mode == "p1" and player.name == "P1") or (
        mode == "p2" and player.name == "P2"
    )

def reset_player_prediction(player: PlayerState, label: str = "warming up")
-> None:
    player.keypoints.clear()
    player.prediction = Prediction(label=label)
    player.pending_prediction = Prediction(label=label)
    player.pending_count = 0

def update_stable_prediction(player: PlayerState, new_prediction:
Prediction, stable_steps: int) -> None:
    stable_steps = max(1, stable_steps)

    if new_prediction.label == "uncertain":
        return

    if new_prediction.label == player.pending_prediction.label:
        player.pending_count += 1
    else:
        player.pending_prediction = new_prediction
        player.pending_count = 1

    if player.pending_count >= stable_steps:
        player.prediction = new_prediction

def select_initial_player_ids(

```

```

    id_box_array: np.ndarray, frame_h: int, frame_w: int, min_area_ratio:
float
) -> tuple[Optional[int], Optional[int]]:

    if id_box_array.shape[0] < 2:

        return None, None

    boxes = id_box_array[:, 1:5]

    areas = np.maximum(0, boxes[:, 2] - boxes[:, 0]) * np.maximum(0, boxes[:,
3] - boxes[:, 1])

    min_area = frame_h * frame_w * min_area_ratio

    candidate_indices = np.where(areas >= min_area)[0]

    if candidate_indices.size < 2:

        candidate_indices = np.argsort(-areas)[:2]

    else:

        candidate_indices = candidate_indices[np.argsort(-
areas[candidate_indices])[:2]]

    candidates = id_box_array[candidate_indices]

    centers_x = (candidates[:, 1] + candidates[:, 3]) / 2

    order = np.argsort(centers_x)

    return int(candidates[order[0], 0]), int(candidates[order[1], 0])

def find_track_box(id_box_array: np.ndarray, track_id: Optional[int]) ->
Optional[np.ndarray]:

    if track_id is None or id_box_array.size == 0:

        return None

    matches = id_box_array[id_box_array[:, 0] == track_id]

    if matches.size == 0:

        return None

    return matches[0]

```

```

def extract_player_keypoints(
    pose_model: YOLO,
    frame: np.ndarray,
    track_box: Optional[np.ndarray],
    args: argparse.Namespace,
) -> tuple[np.ndarray, Optional[np.ndarray]]:
    frame_h, frame_w = frame.shape[:2]

    if track_box is None:
        return empty_keypoints(), None

    padded_box = pad_box(frame_h, frame_w, track_box, args.padding)
    roi = crop_roi(frame, padded_box)
    target_box = track_box_to_roi_box(track_box, padded_box)
    kp_raw = run_yolopose(
        pose_model,
        roi,
        input_size=args.pose_imgsz,
        conf=args.pose_conf,
        iou=args.pose_iou,
        device=args.device,
        target_box=target_box,
    )

    kp_full = normalize_points_to_full_frame(kp_raw, padded_box, frame_h,
frame_w)

    return kp_full, padded_box

def draw_keypoints(

```

```

frame: np.ndarray,
keypoints: np.ndarray,
color: tuple[int, int, int],
score_threshold: float = 0.05,
) -> None:

    frame_h, frame_w = frame.shape[:2]

    for a, b in COCO_EDGES:

        ya, xa, ca = keypoints[a]

        yb, xb, cb = keypoints[b]

        if not all(np.isfinite(v) for v in (ya, xa, yb, xb)):

            continue

        if (np.isfinite(ca) and ca < score_threshold) or (np.isfinite(cb)
and cb < score_threshold):

            continue

        pt_a = (int(np.clip(xa * frame_w, 0, frame_w - 1)), int(np.clip(ya
* frame_h, 0, frame_h - 1)))

        pt_b = (int(np.clip(xb * frame_w, 0, frame_w - 1)), int(np.clip(yb
* frame_h, 0, frame_h - 1)))

        cv2.line(frame, pt_a, pt_b, color, 2, lineType=cv2.LINE_AA)

    for y, x, score in keypoints:

        if not np.isfinite(y) or not np.isfinite(x):

            continue

        if np.isfinite(score) and score < score_threshold:

            continue

        point = (int(np.clip(x * frame_w, 0, frame_w - 1)), int(np.clip(y *
frame_h, 0, frame_h - 1)))

        cv2.circle(frame, point, 3, color, thickness=-1,
lineType=cv2.LINE_AA)

```

```

def draw_box_and_label(
    frame: np.ndarray,
    box: Optional[np.ndarray],
    player: PlayerState,
) -> None:
    if box is None:
        return

    obj_id, x1, y1, x2, y2 = box.astype(int)

    cv2.rectangle(frame, (x1, y1), (x2, y2), player.color, 2,
lineType=cv2.LINE_AA)

    text = f"{player.name} ID:{obj_id}"
    text_y = max(24, y1 - 8)
    cv2.putText(
        frame,
        text,
        (x1, text_y),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.65,
        player.color,
        2,
        lineType=cv2.LINE_AA,
    )

def fit_text_scale(
    text: str,
    max_width: int,
    font_face: int,

```

```

        initial_scale: float,
        thickness: int,
        min_scale: float = 0.45,
    ) -> float:
        scale = initial_scale
        while scale > min_scale:
            text_width = cv2.getTextSize(text, font_face, scale,
thickness)[0][0]
            if text_width <= max_width:
                return scale
            scale -= 0.05
        return min_scale

def draw_translucent_rect(
    frame: np.ndarray,
    x1: int,
    y1: int,
    x2: int,
    y2: int,
    color: tuple[int, int, int],
    alpha: float,
) -> None:
    overlay = frame.copy()
    cv2.rectangle(overlay, (x1, y1), (x2, y2), color, thickness=-1)
    cv2.addWeighted(overlay, alpha, frame, 1.0 - alpha, 0, dst=frame)

def display_move_label(label: str, strip_prefix: str) -> str:
    if strip_prefix and label.startswith(strip_prefix):

```

```

        return label[len(strip_prefix):].strip()

    return label

def parse_hidden_labels(hidden_labels: str) -> set[str]:
    return {item.strip().lower() for item in hidden_labels.split(",") if
item.strip()}

def draw_prediction_panels(
    frame: np.ndarray,
    players: list[PlayerState],
    character_name: str,
    strip_label_prefix: str,
    hidden_labels: set[str],
    alpha: float,
) -> None:
    frame_h, frame_w = frame.shape[:2]
    margin_x = max(24, int(frame_w * 0.025))
    panel_gap = max(28, int(frame_w * 0.08))
    panel_w = int((frame_w - (2 * margin_x) - panel_gap) / 2)
    panel_h = max(120, int(frame_h * 0.24))
    panel_y1 = frame_h - panel_h - max(18, int(frame_h * 0.045))
    panel_y2 = panel_y1 + panel_h

    panel_positions = [
        (margin_x, panel_y1, margin_x + panel_w, panel_y2),
        (frame_w - margin_x - panel_w, panel_y1, frame_w - margin_x,
panel_y2),
    ]

```

```

font = cv2.FONT_HERSHEY_SIMPLEX

for player, (x1, y1, x2, y2) in zip(players, panel_positions):

    draw_translucent_rect(frame, x1, y1, x2, y2, (35, 35, 35), alpha)

    pad_x = max(20, int(panel_w * 0.04))

    title = character_name if character_name else player.name

    raw_move = player.prediction.label

    move = display_move_label(raw_move, strip_label_prefix)

    should_hide = raw_move.lower() in hidden_labels or move.lower() in
hidden_labels

    if should_hide:

        move = ""

    elif player.prediction.confidence > 0:

        move = f"{move} {player.prediction.confidence:.2f}"

    title_scale = fit_text_scale(title, panel_w - 2 * pad_x, font, 1.35,
2)

    move_scale = fit_text_scale(move, panel_w - 2 * pad_x, font, 1.20,
2)

    title_y = y1 + int(panel_h * 0.27)

    move_y = y1 + int(panel_h * 0.58)

    cv2.putText(

        frame,

        title,

        (x1 + pad_x, title_y),

        font,

        title_scale,

        (255, 255, 255),

        2,

```

```

        lineType=cv2.LINE_AA,

    )

    if move:

        cv2.putText(

            frame,

            move,

            (x1 + pad_x, move_y),

            font,

            move_scale,

            (255, 255, 255),

            2,

            lineType=cv2.LINE_AA,

        )

def draw_status(

    frame: np.ndarray,

    frame_idx: int,

    metrics: dict[str, float | int | bool],

    source_fps: float,

    target_fps: float,

    device_label: str,

    window_size: int,

) -> None:

    frame_h, frame_w = frame.shape[:2]

    panel_x1 = 12

    panel_y1 = 12

    panel_x2 = min(frame_w - 12, max(620, int(frame_w * 0.74)))

    panel_y2 = min(frame_h - 12, 126)

```

```

draw_translucent_rect(
    frame, panel_x1, panel_y1, panel_x2, panel_y2, (20, 20, 20), 0.72
)

if not bool(metrics.get("ready", False)):
    remaining = int(metrics.get("warmup_remaining", 0))
    lines = [
        f"Pipeline benchmark warming up: {remaining} frame(s) remaining",
        f"Source {source_fps:.2f} FPS | target {target_fps:.2f} FPS | action window {window_size}",
        f"Device: {device_label}",
    ]
    status_color = (0, 215, 255)
else:
    e2e_ms = float(metrics["e2e"])
    pipeline_fps = float(metrics["fps"])
    realtime_ratio = pipeline_fps / max(target_fps, 1e-9)
    realtime_ok = pipeline_fps >= target_fps
    status = "MEETS TARGET" if realtime_ok else "BELOW TARGET"
    status_color = (70, 220, 90) if realtime_ok else (40, 170, 255)
    lines = [
        (
            f"E2E {e2e_ms:.2f} ms | pipeline {pipeline_fps:.2f} FPS | "
            f"target {target_fps:.2f} FPS ({realtime_ratio:.2f}x) | {status}"
        ),
        (
            f"input {float(metrics['input']):.2f} | "
            f"det+track {float(metrics['det']):.2f} | "
            f"pose {float(metrics['pose']):.2f} | "

```

```

        f"action {float(metrics['action']):.2f} | "
        f"render+output {float(metrics['render']) +
float(metrics['output']):.2f} ms"
    ),
    (
        f"Device: {device_label} | rolling "
        f"{int(metrics['window_frames'])} frame(s) | frame
{frame_idx}"
    ),
]

font = cv2.FONT_HERSHEY_SIMPLEX
max_text_width = panel_x2 - panel_x1 - 24
for line_idx, line in enumerate(lines):
    scale = fit_text_scale(
        line,
        max_text_width,
        font,
        initial_scale=0.56,
        thickness=1,
        min_scale=0.38,
    )
    color = status_color if line_idx == 0 else (245, 245, 245)
    cv2.putText(
        frame,
        line,
        (panel_x1 + 12, panel_y1 + 28 + (line_idx * 34)),
        font,
        scale,
        color,

```

```

        1,
        lineType=cv2.LINE_AA,
    )

def sysctl_value(name: str) -> str:
    """Read one macOS sysctl value without making it a runtime
    requirement."""
    try:
        result = subprocess.run(
            ["sysctl", "-n", name],
            check=True,
            capture_output=True,
            text=True,
            timeout=2,
        )
        return result.stdout.strip()
    except (OSError, subprocess.SubprocessError):
        return ""

def total_memory_gb() -> Optional[int]:
    try:
        total_bytes = os.sysconf("SC_PHYS_PAGES") *
os.sysconf("SC_PAGE_SIZE")
        return round(total_bytes / (1024**3))
    except (AttributeError, OSError, ValueError):
        return None

```

```

def mac_hardware_label() -> str:
    """Return non-identifying Mac model/chip/memory information."""
    if platform.system() != "Darwin":
        return ""
    try:
        result = subprocess.run(
            ["system_profiler", "SPHardwareDataType", "-json"],
            check=True,
            capture_output=True,
            text=True,
            timeout=5,
        )
        entries = json.loads(result.stdout).get("SPHardwareDataType", [])
        if not entries:
            return ""
        hardware = entries[0]
        parts = [
            hardware.get("machine_name", ""),
            hardware.get("chip_type", ""),
            hardware.get("physical_memory", ""),
        ]
        return ", ".join(str(part) for part in parts if part)
    except (OSError, subprocess.SubprocessError, json.JSONDecodeError):
        return ""

def build_device_label(args: argparse.Namespace) -> str:
    if args.device_label.strip():
        hardware = args.device_label.strip()

```

```

else:

    hardware = mac_hardware_label()

    if not hardware:

        chip = sysctl_value("machdep.cpu.brand_string")

        if not chip and platform.system() == "Darwin":

            chip = platform.processor()

        hardware = chip or f"{platform.system()}
{platform.machine()}".strip()

        memory_gb = total_memory_gb()

        if memory_gb:

            hardware = f"{hardware}, {memory_gb} GB"

    return (

        f"{hardware} | detector/pose {args.device.upper()} | "

        f"ST-GCN {args.action_device.upper()}"

    )

def write_metrics_report(

    path: Path,

    args: argparse.Namespace,

    metrics: PipelineMetrics,

    source_fps: float,

    target_fps: float,

    device_label: str,

) -> dict[str, object]:

    report = metrics.summary()

    pipeline_fps = float(report.get("pipeline_fps", 0.0))

    report.update(

        {

```

```

        "created_at_utc": datetime.now(timezone.utc).isoformat(),
        "measurement_scope": (
            "Frame read -> detector/tracker -> ROI pose -> ST-GCN -> "
            "overlay rendering -> video write/display. Model loading is
excluded."
        ),
        "input_video": str(args.video),
        "output_video": str(args.output),
        "completed_frames": metrics.completed_frames,
        "source_fps": source_fps,
        "realtime_target_fps": target_fps,
        "realtime_ratio": pipeline_fps / max(target_fps, 1e-9),
        "meets_realtime_target": pipeline_fps >= target_fps,
        "device": device_label,
        "detector_weights": str(args.detector),
        "pose_weights": str(args.pose_model),
        "pose_imgsz": args.pose_imgsz,
        "action_checkpoint": str(args.checkpoint),
        "yolo_device": args.device,
        "action_device": args.action_device,
        "rolling_window_frames": metrics.window_size,
    }
)
path.parent.mkdir(parents=True, exist_ok=True)
with path.open("w", encoding="utf-8") as report_file:
    json.dump(report, report_file, indent=2)
return report

def process_video(args: argparse.Namespace) -> None:

```

```

require_path(args.video, "Input video")
require_path(args.detector, "Detector weights")
require_path(args.pose_model, "Pose weights")
require_path(args.config, "MMAction config")
require_path(args.checkpoint, "ST-GCN checkpoint")
require_path(args.labels, "Label map")

validate_device(args.device, "Detector/pose")
validate_device(args.action_device, "ST-GCN")


id_to_label = load_labels(args.labels)

detector = YOLO(str(args.detector))
pose_model = YOLO(str(args.pose_model))

action_model, action_pipeline = init_action_model(args.config,
args.checkpoint, args.action_device)

tracker = make_tracker(args.device)


cap = cv2.VideoCapture(str(args.video))
if not cap.isOpened():
    raise RuntimeError(f"Could not open video: {args.video}")


fps = cap.get(cv2.CAP_PROP_FPS) or 60.0
frame_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
args.output.parent.mkdir(parents=True, exist_ok=True)
writer = cv2.VideoWriter(
    str(args.output),
    cv2.VideoWriter_fourcc(*"mp4v"),
    fps,
    (frame_w, frame_h),
)

```

```

if not writer.isOpened():
    raise RuntimeError(f"Could not create output video: {args.output}")

players = [
    PlayerState(name="P1", color=(0, 255, 0),
keypoints=deque(maxlen=args.window_size)),
    PlayerState(name="P2", color=(255, 60, 0),
keypoints=deque(maxlen=args.window_size)),
]

player_set = False

both_missing_frames = 0

frame_idx = 0

pipeline_metrics = PipelineMetrics(
    window_size=args.metrics_window,
    warmup_frames=args.metrics_warmup,
)

device_label = build_device_label(args)

target_fps = args.realtime_target_fps if args.realtime_target_fps > 0
else fps

metrics_path = args.metrics_json or args.output.with_name(
    f"{args.output.stem}_metrics.json"
)

print(f"Input: {args.video}")
print(f"Output: {args.output}")
print(f"Detector: {args.detector}")
print(f"Pose: {args.pose_model} imgsz={args.pose_imgsz}")
print(f"ST-GCN: {args.checkpoint}")
print(f"KP interpolation: {args.kp_interpolation}")
print(f"Benchmark device: {device_label}")
print(f"Real-time target: {target_fps:.2f} FPS")

```

```

print(f"Metrics report: {metrics_path}")
print(f"Labels: {id_to_label}")

while True:
    if args.max_frames and frame_idx >= args.max_frames:
        break

    frame_start = time.perf_counter()
    input_start = frame_start
    ok, frame = cap.read()
    if not ok:
        break

    timings_ms = {
        "input": (time.perf_counter() - input_start) * 1000,
        "det": 0.0,
        "pose": 0.0,
        "action": 0.0,
        "render": 0.0,
        "output": 0.0,
        "e2e": 0.0,
    }

    det_start = time.perf_counter()
    detections = yolo_detect(detector, frame, args.det_conf,
args.det_iou, args.device)

    id_box_array = ocsort_tracking(tracker, detections, frame)
    timings_ms["det"] = (time.perf_counter() - det_start) * 1000

    if not player_set and id_box_array.shape[0] >= 2:

```

```

        p1_id, p2_id = select_initial_player_ids(
            id_box_array, frame_h, frame_w, args.min_box_area_ratio
        )

    if p1_id is not None and p2_id is not None:
        players[0].track_id = p1_id
        players[1].track_id = p2_id
        player_set = True

    player_boxes = [find_track_box(id_box_array, player.track_id) for
player in players]

    if player_set and player_boxes[0] is None and player_boxes[1] is
None:

        both_missing_frames += 1

        if both_missing_frames > args.lost_reset_frames:
            tracker = make_tracker(args.device)
            for player in players:
                player.track_id = None
                reset_player_prediction(player)
            player_set = False
            both_missing_frames = 0
    else:
        both_missing_frames = 0

    pose_start = time.perf_counter()
    padded_boxes: list[Optional[np.ndarray]] = []
    for player, track_box in zip(players, player_boxes):
        if track_box is None:
            player.missing_frames += 1
            if player.missing_frames >= args.clear_missing_frames:
                reset_player_prediction(player, label="")

```

```

        else:

            player.missing_frames = 0

            kp_full, padded_box = extract_player_keypoints(pose_model,
frame, track_box, args)

            player.keypoints.append(kp_full)

            padded_boxes.append(padded_box)

            timings_ms["pose"] = (time.perf_counter() - pose_start) * 1000

            action_start = time.perf_counter()

            if frame_idx % max(1, args.predict_every) == 0:

                for player in players:

                    if not should_classify_player(player,
args.classify_players):

                        player.prediction = Prediction(label="not evaluated")

                        continue

                    if len(player.keypoints) == args.window_size:

                        new_prediction = predict_action(

                            action_model,

                            action_pipeline,

                            player.keypoints,

                            frame_h,

                            frame_w,

                            id_to_label,

                            args.action_conf,

                            args.kp_interpolation,

                        )

                        update_stable_prediction(player, new_prediction,
args.stable_predictions)

                        timings_ms["action"] = (time.perf_counter() - action_start) * 1000

```

```

render_start = time.perf_counter()

for player, padded_box in zip(players, padded_boxes):
    draw_box_and_label(frame, padded_box, player)

    if len(player.keypoints) > 0:
        draw_keypoints(frame, player.keypoints[-1], player.color)

draw_prediction_panels(
    frame,
    players,
    args.character_name,
    args.strip_label_prefix,
    parse_hidden_labels(args.hide_labels),
    args.panel_alpha,
)

draw_status(
    frame,
    frame_idx,
    pipeline_metrics.snapshot(),
    fps,
    target_fps,
    device_label,
    min(len(players[0].keypoints), args.window_size),
)

timings_ms["render"] = (time.perf_counter() - render_start) * 1000

output_start = time.perf_counter()
writer.write(frame)

quit_requested = False

```

```

if args.show:
    cv2.imshow("Tekken ST-GCN inference", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        quit_requested = True

timings_ms["output"] = (time.perf_counter() - output_start) * 1000
timings_ms["e2e"] = (time.perf_counter() - frame_start) * 1000
pipeline_metrics.record(timings_ms)

if frame_idx % 100 == 0:
    snapshot = pipeline_metrics.snapshot()

    rolling_text = (
        f"rolling {float(snapshot['e2e']):.2f} ms / "
        f"{float(snapshot['fps']):.2f} FPS"
        if bool(snapshot.get("ready", False))
        else "benchmark warm-up"
    )

    print(
        f"frame {frame_idx:5d} | "
        f"det {timings_ms['det']:.2f} ms | "
        f"pose {timings_ms['pose']:.2f} ms | "
        f"action {timings_ms['action']:.2f} ms | "
        f"E2E {timings_ms['e2e']:.2f} ms | "
        f"{rolling_text}"
    )

frame_idx += 1

if quit_requested:
    break

```

```

cap.release()

writer.release()

if args.show:
    cv2.destroyAllWindows()

report = write_metrics_report(
    metrics_path,
    args,
    pipeline_metrics,
    fps,
    target_fps,
    device_label,
)

print(f"Done. Wrote: {args.output}")
print(f"Metrics: {metrics_path}")
if "pipeline_fps" in report:
    e2e_stats = report["latency_ms"]["e2e"]
    print(
        f"End-to-end result ({report['measured_frames']} frames): "
        f"{e2e_stats['mean']:.2f} ms mean, "
        f"{e2e_stats['p95']:.2f} ms p95, "
        f"{report['pipeline_fps']:.2f} FPS, "
        f"{report['realtime_ratio']:.2f}x target"
    )

def main() -> None:
    args = parse_args()
    process_video(args)

```

```
if __name__ == "__main__":  
    main()
```


Lampiran 8 Benchmark untuk pose model (benchmark_yolopose_runtime.py):

```
#!/usr/bin/env python3

"""Benchmark YOLO-pose runtime on full frames vs tracked player ROIs.

This script is meant to answer one specific question:

does a larger YOLO-pose model or larger imgsz really cost more in this
pipeline?

It measures wall-clock time with device synchronization around each
prediction,

then writes a compact CSV so the result can be used in thesis notes.
"""

from __future__ import annotations

import argparse
import csv
import time
from pathlib import Path

import cv2
import numpy as np
import torch
from ultralytics import YOLO

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--video",
```

```

        default="practice_videos/Bryan_LR_Complete.mp4",
        help="Video to sample frames from.",
    )

    parser.add_argument(
        "--weights",
        nargs="+",
        default=["yolo11s-pose.pt", "yolo11m-pose.pt", "yolo11l-pose.pt",
"yolo11x-pose.pt"],
        help="YOLO-pose weight files to benchmark. Missing local files are
skipped.",
    )

    parser.add_argument(
        "--skip-yolo",
        action="store_true",
        help="Skip YOLO-pose benchmarks.",
    )

    parser.add_argument(
        "--include-movenet",
        action="store_true",
        help="Also benchmark MoveNet Lightning SinglePose and MoveNet
Lightning MultiPose from TFHub.",
    )

    parser.add_argument(
        "--movenet-models",
        nargs="+",
        choices=["singlepose_lightning", "multipose_lightning"],
        default=["singlepose_lightning", "multipose_lightning"],
        help="MoveNet variants to benchmark when --include-movenet is set.",
    )

    parser.add_argument(
        "--imgsz",

```

```

        nargs="+",
        type=int,
        default=[192, 384],
        help="Image sizes to test.",
    )
    parser.add_argument(
        "--device",
        default="auto",
        help="Device for Ultralytics/PyTorch, e.g. auto, mps, cpu, cuda:0.",
    )
    parser.add_argument(
        "--frames",
        type=int,
        default=120,
        help="Number of measured frames after warmup.",
    )
    parser.add_argument(
        "--warmup",
        type=int,
        default=20,
        help="Warmup frames per benchmark case.",
    )
    parser.add_argument(
        "--stride",
        type=int,
        default=8,
        help="Read every Nth frame from the video.",
    )
    parser.add_argument(

```

```

        "--start",
        type=int,
        default=0,
        help="Starting frame index.",
    )
    parser.add_argument(
        "--roi-tracks",
        nargs="+",
        default=["practice_videos/player1_track.npy",
"practice_videos/player2_track.npy"],
        help="Track .npy files with rows [id, x1, y1, x2, y2].",
    )
    parser.add_argument(
        "--padding",
        type=int,
        default=10,
        help="Padding in pixels around each tracked box before cropping.",
    )
    parser.add_argument("--conf", type=float, default=0.25, help="Pose confidence threshold.")
    parser.add_argument("--iou", type=float, default=0.35, help="Pose NMS IoU threshold.")
    parser.add_argument(
        "--output",
        default="benchmark_yolopose_runtime.csv",
        help="CSV output path.",
    )
    return parser.parse_args()

def resolve_device(requested_device: str) -> str:

```

```

    if requested_device == "auto":
        if hasattr(torch.backends, "mps") and
torch.backends.mps.is_available():

            return "mps"

        if torch.cuda.is_available():

            return "cuda:0"

        return "cpu"

    if requested_device == "mps":
        if hasattr(torch.backends, "mps") and
torch.backends.mps.is_available():

            return "mps"

        print("WARNING: MPS was requested but is not available. Falling back
to CPU.")

        return "cpu"

    if requested_device.startswith("cuda") and not
torch.cuda.is_available():

        print("WARNING: CUDA was requested but is not available. Falling
back to CPU.")

        return "cpu"

    return requested_device

def sync_device(device: str) -> None:

    if device.startswith("cuda") and torch.cuda.is_available():

        torch.cuda.synchronize()

    elif device == "mps" and hasattr(torch, "mps") and hasattr(torch.mps,
"synchronize"):

        torch.mps.synchronize()

```

```

def collect_frames(video_path: Path, total_frames: int, stride: int, start:
int) -> list[tuple[int, np.ndarray]]:

    cap = cv2.VideoCapture(str(video_path))

    if not cap.isOpened():

        raise FileNotFoundError(f"Could not open video: {video_path}")

    cap.set(cv2.CAP_PROP_POS_FRAMES, start)

    frames: list[tuple[int, np.ndarray]] = []

    while len(frames) < total_frames:

        ok, frame = cap.read()

        if not ok:

            break

        frame_idx = int(cap.get(cv2.CAP_PROP_POS_FRAMES)) - 1

        if (frame_idx - start) % stride == 0:

            frames.append((frame_idx, frame))

    cap.release()

    if not frames:

        raise RuntimeError("No frames collected. Check --video, --start, and
--stride.")

    return frames

def load_tracks(track_paths: list[str]) -> list[np.ndarray]:

    tracks = []

    for track_path in track_paths:

```

```

        path = Path(track_path)

        if not path.exists():

            print(f"Skipping missing track file: {path}")

            continue

        tracks.append(np.load(path))

    return tracks

def crop_track_roi(frame: np.ndarray, track_row: np.ndarray, padding: int)
-> np.ndarray | None:

    if track_row.shape[0] >= 5:

        _, x1, y1, x2, y2 = track_row[:5]

    else:

        x1, y1, x2, y2 = track_row[:4]

    if not np.all(np.isfinite([x1, y1, x2, y2])):

        return None

    height, width = frame.shape[:2]

    left = max(0, int(np.floor(x1 - padding)))
    top = max(0, int(np.floor(y1 - padding)))
    right = min(width, int(np.ceil(x2 + padding)))
    bottom = min(height, int(np.ceil(y2 + padding)))

    if right <= left or bottom <= top:

        return None

    return frame[top:bottom, left:right].copy()

```

```

def build_roi_pairs(
    frames: list[tuple[int, np.ndarray]],
    tracks: list[np.ndarray],
    padding: int,
) -> list[tuple[int, list[np.ndarray]]]:
    roi_pairs = []
    for frame_idx, frame in frames:
        rois = []
        valid = True
        for track in tracks:
            if frame_idx >= len(track):
                valid = False
                break
            roi = crop_track_roi(frame, track[frame_idx], padding)
            if roi is None:
                valid = False
                break
            rois.append(roi)

        if valid and rois:
            roi_pairs.append((frame_idx, rois))

    return roi_pairs

def summarize(values: list[float]) -> tuple[float, float, float, float]:
    arr = np.array(values, dtype=np.float64)
    return (
        float(np.mean(arr)),

```

```

        float(np.median(arr)),

        float(np.percentile(arr, 95)),

        float(np.std(arr)),

    )

def model_param_count(model: YOLO) -> int:

    return sum(param.numel() for param in model.model.parameters())

def format_csv_value(value: float | int | str) -> float | int | str:

    if isinstance(value, float):

        if np.isfinite(value):

            return f"{value:.2f}"

        return ""

    return value

def run_full_frame_benchmark(

    model: YOLO,

    frames: list[tuple[int, np.ndarray]],

    imgsz: int,

    device: str,

    warmup: int,

    conf: float,

    iou: float,

) -> dict[str, float | int | str]:

    wall_times = []

    ultra_infer_times = []

```

```

for i, (_, frame) in enumerate(frames):

    sync_device(device)

    start = time.perf_counter()

    result = model(frame, imgsz=imgsz, conf=conf, iou=iou,
verbose=False)[0]

    sync_device(device)

    elapsed_ms = (time.perf_counter() - start) * 1000

    if i >= warmup:

        wall_times.append(elapsed_ms)

        ultra_infer_times.append(float(result.speed.get("inference",
np.nan)))

mean_ms, median_ms, p95_ms, std_ms = summarize(wall_times)
ultra_mean, _, _, _ = summarize(ultra_infer_times)

return {

    "scope": "full_frame",

    "calls_per_frame": 1,

    "measured_frames": len(wall_times),

    "imgsz": imgsz,

    "wall_mean_ms": mean_ms,

    "wall_median_ms": median_ms,

    "wall_p95_ms": p95_ms,

    "wall_std_ms": std_ms,

    "ultralytics_infer_mean_ms": ultra_mean,

}

def run_roi_pair_benchmark(

```

```

model: YOLO,
roi_pairs: list[tuple[int, list[np.ndarray]]],
imgsz: int,
device: str,
warmup: int,
conf: float,
iou: float,
) -> dict[str, float | int | str]:
    wall_times = []
    ultra_infer_times = []

    for i, (_, rois) in enumerate(roi_pairs):
        sync_device(device)

        start = time.perf_counter()

        frame_infer_ms = 0.0

        for roi in rois:
            result = model(roi, imgsz=imgsz, conf=conf, iou=iou,
verbose=False)[0]

            frame_infer_ms += float(result.speed.get("inference", np.nan))

        sync_device(device)

        elapsed_ms = (time.perf_counter() - start) * 1000

        if i >= warmup:
            wall_times.append(elapsed_ms)
            ultra_infer_times.append(frame_infer_ms)

    mean_ms, median_ms, p95_ms, std_ms = summarize(wall_times)
    ultra_mean, _, _, _ = summarize(ultra_infer_times)

    return {
        "scope": "roi_pair_total",

```

```

        "calls_per_frame": len(roi_pairs[0][1]) if roi_pairs else 0,

        "measured_frames": len(wall_times),

        "imgsz": imgsz,

        "wall_mean_ms": mean_ms,

        "wall_median_ms": median_ms,

        "wall_p95_ms": p95_ms,

        "wall_std_ms": std_ms,

        "ultralytics_infer_mean_ms": ultra_mean,

    }

class MoveNetRunner:

    def __init__(self, name: str, input_size: int, model_url: str) -> None:

        import tensorflow as tf

        import tensorflow_hub as hub

        self.name = name

        self.input_size = input_size

        self.tf = tf

        self.model = hub.load(model_url).signatures["serving_default"]

    def __call__(self, image_bgr: np.ndarray) -> None:

        frame_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

        image = self.tf.convert_to_tensor(frame_rgb)

        image = self.tf.expand_dims(image, axis=0)

        image = self.tf.image.resize_with_pad(image, self.input_size,
self.input_size)

        image = self.tf.cast(image, dtype=self.tf.int32)

        outputs = self.model(image)

```

```

        # Force TensorFlow execution before stopping the timer.

        for value in outputs.values():
            _ = value.numpy()

def load_movenet_runner(model_name: str) -> MoveNetRunner:
    model_specs = {
        "singlepose_lightning": (
            192,
            "https://tfhub.dev/google/movenet/singlepose/lightning/4",
        ),
        "multipose_lightning": (
            256,
            "https://tfhub.dev/google/movenet/multipose/lightning/1",
        ),
    }
    input_size, model_url = model_specs[model_name]
    return MoveNetRunner(model_name, input_size, model_url)

def run_movenet_full_frame_benchmark(
    runner: MoveNetRunner,
    frames: list[tuple[int, np.ndarray]],
    warmup: int,
) -> dict[str, float | int | str]:
    wall_times = []

    for i, (_, frame) in enumerate(frames):
        start = time.perf_counter()

```

```

runner(frame)

elapsed_ms = (time.perf_counter() - start) * 1000

if i >= warmup:
    wall_times.append(elapsed_ms)

mean_ms, median_ms, p95_ms, std_ms = summarize(wall_times)
return {
    "scope": "full_frame",
    "calls_per_frame": 1,
    "measured_frames": len(wall_times),
    "imgsz": runner.input_size,
    "wall_mean_ms": mean_ms,
    "wall_median_ms": median_ms,
    "wall_p95_ms": p95_ms,
    "wall_std_ms": std_ms,
    "ultralytics_infer_mean_ms": np.nan,
}

def run_movenet_roi_pair_benchmark(
    runner: MoveNetRunner,
    roi_pairs: list[tuple[int, list[np.ndarray]]],
    warmup: int,
) -> dict[str, float | int | str]:
    wall_times = []

    for i, (_, rois) in enumerate(roi_pairs):
        start = time.perf_counter()

```

```

        for roi in rois:
            runner(roi)

        elapsed_ms = (time.perf_counter() - start) * 1000

        if i >= warmup:
            wall_times.append(elapsed_ms)

    mean_ms, median_ms, p95_ms, std_ms = summarize(wall_times)
    return {
        "scope": "roi_pair_total",
        "calls_per_frame": len(roi_pairs[0][1]) if roi_pairs else 0,
        "measured_frames": len(wall_times),
        "imgsz": runner.input_size,
        "wall_mean_ms": mean_ms,
        "wall_median_ms": median_ms,
        "wall_p95_ms": p95_ms,
        "wall_std_ms": std_ms,
        "ultralytics_infer_mean_ms": np.nan,
    }

def main() -> None:
    args = parse_args()
    device = resolve_device(args.device)
    total_frames = args.frames + args.warmup
    video_path = Path(args.video)

    frames = collect_frames(video_path, total_frames, args.stride,
                             args.start)

    tracks = load_tracks(args.roi_tracks)

```

```

    roi_pairs = build_roi_pairs(frames, tracks, args.padding) if tracks else
[]

    print(f"Video: {video_path}")

    print(f"Collected frames: {len(frames)}")

    print(f"ROI frame pairs: {len(roi_pairs)} from {len(tracks)} track
file(s)")

    print(f"Requested device: {args.device}")

    print(f"Actual device: {device}")

    rows: list[dict[str, float | int | str]] = []

    if not args.skip_yolo:
        for weight_name in args.weights:
            weight_path = Path(weight_name)

            if not weight_path.exists():
                print(f"Skipping missing weights: {weight_path}")

                continue

            print(f"\nLoading {weight_path} ...")

            model = YOLO(str(weight_path))

            model.to(device)

            params = model_param_count(model)

            print(f"Parameters: {params:,}")

            for imgsz in args.imgsz:
                print(f"Benchmarking {weight_path.name}, imgsz={imgsz},
full frame")

                full_row = run_full_frame_benchmark(
                    model, frames, imgsz, device, args.warmup, args.conf,
args.iou

                )

```

```

        full_row.update(
            {"backend": "ultralytics", "weights": weight_path.name,
"params": params, "device": device}
        )
        rows.append(full_row)

    if roi_pairs:
        print(f"Benchmarking {weight_path.name}, imgsz={imgsz},
ROI pair")

        roi_row = run_roi_pair_benchmark(
            model, roi_pairs, imgsz, device, args.warmup,
args.conf, args.iou
        )
        roi_row.update(
            {"backend": "ultralytics", "weights":
weight_path.name, "params": params, "device": device}
        )
        rows.append(roi_row)

    if args.include_movenet:
        for model_name in args.movenet_models:
            print(f"\nLoading MoveNet {model_name} ...")
            try:
                runner = load_movenet_runner(model_name)
            except Exception as exc:
                print(f"Skipping MoveNet {model_name}: {exc}")
                continue

            print(f"Benchmarking MoveNet {model_name},
input={runner.input_size}, full frame")

            full_row = run_movenet_full_frame_benchmark(runner, frames,
args.warmup)

```

```

        full_row.update(
            {"backend": "tensorflow_hub", "weights":
f"movenet_{model_name}", "params": "", "device": "tensorflow"}
        )

        rows.append(full_row)

    if roi_pairs:
        print(f"Benchmarking MoveNet {model_name},
input={runner.input_size}, ROI pair")

        roi_row = run_movenet_roi_pair_benchmark(runner, roi_pairs,
args.warmup)

        roi_row.update(
            {
                "backend": "tensorflow_hub",
                "weights": f"movenet_{model_name}",
                "params": "",
                "device": "tensorflow",
            }
        )

        rows.append(roi_row)

if not rows:
    raise RuntimeError("No benchmark rows were produced. Check weight
paths.")

output_path = Path(args.output)

fieldnames = [
    "backend",
    "weights",
    "params",
    "device",

```

```

        "scope",
        "calls_per_frame",
        "measured_frames",
        "imgsz",
        "wall_mean_ms",
        "wall_median_ms",
        "wall_p95_ms",
        "wall_std_ms",
        "ultralytics_infer_mean_ms",
    ]

    with output_path.open("w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        for row in rows:
            writer.writerow({field: format_csv_value(row.get(field, ""))
for field in fieldnames})

    print(f"\nSaved: {output_path}")
    print("\nSummary:")
    for row in rows:
        print(
            f"{row['weights']:28s}                               imgsz={row['imgsz']:>3}"
{row['scope']:15s} "
            f"wall median={row['wall_median_ms']:6.2f} ms "
            f"p95={row['wall_p95_ms']:6.2f} ms"
        )

if __name__ == "__main__":

```

```
main()
```

Lampiran 9 Requirements.txt :

```
# Python 3.10.11

# Core

numpy>=1.23,<2.0

matplotlib>=3.6,<4.0

# Computer Vision

opencv-python>=4.8,<5.0

# TensorFlow + TF Hub (MoveNet)

tensorflow==2.14.0

tensorflow-metal==1.1.0

tensorflow-hub==0.16.1

# YOLO + ByteTrack tracking support

ultralytics>=8.0.0,<9.0.0

lapx>=0.5.5

# MMEngine (used in cv utility code.ipynb)

mmengine>=0.10.0,<1.0.0

# GUI / Imaging (used by anotator_5.py)

pillow>=9.5,<11.0

# Utilities commonly needed by ultralytics/opencv pipelines

pyyaml>=6.0,<7.0

tqdm>=4.65,<5.0

requests>=2.31,<3.0
```

```
# PyTorch
torch
torchvision
torchaudio

# Boxmot
Boxmot
```

Lampiran 10 Semua code penting di luar pipeline utama (cv_utility_code.ipynb) :

```
# Libraries Import

import os, cv2, json, math, pickle, random

import numpy as np

import matplotlib.pyplot as plt

from collections import defaultdict

import tensorflow as tf

import tensorflow_hub as hub

from ultralytics import YOLO

from boxmot.trackers.ocsort.ocsort import OcSort

import torch


#Local Base Path

base_path = ""


# === Dataset Paths ===

data_path = os.path.join(base_path, "practice_videos")

video_path = os.path.join(data_path, "Bryan_LR_Complete.mp4")

annotation_path = os.path.join(data_path, "Bryan_LR_Complete.json")


labels_path = os.path.join(data_path, "move_labels.json") # move class labels

skeleton_dataset = os.path.join(data_path, "skeleton_dataset.pkl") # STGCN++ dataset

output_dir = os.path.join(data_path, "frames")

kp_dir = os.path.join(data_path)


# === Load movenet and YOLO models ===

movenet = hub.load("https://tfhub.dev/google/movenet/singlepose/lightning/4").signatures['serving_default']
```

```

yolo = YOLO("runs/detect/twt/weights/best.pt")
yolo.to("mps")

# Adjust Annotation
import json
import os

def adjust_annotations(input_file, output_file=None, frame_adjustment=-1):
    """
    Adjust all frame numbers and recalculate timing data

    Args:
        input_file: Path to your current annotations JSON
        output_file: Path for adjusted annotations (if None, creates backup
and overwrites)
        frame_adjustment: Number of frames to adjust (negative to reduce,
positive to increase)
    """

    # Load existing annotations
    with open(input_file, 'r') as f:
        annotations = json.load(f)

    # Get FPS from user or estimate (you might want to hardcode this)
    fps = float(input("Enter video FPS (usually 60 for Tekken): ") or "60")

    # Create backup if overwriting
    if output_file is None:
        backup_file = input_file.replace('.json', '_backup.json')
        print(f"Creating backup: {backup_file}")

```

```

with open(backup_file, 'w') as f:
    json.dump(annotations, f, indent=2)

output_file = input_file

adjusted_annotations = {}

for seq_id, seq_data in annotations.items():
    # Adjust frame numbers
    new_start_frame = max(0, seq_data["start_frame"] +
frame_adjustment)

    new_end_frame = max(new_start_frame, seq_data["end_frame"] +
frame_adjustment)

    # Recalculate timing data
    new_start_time = new_start_frame / fps
    new_end_time = new_end_frame / fps
    new_duration_frames = new_end_frame - new_start_frame

    # Create adjusted sequence
    adjusted_seq = seq_data.copy()
    adjusted_seq.update({
        "start_frame": new_start_frame,
        "end_frame": new_end_frame,
        "start_time": new_start_time,
        "end_time": new_end_time,
        "duration_frames": new_duration_frames
    })

    adjusted_annotations[seq_id] = adjusted_seq

```

```

    # Print adjustment info

    old_duration = seq_data["duration_frames"]

    print(f"{seq_id}: {old_duration} → {new_duration_frames} frames "
          f"({seq_data['start_frame']}-{seq_data['end_frame']} → {new_start_frame}-{new_end_frame})")

# Save adjusted annotations

with open(output_file, 'w') as f:
    json.dump(adjusted_annotations, f, indent=2)

print(f"\nAdjustments saved to: {output_file}")

print(f"Total sequences adjusted: {len(adjusted_annotations)}")

# Usage examples:

if __name__ == "__main__":
    # Adjust your current annotations

    input_file = "match_videos\Knee(Bryan) vs Double(Law) TWT 2024.json" #
    Replace with your file path

    print("Adjustment options:")
    print("1. Reduce end frames by 1 (end earlier)")
    print("2. Reduce both start and end by 1 (shift entire sequence)")
    print("3. Custom adjustment")

    # choice = input("Choose option (1/2/3): ")
    choice = "2"

    if choice == "1":
        # Only adjust end frames

        with open(input_file, 'r') as f:

```

```

        annotations = json.load(f)

        fps = float(input("Enter video FPS: ") or "60")

        for seq_id, seq_data in annotations.items():
            seq_data["end_frame"] = max(seq_data["start_frame"],
seq_data["end_frame"] - 1)

            seq_data["end_time"] = seq_data["end_frame"] / fps

            seq_data["duration_frames"] = seq_data["end_frame"] -
seq_data["start_frame"]

        backup_file = input_file.replace('.json', '_backup.json')
        with open(backup_file, 'w') as f:
            json.dump(annotations, f, indent=2)

        with open(input_file, 'w') as f:
            json.dump(annotations, f, indent=2)

        print(f"End frames reduced by 1. Backup saved as {backup_file}")

    elif choice == "2":
        adjust_annotations(input_file, frame_adjustment=-1)

    elif choice == "3":
        adjustment = int(input("Enter frame adjustment (negative to reduce):
"))
        adjust_annotations(input_file, frame_adjustment=adjustment)

# Recompute Annotation
import json
import os

```

```

import re

def adjust_annotations(input_file, output_file=None, frame_adjustment=0,
reindex_only=False,
                        fps=60.0, id_prefix="seq_", preserve_prefix=True,
pad_width=3, start_index=1):
    """
    Adjust frames and/or reindex sequence IDs.

    - reindex_only=True: keep frames as-is, renumber keys to contiguous
sequence numbers.

    If preserve_prefix=True the function keeps the original prefix (e.g.
"p1_sequence_")
    and only replaces the numeric suffix.

    - pad_width controls zero-padding of the new numeric suffix.
    - start_index allows starting numbering at a given integer.
    """
    with open(input_file, 'r', encoding='utf-8') as f:
        annotations = json.load(f)

    if output_file is None:
        backup_file = input_file.replace('.json', '_backup.json')
        print(f"Creating backup: {backup_file}")
        with open(backup_file, 'w', encoding='utf-8') as f:
            json.dump(annotations, f, indent=2, ensure_ascii=False)
        output_file = input_file

    if reindex_only:
        items = sorted(annotations.items(), key=lambda kv:
int(kv[1].get("start_frame", 0)))
        adjusted_annotations = {}
        mapping = {}

```

```

        for idx, (old_id, seq_data) in enumerate(items, start=start_index):
            # determine prefix to preserve (e.g. "p1_sequence_"); fallback
            to id_prefix

            if preserve_prefix:
                m = re.match(r'^(.*?_sequence_)', old_id)

                if m:
                    prefix = m.group(1)
                else:
                    parts = old_id.rsplit('_', 1)
                    prefix = (parts[0] + '_') if len(parts) > 1 else
id_prefix

            else:
                prefix = id_prefix

            # create numeric suffix with optional zero-padding

            if pad_width and int(pad_width) > 0:
                new_num = f"{idx:0{pad_width}d}"
            else:
                new_num = str(idx)

            new_id = f"{prefix}{new_num}"

            new_seq = seq_data.copy()
            start_f = int(new_seq.get("start_frame", 0))
            end_f = int(new_seq.get("end_frame", 0))

            if end_f < start_f:
                end_f = start_f

            new_seq["start_frame"] = start_f

```

```

        new_seq["end_frame"] = end_f

        new_seq["duration_frames"] = end_f - start_f

        new_seq["start_time"] = start_f / float(fps)

        new_seq["end_time"] = end_f / float(fps)

        adjusted_annotations[new_id] = new_seq

        mapping[old_id] = new_id

    with open(output_file, 'w', encoding='utf-8') as f:
        json.dump(adjusted_annotations, f, indent=2,
ensure_ascii=False)

    print(f"Reindexed {len(mapping)} sequences. Mapping (old -> new):")
    for old, new in mapping.items():
        print(f"    {old} -> {new}")

    print(f"Saved reindexed annotations to: {output_file}")

    return

# fallback: perform frame shift + recompute (unchanged)
adjusted_annotations = {}

for seq_id, seq_data in annotations.items():
    new_start_frame = max(0, int(seq_data.get("start_frame", 0)) +
int(frame_adjustment))

    new_end_frame = max(new_start_frame, int(seq_data.get("end_frame",
0)) + int(frame_adjustment))

    new_start_time = new_start_frame / float(fps)

    new_end_time = new_end_frame / float(fps)

    new_duration_frames = new_end_frame - new_start_frame

    adjusted_seq = seq_data.copy()

```

```

        adjusted_seq.update({
            "start_frame": new_start_frame,
            "end_frame": new_end_frame,
            "start_time": new_start_time,
            "end_time": new_end_time,
            "duration_frames": new_duration_frames
        })

    adjusted_annotations[seq_id] = adjusted_seq

    old_duration = seq_data.get("duration_frames", new_duration_frames)
    print(f"{seq_id}: {old_duration} → {new_duration_frames} frames "
          f"({seq_data.get('start_frame')}-{seq_data.get('end_frame')}"
          → {new_start_frame}-{new_end_frame})")

    with open(output_file, 'w', encoding='utf-8') as f:
        json.dump(adjusted_annotations, f, indent=2, ensure_ascii=False)

    print(f"\nAdjustments saved to: {output_file}")
    print(f"Total sequences adjusted: {len(adjusted_annotations)}")

adjust_annotations(
    input_file="match_videos\\Knee(Bryan) vs Double(Law) TWT 2024.json",
    output_file="match_videos\\Knee_reindexed.json",
    reindex_only=True,
    fps=60.0,
    preserve_prefix=True,
    pad_width=0,
    start_index=1
)

```

```

# Export frame and kp to vid

import subprocess, os, json, tempfile, pathlib

import cv2

import numpy as np

slow_factor = 4.0

ffmpeg = "ffmpeg"

font = "/Library/Fonts/Arial.ttf"

# If these exist in kp_dir, they will be drawn automatically.
kp_files = {
    "player1": os.path.join(kp_dir, "player1_kp.npy"),
    "player2": os.path.join(kp_dir, "player2_kp.npy"),
}

COCO_EDGES = [
    (0, 1), (0, 2), (1, 3), (2, 4),
    (5, 7), (7, 9), (6, 8), (8, 10),
    (5, 6), (5, 11), (6, 12), (11, 12),
    (11, 13), (13, 15), (12, 14), (14, 16),
]

def _escape_drawtext(s: str) -> str:
    s = s.replace("\\", "\\\\")
    s = s.replace(":", "\\:")
    s = s.replace("'", "\\'")
    s = s.replace("%", "\\%")
    s = s.replace("\n", " ")

```

```

s = s.replace("\r", " ")

return s

def _kp_point(kp, width, height, conf_thr=0.15):
    y, x, c = kp
    if not np.isfinite(x) or not np.isfinite(y) or not np.isfinite(c):
        return None
    if c < conf_thr:
        return None

    # Your saved KP format is normalized [y, x, confidence].
    if 0 <= x <= 1.5 and 0 <= y <= 1.5:
        x *= width
        y *= height

    return int(x), int(y)

def draw_kp(frame, kp_frame, color, conf_thr=0.15, draw_skeleton=True):
    height, width = frame.shape[:2]
    points = [_kp_point(kp, width, height, conf_thr) for kp in kp_frame]

    if draw_skeleton:
        for a, b in COCO_EDGES:
            if points[a] is not None and points[b] is not None:
                cv2.line(frame, points[a], points[b], color, 2,
cv2.LINE_AA)

    for pt in points:
        if pt is not None:

```

```

        cv2.circle(frame, pt, 3, color, thickness=2,
lineType=cv2.LINE_AA)

def _norm_player_name(player):
    player = str(player).lower().replace(" ", "").replace("_", "")
    if player in {"player1", "p1"}:
        return "player1"
    if player in {"player2", "p2"}:
        return "player2"
    return player

def make_source_with_sequence_kp(video_path, kp_files, annotations,
tmpdir):
    loaded = {}

    for player, path in kp_files.items():
        if os.path.exists(path):
            loaded[player] = np.load(path)

    if not loaded:
        print("No KP files found, using original video.")
        return video_path

    frame_to_players = {}

    for seq_id, info in annotations.items():
        player = _norm_player_name(info.get("player", ""))

        if player not in loaded:
            print(f"Skipping {seq_id}: unknown/missing player
'{info.get('player')}')")

```

```

        continue

    start = int(info["start_frame"])
    end = int(info["end_frame"])

    for frame_idx in range(start, end + 1):
        frame_to_players.setdefault(frame_idx, set()).add(player)

cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    raise ValueError(f"Could not open video: {video_path}")

fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

out_kp_video = str(tmpdir / "source_with_sequence_kp.mp4")
writer = cv2.VideoWriter(
    out_kp_video,
    cv2.VideoWriter_fourcc(*"mp4v"),
    fps,
    (width, height),
)

colors = {
    "player1": (0, 255, 0),  # green
    "player2": (0, 0, 255),  # red
}

```

```

frame_idx = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    players_to_draw = frame_to_players.get(frame_idx, set())

    for player in players_to_draw:
        kp_arr = loaded[player]
        if frame_idx < len(kp_arr):
            draw_kp(frame, kp_arr[frame_idx], colors[player])

    writer.write(frame)
    frame_idx += 1

cap.release()
writer.release()

print("Temporary sequence-specific KP video:", out_kp_video)
return out_kp_video

with open(annotation_path, "r", encoding="utf-8") as f:
    annotations = json.load(f)

items = sorted(annotations.items(), key=lambda kv:
int(kv[1].get("start_frame", 0)))
if not items:
    raise ValueError("No annotations found.")

```

```

os.makedirs(output_dir, exist_ok=True)

tmpdir = pathlib.Path(tempfile.mkdtemp())

# This becomes the ffmpeg input. If KP exists, it uses the KP-rendered temp
video.

source_video = make_source_with_sequence_kp(video_path, kp_files,
annotations, tmpdir)

parts = []
concat_inputs = []
valid = 0

for _, (seq_id, info) in enumerate(items):
    start = int(info["start_frame"])
    end = int(info["end_frame"])
    if end < start:
        continue

    label = str(info.get("move", info.get("label", "")))
    text = _escape_drawtext(f"{seq_id} {label}".strip())

    parts.append(
        f"[0:v]"
        f"select='between(n\\, {start}\\, {end})',"
        f"setpts=PTS-STARTPTS,"
        f"setpts={slow_factor}*PTS,"
        f"drawtext=fontfile='{font}':text='{text}':x=10:y=20:fontsize=24:"
        f"fontcolor=white:box=1:boxcolor=0x00000099:boxborderw=6"

```

```

        f"[v{valid}]"
    )
    concat_inputs.append(f"[v{valid}]")
    valid += 1

if valid == 0:
    raise ValueError("No valid (start_frame,end_frame) ranges found.")

filter_complex = ";".join(parts) + ";" + "".join(concat_inputs) +
f"concat=n={valid}:v=1:a=0[outv]"

out_final = os.path.join(output_dir,
f"all_sequences_slow_{int(slow_factor)}x_ffmpeg_overlay_kp.mp4")

fg_path = tmpdir / "filtergraph.txt"
fg_path.write_text(filter_complex, encoding="utf-8")

cmd = [
    ffmpeg, "-y",
    "-i", source_video,
    "-filter_complex_script", str(fg_path),
    "-map", "[outv]",
    "-an",
    "-c:v", "libx264",
    "-preset", "veryfast",
    "-crf", "20",
    out_final
]

print(f"Running ffmpeg with overlay + KP for {valid} segments...")

```

```

print("Filtergraph file:", fg_path)

try:
    subprocess.run(cmd, check=True, capture_output=True, text=True)
except subprocess.CalledProcessError as e:
    print("ffmpeg failed")
    print("stdout:", e.stdout)
    print("stderr:", e.stderr)
    raise

print("Saved:", out_final)

# Review KP
# Load the saved data
p1_path = os.path.join(kp_dir, "player1_kp.npy")
p1_data = np.load(p1_path)

print(f"--- Data Diagnostics ---")
print(f"Shape: {p1_data.shape} (Expected: [Frames, 17, 3])")

# Check Frame 0 specifically (since that's where your image shows the issue)
frame0 = p1_data[0]
print(f"\nFrame 0 Sample (Nose [y, x, conf]): {frame0[0]}")

# Check for NaNs
print(f"Has NaNs: {np.isnan(p1_data).any()}")

# Check Value Range (To determine if it's Normalized 0-1 or Pixels)
y_coords = p1_data[:, :, 0]

```

```

x_coords = p1_data[:, :, 1]

print(f"\nY          Range:          Min={np.nanmin(y_coords):.4f},
Max={np.nanmax(y_coords):.4f}")

print(f"\nX          Range:          Min={np.nanmin(x_coords):.4f},
Max={np.nanmax(x_coords):.4f}")

if np.nanmax(y_coords) > 1.5 or np.nanmax(x_coords) > 1.5:

    print("\n⚠️ DIAGNOSIS: Data is likely stored in PIXELS (Denormalized).")

    print("ACTION: Remove the 'x * width' and 'y * height' multiplication
in the viz script.")
else:

    print("\n✅ DIAGNOSIS: Data is likely NORMALIZED (0.0 - 1.0).")

    print("ACTION: Ensure width/height in viz script match the video source
exactly.")

# Draw kp on vid

import os
import cv2
import numpy as np

# MoveNet 17-keypoint skeleton edges
# (start_idx, end_idx)
MOVENET_EDGES = [

    (0, 1), (0, 2), (1, 3), (2, 4),          # head
    (5, 6),                                # shoulders
    (5, 7), (7, 9),                          # left arm
    (6, 8), (8, 10),                         # right arm <-- add these
    (5, 11), (6, 12), (11, 12),             # torso
    (11, 13), (13, 15),                     # left leg
    (12, 14), (14, 16)                      # right leg

```

```

]

def _to_pixel_xy(kp, width, height, is_normalized):

    y, x, conf = kp

    if np.isnan(y) or np.isnan(x):

        return None

    if is_normalized:

        px = int(x * width)

        py = int(y * height)

    else:

        px = int(x)

        py = int(y)

    # clamp to frame

    px = max(0, min(width - 1, px))

    py = max(0, min(height - 1, py))

    return (px, py, conf)

def _draw_id_box(frame, id_box, color, label):

    if id_box is None or np.any(np.isnan(id_box)):

        return

    track_id, x1, y1, x2, y2 = id_box

    h, w = frame.shape[:2]

    x1 = max(0, min(w - 1, int(round(x1))))

    y1 = max(0, min(h - 1, int(round(y1))))

    x2 = max(0, min(w - 1, int(round(x2))))

    y2 = max(0, min(h - 1, int(round(y2))))

    if x2 <= x1 or y2 <= y1:

        return

```

```

cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2, cv2.LINE_AA)

text = f"{label} ID:{int(track_id)}"

y_text = y1 - 8 if y1 > 20 else y1 + 20

cv2.putText(frame, text, (x1, y_text), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
color, 2, cv2.LINE_AA)

def visualize_robust(video_input_path, output_path, p1_kp_path,
p2_kp_path, p1_track_path=None, p2_track_path=None, conf_th=0.2):

    p1_kp = np.load(p1_kp_path)

    p2_kp = np.load(p2_kp_path)

    p1_track = np.load(p1_track_path) if p1_track_path and
os.path.exists(p1_track_path) else None

    p2_track = np.load(p2_track_path) if p2_track_path and
os.path.exists(p2_track_path) else None

    cap = cv2.VideoCapture(video_input_path)

    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))

    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    fps = cap.get(cv2.CAP_PROP_FPS) or 30.0

    print(f"Video Resolution: {width}x{height}")

    is_normalized = np.nanmax(p1_kp[:, :, 0]) <= 1.5

    print(f>Data Detected as: {'NORMALIZED (0-1)' if is_normalized else
'PIXELS'}")

    out = cv2.VideoWriter(output_path, cv2.VideoWriter_fourcc(*'mp4v'),
fps, (width, height))

    n_frames = min(len(p1_kp), len(p2_kp))

    if p1_track is not None:

```

```

        n_frames = min(n_frames, len(p1_track))

    if p2_track is not None:
        n_frames = min(n_frames, len(p2_track))

    if p1_track is None:
        print("player1_track.npy not found: P1 box/ID overlay disabled.")
    if p2_track is None:
        print("player2_track.npy not found: P2 box/ID overlay disabled.")

    frame_idx = 0

    while cap.isOpened():
        ret, frame = cap.read()

        if not ret or frame_idx >= n_frames:
            break

        for kp_array, color in [(p1_kp[frame_idx], (0, 255, 0)),
                                (p2_kp[frame_idx], (255, 0, 0))]:
            pts = [_to_pixel_xy(kp, width, height, is_normalized) for kp in
                    kp_array]

            # Draw limbs (lines)
            for i, j in MOVENET_EDGES:
                pi = pts[i]
                pj = pts[j]

                if pi is None or pj is None:
                    continue

                if pi[2] < conf_th or pj[2] < conf_th:
                    continue

                cv2.line(frame, (pi[0], pi[1]), (pj[0], pj[1]), color, 2,
cv2.LINE_AA)

```

```

        # Draw joints (circles)

        for p in pts:

            if p is None:

                continue

            if p[2] < conf_th:

                continue

            cv2.circle(frame, (p[0], p[1]), 4, color, -1, cv2.LINE_AA)

        if p1_track is not None:

            _draw_id_box(frame, p1_track[frame_idx], (0, 255, 0), "P1")

        if p2_track is not None:

            _draw_id_box(frame, p2_track[frame_idx], (255, 0, 0), "P2")

        out.write(frame)

        frame_idx += 1

    cap.release()

    out.release()

    print("Done.")

# --- Execution ---

# Update these paths to match your directory structure
p1_path = os.path.join(kp_dir, "player1_kp.npy")
p2_path = os.path.join(kp_dir, "player2_kp.npy")
p1_track_path = os.path.join(kp_dir, "player1_track.npy")
p2_track_path = os.path.join(kp_dir, "player2_track.npy")
output_vid = os.path.join(kp_dir, "kp_visualization.mp4")

# Run this

```

```

visualize_robust(
    video_path,
    output_vid,
    p1_path,
    p2_path,
    p1_track_path=p1_track_path,
    p2_track_path=p2_track_path,
)

# Create Move Labels
move_labels = {}

i = 0

# JSON Annotation
with open(annotation_path) as f:
    annotations = json.load(f)

for sequence_id, data in annotations.items():
    move_class = data['character'] + " " + data['move']

    if move_class in move_labels:
        print("label exist")

    else:
        move_labels[move_class] = i
        i += 1

print(move_labels)

with open(labels_path, "w") as f:
    json.dump(move_labels, f, indent=2)

```

Lampiran 11 Kuesioner google form untuk survey:

Section 1 of 4

Understanding the Learning & Spectating Experience in Tekken

B I U ↺ ↻

Hi everyone! I'm a university student working on my bachelor thesis about improving the experience for new Tekken players and spectators. This survey is designed to understand the challenges people face when trying to learn the game or follow high-level matches. Your feedback would be incredibly valuable. This survey is anonymous and should only take about 5-7 minutes. Thank you for your help!

How would you primarily describe yourself in the Tekken community?

- ☐ Mainly a Player
- ☐ Mainly a Viewer/Spectator
- ☐ Both a Player and a Viewer
- ☐ New and just getting interested

How long have you been playing or following Tekken (or other traditional fighting games)?

- ☐ Less than 6 months
- ☐ 6 months to 2 years
- ☐ 2 to 5 years
- ☐ More than 5 years

How often do you watch high-level Tekken gameplay (tournaments, pro player streams, etc.)? *

- ☐ Rarely (a few times a year)
- ☐ Occasionally (a few times a month)
- ☐ Frequently (a few times a week)
- ☐ Daily

What is your current Tekken 8 rank (or closest equivalent)? *

- ☐ Beginner – Brown, Silver, Turquoise, Green
- ☐ Intermediate – Yellow, Orange (Warrior → Eliminator)
- ☐ Advanced – Red, Ruler tier (Garyu → Battle Ruler)
- ☐ Expert – Blue, Purple/Gold (Fujin → Tekken God Supreme)
- ☐ Master/Top-tier – God of Destruction ranks
- ☐ I don't play ranked

After section 1 Continue to next section

Section 2 of 4

The Learning and Viewing Experience

Description (optional)

When I first started watching high-level Tekken, I found it difficult to understand what was happening on screen. *

- | | | | | | | |
|-------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|----------------|
| | 1 | 2 | 3 | 4 | 5 | |
| Strongly Disagree | ☐ | ☐ | ☐ | ☐ | ☐ | Strongly Agree |

Even now, I sometimes have trouble identifying specific moves or combos used by pro players in the middle of a fast-paced match. *

1 2 3 4 5
Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I have to re-watch match VODs, often at a slower speed, to understand a specific interaction or combo. *

1 2 3 4 5
Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I find that I learn more about the game from dedicated YouTube tutorials or guides than from watching live tournament streams. *

1 2 3 4 5
Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

The commentary during tournaments is sufficient for me to understand all the complex strategies and moves being used. *

1 2 3 4 5
Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

Section 3 of 4

Potential Solutions



Description (optional)

How useful would a real-time on-screen overlay be if it automatically identified the moves and combos being performed during a live match?



1

2

3

4

5

Not useful at all

☐☐☐☐☐

Extremely useful

Section 4 of 4

Open-Ended Feedback

Description (optional)

In your own words, what is the single biggest challenge for a new person trying to learn Tekken?

Short-answer text

Do you have any other ideas or suggestions for tools that could help new players learn the game or make tournaments more accessible for new viewers?

Long-answer text

Timestamp	How would you primarily describe yourself in the Tekken community?	How long have you been playing or following Tekken (or other traditional fighting games)?	How often do you watch high-level Tekken gameplay (tournaments, pro player streams, etc.)?	When I first started watching high-level Tekken, I found it difficult to understand what was happening on screen.
26/09/2025 04:34:23	Both a Player and a Viewer	More than 5 years	Rarely (a few times a year)	2
26/09/2025 04:50:16	Both a Player and a Viewer	Less than 6 months	Occasionally (a few times a month)	4
26/09/2025 04:50:35	Both a Player and a Viewer	Less than 6 months	Rarely (a few times a year)	5
26/09/2025 06:09:02	Both a Player and a Viewer	6 months to 2 years	Occasionally (a few times a month)	4
26/09/2025 07:33:33	Mainly a Player	6 months to 2 years	Daily	1
26/09/2025 07:48:51	Both a Player and a Viewer	More than 5 years	Occasionally (a few times a month)	4

26/09/2025 08:36:47	New and just getting interested	Less than 6 months	Frequently (a few times a week)	4
26/09/2025 08:55:55	Both a Player and a Viewer	2 to 5 years	Occasionally (a few times a month)	2
26/09/2025 10:09:56	New and just getting interested	Less than 6 months	Occasionally (a few times a month)	5
01/10/2025 19:38:59	New and just getting interested	Less than 6 months	Occasionally (a few times a month)	5
27/10/2025 01:29:16	Mainly a Player	More than 5 years	Rarely (a few times a year)	2
27/10/2025 01:37:08	Mainly a Player	More than 5 years	Rarely (a few times a year)	2
27/10/2025 01:39:34	Mainly a Player	6 months to 2 years	Rarely (a few times a year)	2
27/10/2025 02:05:02	Mainly a Viewer/Spectator	6 months to 2 years	Rarely (a few times a year)	5
27/10/2025 02:08:17	New and just getting interested	Less than 6 months	Rarely (a few times a year)	5
27/10/2025 02:30:55	New and just getting interested	6 months to 2 years	Rarely (a few times a year)	2
27/10/2025 02:36:47	Mainly a Player	2 to 5 years	Daily	5
27/10/2025 05:57:57	New and just getting interested	Less than 6 months	Rarely (a few times a year)	4
28/10/2025 01:27:56	Both a Player and a Viewer	6 months to 2 years	Daily	1
28/10/2025 01:31:14	Both a Player and a Viewer	2 to 5 years	Rarely (a few times a year)	1
30/10/2025 02:50:41	New and just getting interested	Less than 6 months	Rarely (a few times a year)	5

Even now, I sometimes have trouble identifying specific moves or combos used by pro players in the middle of a fast-paced match.	I have to re-watch match VODs, often at a slower speed, to understand a specific interaction or combo.	I find that I learn more about the game from dedicated YouTube tutorials or guides than from watching live tournament streams	The commentary during tournaments is sufficient for me to understand all the complex strategies and moves being used.
1	2	4	5
4	4	5	3
5	5	5	1
5	3	4	2
3	1	5	4

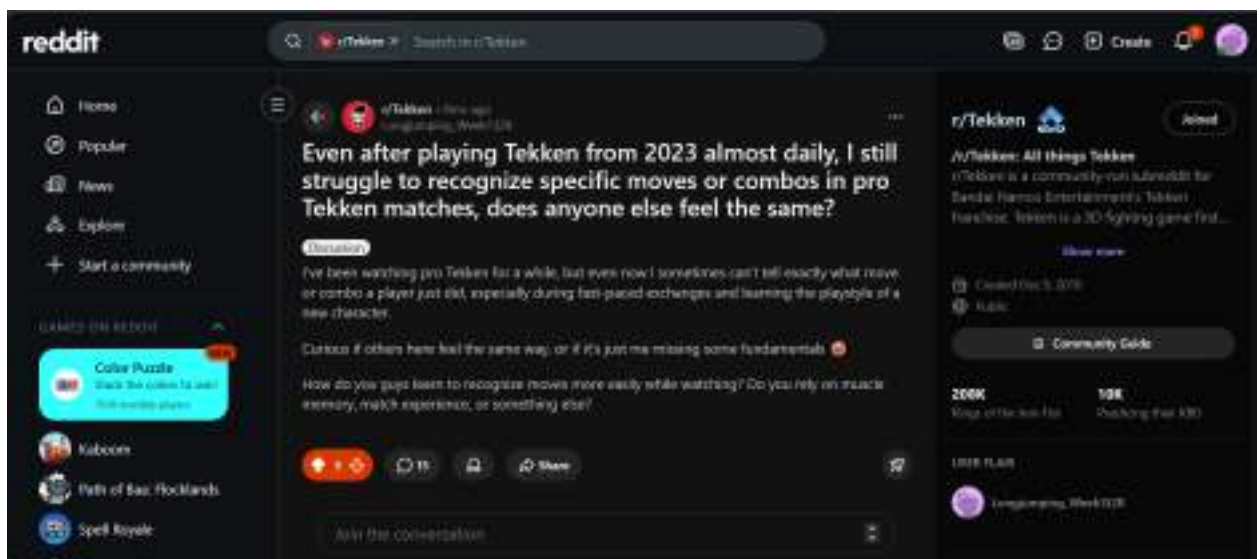
4	3	2	4
5	4	5	3
3	3	3	3
5	3	2	4
5	4	4	3
4	2	4	5
4	3	4	2
5	4	5	5
4	3	4	4
5	5	5	4
3	4	2	3
4	5	5	4
4	5	2	3
2	1	3	3
3	2	3	4
5	5	5	2

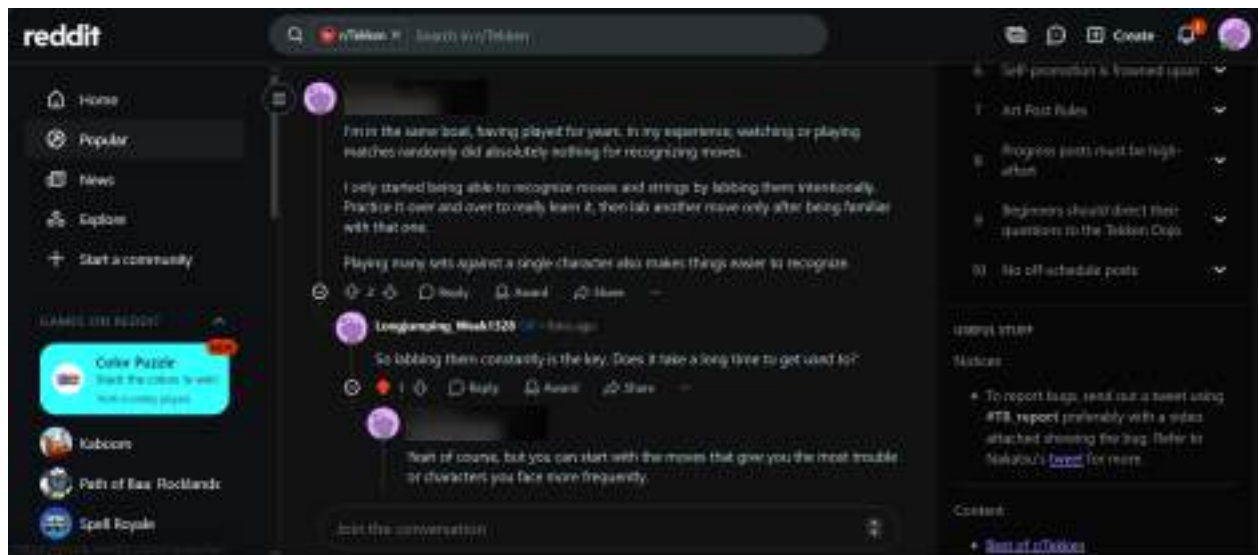
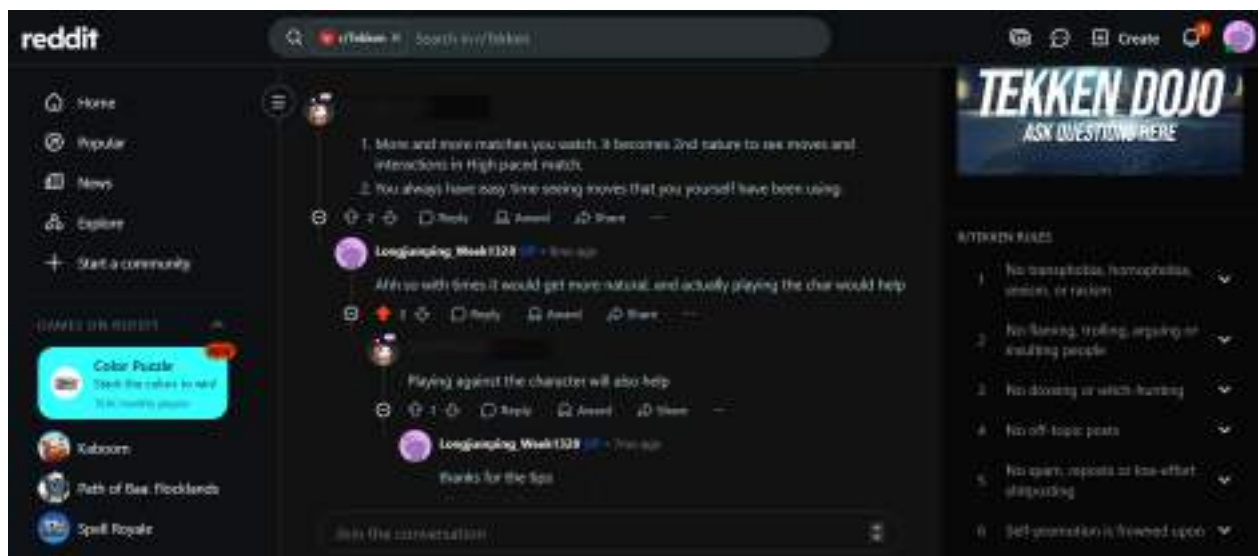
How useful would a real-time on-screen overlay be if it automatically identified the moves and combos being performed during a live match?	In your own words, what is the single biggest challenge for a new person trying to learn Tekken?	Do you have any other ideas or suggestions for tools that could help new players learn the game or make tournaments more accessible for new viewers?	What is your current Tekken rank (or closest equivalent)?
5	Movement	-	
4	For a new player, I think they would need to pick a certain character they like first. Then the biggest challenge is utilizing that character to the best and knowing how to do it efficiently.	Maybe also provide video how to use it in controller (actually footage of the combo) for improved understanding	
3	string of combo		
4	Understanding combos	timing the combos	2nd Dan

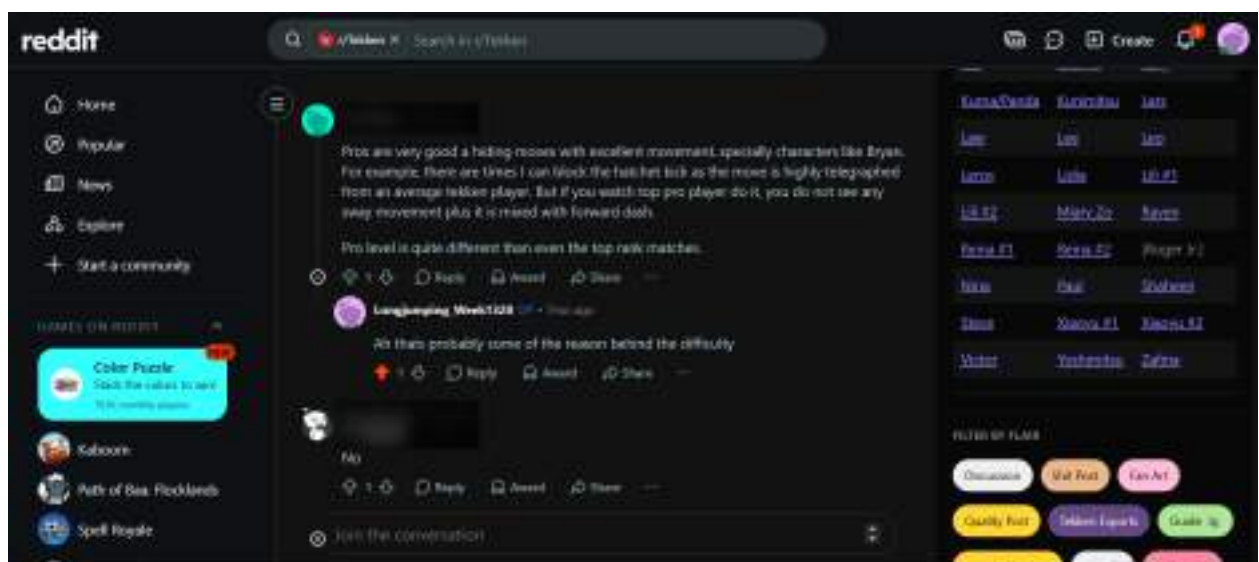
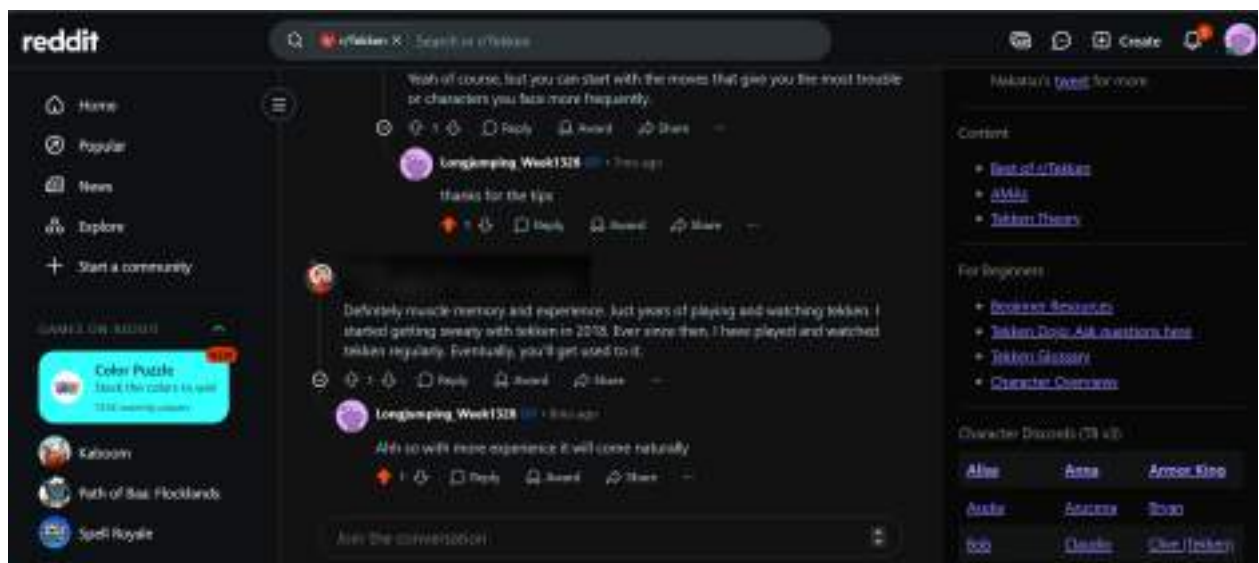
4	pewgf,tju,learn high damaging combos	just use the frame data	Expert – Blue, Purple/Gold (Fujin → Tekken God Supreme)
4	Reaction to character moves or throws	No, I would say it's good enough because Tekken 8 has "Punishment Training" (It was introduced in Tekken 7 Season 3 free update) and Frame Data (Also in Season 3 update, however it's paid feature) to learn it. Also, Tekken 8 has an update for Practice Mode in Season 2 called "Throw Break Training". It allows to practice player's reaction to throws and it has 4 difficulties (Easy, Medium, Hard and Ultra Hard).	I don't play ranked
4	learn various combos from one character and predict the enemy's next move	maybe we can see the movements and also the combos of these movements so we can learn better	I don't play ranked
3	learning matches	learn the basic attacks, then do the combos	Expert – Blue, Purple/Gold (Fujin → Tekken God Supreme)
5	Trying to remember the moveset of each character	The button and the combos on the screen would be really helpful for us as a new player/viewers to understand the moveset of the characters.	Beginner – Brown, Silver, Turquoise, Green
4	Personally my biggest challenge is building up my own game-sense		I don't play ranked
5	I think it's adjustment to the gameplay controls, especially making new choreographies to defeat your opponent in battle whether that is solo mode or multiplayer mode.	Not really any suggestions other than just have some fun for this game just like mortal kombat and street fighter too.	I don't play ranked
3	The combos	Combo tutorials / guidelines	I don't play ranked
5	Adapt to tekken mechanism	Play in practice mode	I don't play ranked
5	For me, The biggest challenge for a new person trying to learn tekken is the transition between combos are really hard for a new player and the complex button (ex R2+X+■)	You can make a simple button method like mortal combat, just use 2 buttons and you can execute combos automatically, however this tools only accessible for beginners match	I don't play ranked

5	how to ultimate the enemy		I don't play ranked
5	kompleksitas tinggi, kurva belajar curam, eksekusi gerakan yang sulit, minimnya sumber daya belajar, dan beratnya kompetisi online	tutorial interaktif, visualisasi frame data, pelatih kombo, analisis pertandingan, kontrol sederhana, fitur tontonan, komunitas, program mentor, konten edukasi, dan turnamen pemula.	Beginner – Brown, Silver, Turquoise, Green
5	Reading the combo, the annotation is so weird	we need something to tell us the move that they are doing, but i dont have any suggestion yet.	Expert – Blue, Purple/Gold (Fujin → Tekken God Supreme)
4	Trying to learn one character's moveset while also keeping note of the opponent's hands.	I think the proposed idea is already unique and brilliant. I look forward to seeing a demo!	I don't play ranked
3	Learn for punishment, move list, mix up+ frame trap, the last thing you would like to learn are combos. Use simple combos first then go for advance combo because advance combos require precision and timing especially for Bryan	Watch and learn, find the moveset on movelist, check the frame data so that you know when to use the move you learn. Always do practice on long distance wall to experimenting with many combos and moves.	Advanced – Red, Ruler tier (Garyu → Battle Ruler)
5	defense mechanism by not always pressing the controller		I don't play ranked
4	Frame data		I don't play ranked

Lampiran 12 Kuesioner reddit r/Tekken:







Lampiran 13 Logbook bimbingan

8/2/2026, 11:07 AM
alamat: ptefita.ac.id/LogbookMahasiswa/ - v.022026



**LOGBOOK MAHASISWA
TUGAS AKHIR**

NIM : 1210101010

NAMA : JULIAN YANG

PERIODE : 2025/2026 - GENAP

Tanggal	Topik pembahasan	Evaluasi	Approval
27/06/2026	Mendapat revisi untuk formatting draft - 20/06/2026 Membuat daftar table, daftar gambar, dan beberapa perubahan formatting lainnya untuk menyesuaikan dengan panduan	Bisa mendaftar sidang akhir Dr.Eng. Hardi Santoso, S.I., M.Eng	✓
27/06/2026	Penyelesaian draft sampai bab 5 dan pengecekan 23-6-2026 Menunjukkan hasil pipeline akhir, beserta draft untuk merevisi penulisan	Progress sesuai Dr.Eng. Hardi Santoso, S.I., M.Eng	✓
27/06/2026	Bimbingan terakhir untuk revisi - 27-06-2026 Melakukan pengecekan akhir pada draft dan mendapat berbagai feedback untuk meningkatkan draft yang ada	Progress sesuai Dr.Eng. Hardi Santoso, S.I., M.Eng	✓
27/06/2026	Mengganti judul skripsi ke dalam bentuk yang di scope down - 24-06-2026 Setelah scoping down ke practice mode dari target awal untuk real time pada game tekken 8, judul diupdate	Progress sesuai Dr.Eng. Hardi Santoso, S.I., M.Eng	✓
27/06/2026	Strukturalisasi Teknis & Formulasi Metodologi - 23/05/2026 • Memindahkan draf motivasi ke kolom Tujuan (fokus meningkatkan skill pemula). • Menyusun Rumusan Masalah (Poin 1). Perancangan pipeline; Poin 2: Implementasi/pengujian). • Mengunci Machine Learning Ops (MLOps) sebagai kerangka metode besar.	Progress sesuai Dr.Eng. Hardi Santoso, S.I., M.Eng	✓
27/06/2026	Penguatan Fondasi Literatur & Latar Belakang - 15/05/2026 • Studi literatur buku (memenuhi batas minimal). • Pencarian jurnal ilmiah ber-ISSN. • Riset metode tracking eksisting (ByteTrack). • Menyusun draf Latar Belakang (Populansi Tekken & gap pemula vs pro player).	Progress sesuai Dr.Eng. Hardi Santoso, S.I., M.Eng	✓

https://logbook.ptefita.ac.id/LogbookMahasiswa/ - v.022026
102

Tanggal	Topik pembahasan	Evaluasi	Approval
27/06/2026	Percobaan menggunakan model dengan dasar DETR untuk object detection, serta menggunakan OC SORT untuk tracker baru - 19 April 2026 1. Disarankan model object detection baru (RF-DETR) dan object tracking baru (OC-SORT) 2. Diajarkan pas sidang bakal ditanya beberapa tipe pertanyaan (harus paham struktur model luar dalam, dan proses detailnya)	Progress sesuai Dr.Eng. Handi Santoso, S.S., M.Eng.	✓
27/06/2026	Melakukan update pada draft sempro - 11 April 2026 1. Di bagian bab 3, execution ganti ke namanya model training, implementation ke deployment 2. Evaluasi digabung ke deployment, deployment dibesarkan dan beri garis penghubung seperti loop	Progress sesuai Dr.Eng. Handi Santoso, S.S., M.Eng.	✓



Dosen Koordinator TUGAS
AKHIR
(Enik Daudi, S.Kom, M.Kom)

Dosen Pembimbing
(Dr.Eng. Handi Santoso, S.S., M.Eng.)

